Lecture Notes for Category Theory

Niels van der Weide (vanderweide@inf.elte.hu)

Contents

Preface **4**

1 Categories **5**
1.1 Categories and Examples 5
1.2 Special Morphisms 14
1.3 Exercises 18

2 Constructions in Categories **20**
2.1 Terminal and Initial Objects 20
2.2 Binary Products and Coproducts 22
2.3 Pullbacks 28
2.4 Exercises 30

3 Functors and Natural Transformations **32**
3.1 Functors 32
3.2 Natural Transformations 37
3.3 Categories of Functors 40
3.4 Exercises 43

4 Algebras and Coalgebras **44**
4.1 Algebras for a Functor 44
4.2 Coalgebras for a Functor 51
4.3 Exercises 53

5 Adjunctions **54**
5.1 Adjunctions and Examples 54
5.2 Adjunctions via Units and Counits 59
5.3 Adjunctions and Limits 61
5.4 Exercises 62

6 Monads **63**
6.1 Monads and Examples 63
6.2 The Kleisli Category 69
6.3 Exercises 72

7 The Yoneda Lemma **73**
7.1 Presheaves 73
7.2 Exercises 74

8 Heyting Algebras **75**
8.1 Heyting Algebras 75
8.2 Exercises 80

9 Hyperdoctrines **81**
9.1 Hyperdoctrines 81
9.2 Quantifiers as Adjoints 81

A First-Order Predicate Logic **82**
A.1 Stuctural Rules 84
A.2 Truth Formula 84
A.3 Falsity formula 85
A.4 Conjunction 85
A.5 Disjunction 85
A.6 Implication 86
A.7 Universal Quantification 86
A.8 Existential Quantification 87
A.9 Equality 87

Preface

These lecture notes are for the course "Category Theory" taught in the Autumn semester of 2026-2027 at Eötvös Loránd University. You can find the course information here:

`https://nmvdw.github.io/education/ct-autumn-2026.html`

These notes consist of two parts. The first sections (Sections 2 to 7) serve as a general introduction to category theory. In this part, we discuss various core topics that form the basis of category theory. In the later sections (Sections 8 and 9), we look at a more specialised topic. Specifically, we study the applications of category theory to logic, and we study categorical structures that can be used to model various flavours of logic.

Throughout these lecture notes, one can find various digressions. Students are not expected to comprehend these digressions. They contain extra information for those who are interested in these topics, and they are suggestions for topics that one could study further.

Foundations. We use set theoretic foudnations throughout these notes, although we do not formally use any system of axioms for set theory. For instance, when we talk about equality of sets, we mean that they have the same elements. When we talk about the equality of functions $f, g : X \to Y$, we mean that we have $f(x) = g(x)$ for every $x \in X$. We also frequently talk about unique existence. We say that there exists a unique $x \in X$ for which some property P holds whenever we have $x \in X$ with $P(x)$ and if we have $x = y$ for all $x, y \in X$ such that $P(x)$ and $P(y)$.

Acknowledgements. I would like to thank Janet/Zhenyun Yin, Wenhan Mao, and Botond Mészáros for reading these notes and for their feedback.

1 Categories

1.1 Categories and Examples

Definition 1.1. A **category** $\mathcal{C}$ consists of

- a collection $\mathcal{C}_0$ of **objects**;
- for all objects $x, y \in \mathcal{C}_0$, a collection $\mathcal{C}_1(x, y)$ of **morphisms**;
- for all objects $x \in \mathcal{C}_0$, a morphism $\mathsf{id}_x : \mathcal{C}_1(x, x)$ called the **identity morphism**;
- for all objects $x, y, z \in \mathcal{C}_0$ and morphisms $f : \mathcal{C}_1(x, y)$ and $g : \mathcal{C}_1(y, z)$, a morphism $g \circ_{\mathcal{C}} f : \mathcal{C}_1(x, z)$ called the **composition**,

such that the following equations holds

- for all objects $x, y \in \mathcal{C}_0$ and morphisms $f : \mathcal{C}_1(x, y)$, we have $\mathsf{id}_y \circ_{\mathcal{C}} f = f$;
- for all objects $x, y \in \mathcal{C}_0$ and morphisms $f : \mathcal{C}_1(x, y)$, we have $f \circ_{\mathcal{C}} \mathsf{id}_x = f$;
- for all objects $w, x, y, z \in \mathcal{C}_0$ and morphisms $f : \mathcal{C}_1(w, x)$, $g : \mathcal{C}_1(x, y)$, and $h : \mathcal{C}_1(y, z)$, we have $h \circ_{\mathcal{C}} (g \circ_{\mathcal{C}} f) = (h \circ_{\mathcal{C}} g) \circ_{\mathcal{C}} f$.

The equations $\mathsf{id}_y \circ_{\mathcal{C}} f = f$ and $f \circ_{\mathcal{C}} \mathsf{id}_x = f$ say that the morphism id is a unit element for composition, and we call them the **left and right unitality laws**. The equation $h \circ_{\mathcal{C}} (g \circ_{\mathcal{C}} f) = (h \circ_{\mathcal{C}} g) \circ_{\mathcal{C}} f$ says that composition is **associative**, and this equation makes $f \circ_{\mathcal{C}} g \circ_{\mathcal{C}} h$ unambiguous.

To lighten the notation, we often write $x \in \mathcal{C}$ instead of $x \in \mathcal{C}_0$. We also write $f : x \to_{\mathcal{C}} y$ instead of $f \in \mathcal{C}_1(x, y)$, and if $\mathcal{C}$ is clear from the context, we write $f : x \to y$ instead. In addition, we often write id instead of id_x if x is clear from the context, and we write $f \circ g$ for $f \circ_{\mathcal{C}} g$ if $\mathcal{C}$ is clear from the context.

Digression. In Definition 1.1 it is necessary to distinguish sets and classes. Such a distinction is necessary to define, for instance, the category of sets (Example 1.2). The objects of this category should be all sets, but as there is no sets of all sets, such a category cannot exist if we require the objects of a category to form a set. We solve this problem by relaxing the definition of category to allow the objects and morphisms to form a class rather than a set. Another solution would be restrict this category to all set whose cardinality is at most κ for some cardinal number κ. However, there is a disadvantage to this solution: we need κ to be an inaccessible cardinal in order to show that the resulting category of sets has desirable properties (e.g., Cartesian closed). Throughout these notes, we disregard issues regarding sets versus classes, and we use the word "collection" instead.

Let us discuss some examples of categories. We start by examples of categories that arise from mathematical structures. In Table 1, we summarise the examples of categories that we discuss throughout this section.

Example 1.2. We define the **category of sets**, written as Set, as follows

- the objects of Set are sets;
- the morphisms from a set X to a set Y are functions from X to Y.

The identity morphism $\mathsf{id} : X \to X$ is defined to be the identity function, i.e., $\mathsf{id}(x) = x$. The composition of morphisms $f : X \to Y$ and $g : Y \to Z$ is defined to be the composition of functions, i.e., $(g \circ f)(x) = g(f(x))$. Let $f : X \to Y$ be a morphism. Left unitality ($\mathsf{id} \circ f = f$) follows from the following equations, where $x \in X$ is an arbitrary element.

$$\begin{aligned}(\mathsf{id} \circ f)(x) &= \mathsf{id}(f(x)) \\ &= f(x)\end{aligned}$$

Right unitality ($f \circ \mathsf{id} = f$) is proven similarly.

$$\begin{aligned}(f \circ \mathsf{id})(x) &= f(\mathsf{id}(x)) \\ &= f(x)\end{aligned}$$

Now suppose we have morphisms $f : W \to X$, $g : X \to Y$, and $h : Y \to Z$. To prove associativity, we take an arbitrary $w \in W$, and we use the following equalities.

$$\begin{aligned}(h \circ (g \circ f))(w) &= h((g \circ f)(w)) \\ &= h(g(f(w))) \\ &= (h \circ g)(f(w)) \\ &= ((h \circ g) \circ f)(w)\end{aligned}$$

Hence, Set indeed forms a category.

Example 1.3. A **monoid** consists of a set M together an element $e_M \in M$ and an operation $m : M \times M \to M$ such that the following equations holds

- $m(e_M, x) = x$ for all $x \in M$;
- $m(x, e_M) = x$ for all $x \in M$;
- $m(x, m(y, z)) = m(m(x, y), z)$ for all $x, y, z \in M$.

We denote $m(x, y)$ by $x \cdot y$, and we denote a monoid by M. A **homomorphism** from a monoid M to a monoid N is a function $f : M \to N$ such that $f(e_M) = e_N$ and $f(x \cdot y) = f(x) \cdot f(y)$. The identity function $\mathsf{id} : M \to M$ is a homomorphism, because

$$\mathsf{id}(e_M) = e_M, \qquad \mathsf{id}(x \cdot y) = x \cdot y = \mathsf{id}(x) \cdot \mathsf{id}(y)$$

If $f : M \to N$ and $g : N \to K$ are homomorphisms, then their composition is a homomorphism as well since

$$(g \circ f)(e_M) = g(f(e_M)) = g(e_N) = e_K$$

$$(g \circ f)(x \cdot y) = g(f(x \cdot y)) = g(f(x) \cdot f(y)) = g(f(x)) \cdot g(f(y))$$

Note that two homorphisms $f, g : M \to N$ are the same if their underlying functions are the same, meaning that to prove $f = g$, it suffices to show that $f(m) = g(m)$ for all $m \in M$.

Now we define the **category of monoids**, writen Mon, as follows.

- the objects of Mon are monoids;
- the morphisms from a monoid M to a monoid N are homomorphisms from M to N.

The identity $\mathsf{id} : M \to M$ is given by the identity homomorphism, and the composition is given by the composition of homomorphisms. We prove that Mon is a category in the same way as we prove that Set is a category. To prove left unitality ($\mathsf{id} \circ f = f$ for all homomorphisms f), we take an arbitrary $m \in M$, and we note

$$\begin{aligned}(\mathsf{id} \circ f)(m) &= \mathsf{id}(f(m)) \\ &= f(m)\end{aligned}$$

Example 1.4. A **group** G consists of a set G together an element $e_G \in G$ and operations $i : G \to G$ and $m : G \times G \to G$ such that the following equations holds

- $m(e_G, g) = g$ for all $g \in G$;
- $m(g, e_G) = g$ for all $g \in G$;
- $m(g, m(h, k)) = m(m(g, h), k)$ for all $g, h, k \in G$;
- $m(g, i(g)) = e_G$ for all $g \in G$;
- $m(i(g), g) = e_G$ for all $g \in G$.

We denote $m(g, h)$ by $g \cdot h$, and we denote $i(g)$ by g^{-1}. A **homomorphism** from a group G to a group H is a function $f : G \to H$ such that $f(e_G) = e_H$ and $f(g \cdot h) = f(g) \cdot f(h)$. Note that we can prove $f(g^{-1}) = (f(g))^{-1}$.

We define the **category of groups**, written Grp, as follows

- the objects of Grp are groups;
- the morphisms from a group G to a group H are homomorphisms from G to H.

We prove that Grp is a category the same way as we proved that Mon is a category.

Another example of a category is the category PreOrd (Exercise 1.4). Next we look at constructions of categories.

Example 1.5. We write $\mathbb{1}$ for the set $\{*\}$. We define the **unit category**, denoted by unit, as follows.

- The objects of unit are elements of $\mathbb{1}$;
- The morphisms from $*$ to $*$ are elements of $\mathbb{1}$

The identity morphism is given by $*$. To define the composition, we assume that we have two morphisms $f, g : * \to_{\mathsf{unit}} *$. Both f and g are elements of $\mathbb{1}$, and thus they are equal to $*$, and we define their composition to be $*$. The necessary laws follow directly, because all elements of $\mathbb{1}$ are equal, and hence all morphisms in unit are equal.

We can visualise the unit category as follows.



We have a single object, $*$, and we have a morphism id_* from $*$ to $*$, which also is defined to be $*$.

Example 1.6. Let $\mathcal{C}$ be a category. We define **opposite category**, denoted by $\mathcal{C}^{\mathsf{op}}$, as follows.

- The objects of $\mathcal{C}^{\mathsf{op}}$ are objects of $\mathcal{C}$;
- The morphisms from x to y in $\mathcal{C}^{\mathsf{op}}$ are morphisms from y to x in $\mathcal{C}$.

Hence, $\mathcal{C}^{\mathsf{op}}$ is $\mathcal{C}$ but with the morphisms reversed. The identity morphism $\mathsf{id}_x : \mathcal{C}_1^{\mathsf{op}}(x, x)$ in $\mathcal{C}^{\mathsf{op}}$ is the identity morphism $\mathsf{id}_x : \mathcal{C}_1(x, x)$ in $\mathcal{C}$. Suppose that we have morphisms $f : \mathcal{C}_1^{\mathsf{op}}(x, y)$ and $g : \mathcal{C}_1^{\mathsf{op}}(y, z)$, which are the same as morphisms $f : \mathcal{C}_1(y, x)$ and $g : \mathcal{C}_1(z, y)$. We define the composition $g \circ_{\mathcal{C}^{\mathsf{op}}} f$ to be $f \circ_{\mathcal{C}} g$. Left unitality is proven as follows

$$\begin{aligned} \mathsf{id} \circ_{\mathcal{C}^{\mathsf{op}}} f &= f \circ_{\mathcal{C}} \mathsf{id} \\ &= f \end{aligned}$$

Here we use right unitality of $\mathcal{C}$. Right unitality for $\mathcal{C}^{\mathsf{op}}$ is left as an exercise. To prove associativity for $\mathcal{C}^{\mathsf{op}}$, we assume that we have morphisms $f : \mathcal{C}_1^{\mathsf{op}}(w, x)$, $g : \mathcal{C}_1^{\mathsf{op}}(x, y)$, and $h : \mathcal{C}_1^{\mathsf{op}}(y, z)$. Note that we have the following equalities.

$$\begin{aligned} h \circ_{\mathcal{C}^{\mathsf{op}}} (g \circ_{\mathcal{C}^{\mathsf{op}}} f) &= (g \circ_{\mathcal{C}^{\mathsf{op}}} f) \circ_{\mathcal{C}} h \\ &= (f \circ_{\mathcal{C}} g) \circ_{\mathcal{C}} h \\ &= f \circ_{\mathcal{C}} (g \circ_{\mathcal{C}} h) \\ &= (g \circ_{\mathcal{C}} h) \circ_{\mathcal{C}^{\mathsf{op}}} f \\ &= (h \circ_{\mathcal{C}^{\mathsf{op}}} g) \circ_{\mathcal{C}^{\mathsf{op}}} f \end{aligned}$$

The compositions above make sense, because $f : \mathcal{C}_1^{\mathsf{op}}(w, x)$, $g : \mathcal{C}_1^{\mathsf{op}}(x, y)$, and $h : \mathcal{C}_1^{\mathsf{op}}(y, z)$ are the same as morphisms $f : \mathcal{C}_1(x, w)$, $g : \mathcal{C}_1(y, x)$, and $h : \mathcal{C}_1(z, y)$ in $\mathcal{C}$.

In the next example, we use the product of sets. If we have sets X and Y, then their product $X \times Y$ is the set $\{(x, y) \mid x \in X, y \in Y\}$. We call the elements of $X \times Y$ pairs.

Example 1.7. Suppose that we have categories $\mathcal{C}_1$ and $\mathcal{C}_2$. We define the **product category**, denoted by $\mathcal{C}_1 \times \mathcal{C}_2$, as follows.

- The objects of $\mathcal{C}_1 \times \mathcal{C}_2$ are pairs (x, y) where $x \in \mathcal{C}_1$ and $y \in \mathcal{C}_2$;
- The morphisms from (x_1, y_1) to (x_2, y_2) in $\mathcal{C}_1 \times \mathcal{C}_2$ are pairs (f, g) of morphisms where $f : x_1 \to_{\mathcal{C}_1} x_2$ and $g : y_1 \to_{\mathcal{C}_2} y_2$.

We leave it as an exercise to construct the identity morphisms and the composition of morphisms, and to prove that it indeed forms a category.

Example 1.8. Let $\mathcal{C}$ be a category and let P be a predicate on the objects of $\mathcal{C}$. We define the **full subcategory** $\mathsf{Full}(\mathcal{C}, \mathsf{P})$ as follows.

- The objects of $\mathsf{Full}(\mathcal{C}, \mathsf{P})$ are objects of $x \in \mathcal{C}$ such that $\mathsf{P}(x)$
- Given objects $x, y \in C$ such that $\mathsf{P}(x)$ and $\mathsf{P}(y)$, then morphisms from x to y in $\mathsf{Full}(\mathcal{C}, \mathsf{P})$ are the same as morphisms $x \to_{\mathcal{C}} y$.

The identity morphisms and composition operations in $\mathsf{Full}(\mathcal{C}, \mathsf{P})$ are inherited from $\mathcal{C}$, and we leave it as an exercise to the reader to verify that we indeed have a category.

Next we show that categories generalize certain mathematical structures. Specifically, we show how to construct categories from other mathematical structures.

Example 1.9. A **preorder** consists of a set X together with a relation $\leq$ on X that is reflexive ($x \leq x$ for each x) and transitive (if $x \leq y$ and $y \leq z$, then $x \leq z$). Every preorder $(X, \leq)$ gives rise to a category $\widetilde{(X, \leq)}$ as follows.

- The objects of $\widetilde{(X, \leq)}$ are elements of X;
- There is a unique morphism from $x \in X$ to $y \in X$ if and only if $x \leq y$.

Since $\leq$ is reflexive, we have an identity morphism. To construct the composition of morphisms in $\widetilde{(X, \leq)}$, we use that $\leq$ is transitive. The desired laws follow from the fact that morphisms in $\widetilde{(X, \leq)}$ are unique.

Example 1.10. Let M be a monoid. We define the category $\mathsf{B}(M)$ as follows.

- The set of objects of $\mathsf{B}(M)$ is $\mathbb{1}$;
- morphisms from $*$ to $*$ are the same as elements of M

The identity morphism is given by e_M, and if we have morphisms $m_1, m_2 : * \to *$, then their composition $m_1 \circ_{\mathsf{B}(M)} m_2$ is given by $m_1 \cdot m_2$. The laws are proven as follows.

$$m \circ_{\mathsf{B}(M)} \mathsf{id} = m \cdot e_M = m$$

$$\mathsf{id} \circ_{\mathsf{B}(M)} m = e_M \cdot m = m$$

$$m_1 \circ_{\mathsf{B}(M)} (m_2 \circ_{\mathsf{B}(M)} m_3) = m_1 \cdot (m_2 \cdot m_3) = (m_1 \cdot m_2) \cdot m_3 = (m_1 \circ_{\mathsf{B}(M)} m_2) \circ_{\mathsf{B}(M)} m_3$$

Since every group is in particular a monoid, we can instantiate Example 1.10 to groups as well. We can visualise $\mathsf{B}(M)$ as follows.



We have a single object $*$, and for each element $m \in M$ we have a morphism $m : * \to *$. The identity and composition is determined by the monoid structure of M.

Digression. Another interesting example comes from the **λ-calculus**. Recall that the terms of the λ-calculus are defined using the following grammar

$$\Lambda := x \mid \Lambda\Lambda \mid \lambda x.\Lambda$$

Arbitrary terms are denoted by M. Types in the λ-calculus are defined using the following grammar.

$$\mathsf{Ty} := \iota \mid \mathsf{Ty} \Rightarrow \mathsf{Ty}$$

Here ι is the base type, and we denote arbitrary types by τ and σ. The typing relation $x_1 : \tau_1, \ldots, x_n : \tau_n \vdash M : \sigma$ is defined using the usual derivation rules We define the **syntactic category of the simply typed λ-calculus**, denoted by STLC, as follows.

- The objects of STLC are types
- The morphisms from τ to σ are equivalence classes of terms t such that $x : \tau \vdash t : \sigma$, where terms are identified using $\beta\eta$-conversion.

The identity morphism from τ to τ is given by the variable $x : \tau \vdash x : \tau$, and if we have terms $x : \tau_1 \vdash s : \tau_2$ and $y : \tau_2 \vdash t : \tau_3$, then their composition is $x : \tau_1 \vdash t[y \mapsto s] : \tau_3$.

This example is inspiring: we can turn various typed languages into categories. For instance, we can extend this example to include various types, such as sum-types and product types. One might wonder: can we do the same for Haskell? Can we make a category whose objects are types (as in Haskell) and whose morphisms are Haskell functions. Doing so comes with various complications. Since Haskell's type system includes polymorphism, we need to take into account that Haskell types could have type variables. However, there is an

even more fundamental issue. Haskell has a function $\mathsf{seq} : a \to b \to b$ that evaluates its first argument before returning its second, and we also have a function $\mathsf{undefined} : a \to b$ that is undefined for each argument. Now we can show the following

$$\mathsf{seq}\ \mathsf{undefined}\ () = \mathsf{undefined}$$

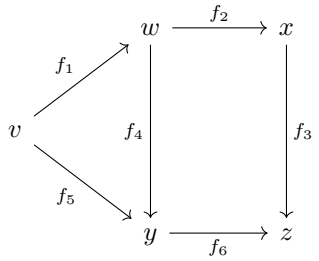
$$\mathsf{seq}\ (\mathsf{undefined} \circ \mathsf{id})() = ()$$

Note that $\mathsf{undefined} \circ \mathsf{id}$ evaluates to itself, which is why the second equation holds. However, from these equations we can conclude that $\mathsf{undefined}$ and $\mathsf{undefined} \circ \mathsf{id}$ cannot be equal, and thus the category laws do not hold.

When we talk about categories, we generally use the language of **diagrams**. A diagram in a category $\mathcal{C}$ is given by a graph whose nodes are objects in $\mathcal{C}$ and whose edges from x to y are labelled by a morphism $f : x \to_{\mathcal{C}} y$. For instance, suppose that we have a category $\mathcal{C}$ and let v, w, x, y, z be objects in $\mathcal{C}$. Suppose that we have morphisms

$$f_1 : v \to w, \quad f_2 : w \to x, \quad f_3 : x \to z, \quad f_4 : w \to y, \quad f_5 : v \to y, \quad f_6 : y \to z$$

An example of a diagram is given below

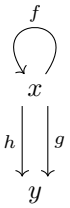


Each path in this graph gives rise to a composition of morphisms. For instance, we have two paths from v to y, namely $f_4 \circ f_1$ and f_5. We have three paths from v to z, which are given by the following compositions

$$f_3 \circ f_2 \circ f_1, \quad f_6 \circ f_4 \circ f_1, \quad f_6 \circ f_5$$

Since composition is associative, every path describes a unique morphism. A diagram is said to be **commutative** if for all nodes x and y every path from x to y denotes the same morphism.

In these diagrams, one is allowed to add loops and to have multiple edges between the same vertices. For instance, the following is a diagram.

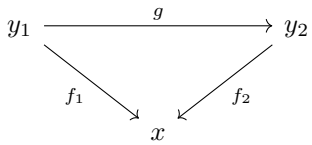


Note: if we say such a diagram commutes, we require, in particular, that $g = h$[1].

To get familiar with the language of diagrams, we look at other examples of categories. These examples are more abstract in nature than the ones we discussed before, but they provide a nice opportunity to familiarise oneself with the language of commutative diagrams.

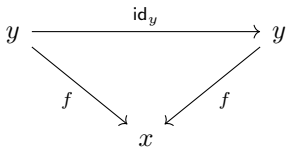
Example 1.11. Let $\mathcal{C}$ be a category and let x be an object in $\mathcal{C}$. We define the **slice category** $\mathcal{C}/x$ as follows.

- The objects of $\mathcal{C}/x$ are pairs of an object $y \in Y$ together with a morphism $f : y \to x$.
- Given objects $y_1, y_2 \in \mathcal{C}$ and morphisms $f_1 : y_1 \to x$ and $f_2 : y_2 \to x$, then morphisms from (y_1, f_1) to (y_2, f_2) in $\mathcal{C}/x$ are morphisms $g : y_1 \to y_2$ such that the following diagram commutes.



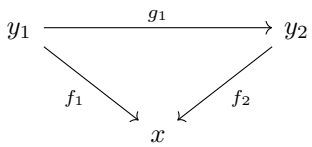
  Concretely, this means that $f_1 = f_2 \circ g$.

The identity morphism on (y, f) is given by id_y. Note that id_y indeed gives a morphism in $\mathcal{C}/x$ since the following diagram commutes



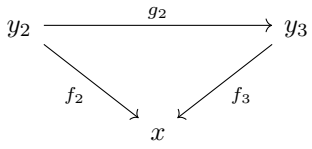
Concretely, we have to show that $f = f \circ \mathsf{id}_y$, which holds because id is the unit for composition.

Now suppose that we have three objects in $\mathcal{C}/x$, which are given by objects $y_1, y_2, y_3 \in \mathcal{C}$ together with morphisms $f_1 : y_1 \to x$, $f_2 : y_2 \to x$, and $f_3 : y_3 \to x$. We also assume that we have morphisms $g_1 : (y_1, f_1) \to_{\mathcal{C}/x} (y_2, f_2)$ and $g_2 : (y_2, f_2) \to_{\mathcal{C}/x} (y_3, f_3)$. This means that we have morphisms $g_1 : y_1 \to_{\mathcal{C}} y_2$ making the following triangle commute.

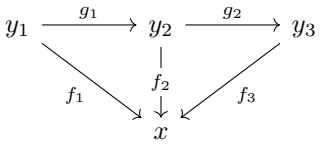


[1]Be aware of the fact that there are different conventions regarding commutativity across the literature

Similarly, we have $g_2 : y_2 \to_{\mathcal{C}} y_3$ making the following triangle commute.



Their composition in $\mathcal{C}/x$ is given by $g_2 \circ g_1$. We need to show that the following diagram commutes.



This diagram commutes, because both triangles commute.

We can make this reasoning concrete by presenting it in an equational style. We need to show that $f_1 = f_3 \circ (g_2 \circ g_1)$.

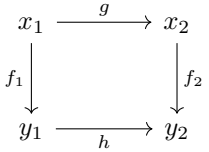
$$\begin{aligned} f_1 &= f_2 \circ g_1 && \text{since } g_1 \text{ is a morphism in } \mathcal{C}/x \\ &= (f_3 \circ g_2) \circ g_1 && \text{since } g_2 \text{ is a morphism in } \mathcal{C}/x \\ &= f_3 \circ (g_2 \circ g_1) && \text{associativity} \end{aligned}$$

Hence, $g_2 \circ g_1$ satisfies the desired requirement.

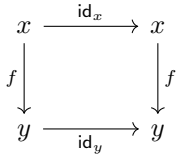
We leave it to the reader to verify the category laws.

Example 1.12. Let $\mathcal{C}$ be a category. We define the **arrow category** $\mathcal{C}^{\to}$ as follows.

- The objects of $\mathcal{C}^{\to}$ are triples of objects $x, y \in \mathcal{C}$ together with a morphism $f : x \to y$. We denote such objects by $f : x \to y$.
- Given morphisms $f_1 : x_1 \to y_1$ and $f_2 : x_2 \to y_2$, morphisms $f_1 \to_{\mathcal{C}^{\to}} f_2$ are pairs of morphisms $g : x_1 \to x_2$ and $h : y_1 \to y_2$ making the following square commute.



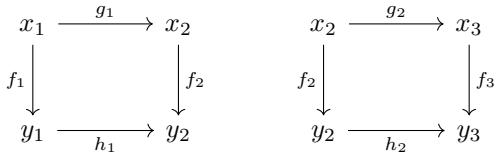
The identity morphism is given by the following square



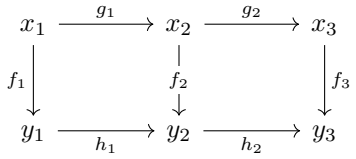
Category	Notation	Objects	Morphisms
Category of sets	Set	Sets	Functions
Category of monoids	Mon	Monoids	Homomorphisms
Category of groups	Grp	Groups	Homomorphisms
Unit category	unit	Single element	Single morphism
Opposite category	$\mathcal{C}^{\mathsf{op}}$	Objects in $\mathcal{C}$	Morphisms in $\mathcal{C}$ (reversed)
Product category	$\mathcal{C}_1 \times \mathcal{C}_2$	Pairs of objects	Pairs of morphisms
Full subcategory	$\mathsf{Full}(\mathcal{C}, \mathsf{P})$	Objects in $\mathcal{C}$	Morphisms in $\mathcal{C}$
Slice category	$\mathcal{C}/x$	Pairs of $y \in \mathcal{C}$ with $f : y \to x$	Commutative triangles
Arrow category	$\mathcal{C}^{\to}$	Triples of $x, y \in \mathcal{C}$ and $f : x \to y$	Commutative squares
Preorder	$\widetilde{(X, \leq)}$	Elements in X	Given by $\leq$
Monoids	$\mathsf{B}(M)$	Single element	Elements of M

Table 1: Examples of categories

To define the composition of morphisms, we assume that we have two commutative squares as indicated below.



Their composition is defined as the following square.



We leave it as an exercise to the reader to verify that composition and identity indeed give rise to morphisms and to verify the category laws.

1.2 Special Morphisms

Definition 1.13. Let $\mathcal{C}$ be a category, and let $x, y : \mathcal{C}$ be objects. We say that $f : x \to y$ is a **monomorphism** if for all objects $w : \mathcal{C}$ and morphisms $g_1, g_2 : w \to x$ such that $f \circ g_1 = f \circ g_2$, we have $g_1 = g_2$.

Note that the notion of monomorphism says that we can cancel f is it appears on the left of a composition. If we have $f \circ g_1 = f \circ g_2$, then cancelling f gives us $g_1 = g_2$, which is exactly what monomorphisms allow us to do.

Definition 1.14. Let $\mathcal{C}$ be a category, and let $x, y : \mathcal{C}$ be objects. We say that $f : x \to y$ is a **epimorphism** if for all objects $z : \mathcal{C}$ and morphisms $g_1, g_2 : y \to z$ such that $g_1 \circ f = g_2 \circ f$, we have $g_1 = g_2$.

The notion of epimorphism is similar to that of a monomorphism, except that it allows us to cancel morphisms that appear on the right of a composition: If we have $g_1 \circ f = g_2 \circ f$, then cancelling f gives us $g_1 = g_2$, which is exactly what epimorphisms allow us to do.

Definition 1.15. Let $\mathcal{C}$ be a category, and let $x, y : \mathcal{C}$ be objects. We say that $f : x \to y$ is an **isomorphism** if there is a morphism $g : y \to x$ such that $f \circ g = \mathsf{id}_y$ and $g \circ f = \mathsf{id}_x$. The collection of isomorphisms is denoted by $x \cong y$.

To get a better understanding of these classes of morphisms, we study them for concrete instances of categories. First, we look at Set.

Proposition 1.16. *A function $f : X \to Y$ of sets is injective if and only if it is a monomorphism in the category Set.*

Proof. Suppose that $f : X \to Y$ is injective, and suppose that we have functions $g_1, g_2 : W \to X$ such that $f \circ g_1 = f \circ g_2$. To show that $g_1 = g_2$, we take $w \in W$, and we must show that $g_1(w) = g_2(w)$. Since f is injective, it suffices to show $f(g_1(w)) = f(g_2(w))$, which holds by the assumption that $f \circ g_1 = f \circ g_2$.

Next we assume that $f : X \to Y$ is a monomorphism, and we show that it is injective. Suppose that we have $x_1, x_2 \in X$ such that $f(x_1) = f(x_2)$. Note that from x_1, x_2 we get morphisms $\overline{x_1}, \overline{x_2} : \mathbb{1} \to X$ (recall that $\mathbb{1} = \{*\}$):

$$\overline{x_1}(*) = x_1, \quad \overline{x_2}(*) = x_2$$

Note that we have that $f \circ \overline{x_1} = f \circ \overline{x_2}$, since

$$\begin{aligned}(f \circ \overline{x_1})(*) &= f(\overline{x_1}(*)) \\ &= f(x_1) \\ &= f(x_2) \\ &= f(\overline{x_2}(*)) \\ &= (f \circ \overline{x_2})(*)\end{aligned}$$

Since f is a monomorphism, we must thus have $\overline{x_1} = \overline{x_2}$. Hence,

$$x_1 = \overline{x_1}(*) = \overline{x_2}(*) = x_2$$ □

Proposition 1.17. *A function $f : X \to Y$ of sets is surjective if and only if it is an epimorphism in the category Set.*

Proof. Suppose that $f : X \to Y$ is surjective, and suppose that we have functions $g_1, g_2 : Y \to Z$ such that $g_1 \circ f = g_2 \circ f$. We must show that $g_1 = g_2$, so we take an arbitrary $y \in Y$, and we must show $g_1(y) = g_2(y)$. Since f is surjective, we can find $x \in X$ such that $f(x) = y$. We can thus conclude that

$$\begin{aligned}g_1(y) &= g_1(f(x)) \\ &= g_2(f(x)) \\ &= g_2(y)\end{aligned}$$

Here we used our assumption that $g_1 \circ f = g_2 \circ f$.

Next we assume that $f : X \to Y$ is an epimorphism, and we need to show that f is surjective. Let $y \in Y$ be arbitrary. We use classical logic to prove that there is $x \in X$ such that $f(x) = y$, so suppose that for each $x \in X$ we have $f(x) \neq y$. Without loss of generality, we assume that $* \notin Y$. We define a maps $g_1, g_2 : Y \to Y \cup \mathbb{1}$ as follows

$$g_1(y') = y'$$
$$g_2(y') = \begin{cases} y' & \text{if } y \neq y' \\ * & \text{if } y = y' \end{cases}$$

Note that $g_1 \neq g_2$, so to acquire a contradiction, it suffices to show $g_1 \circ f = g_2 \circ f$ since f is an epimorphism. Let $x \in X$. Since $f(x) \neq y$ by assumption, we have

$$g_2(f(x)) = f(x) = g_1(f(x))$$

Hence, f is surjective. □

Proposition 1.18. *A function $f : X \to Y$ of sets is bijective if and only if it is an isomorphism in the category* Set.

Proof. Suppose that $f : X \to Y$ is bijective. To define $g : Y \to X$, we note that for each $y \in Y$ there is a unique x such that $f(x) = y$, and we define $g(y)$ to be that x. From the description of g, we can immediately see that $f(g(y)) = y$: $g(y)$ is the unique element with $f(g(y)) = y$. To show that $g(f(x)) = x$, we use that f is injective, so it suffices to show that $f(g(f(x))) = f(x)$. This follows from the fact that we just showed that $f(g(y)) = y$ where we take y to be $f(x)$.

Next we assume that $f : X \to Y$ is an isomorphism, which means that we have a function $g : Y \to X$ such that $g \circ f = \mathsf{id}_X$ and $f \circ g = \mathsf{id}_Y$. To show that f is injective, we assume that we have $x_1, x_2 \in X$ such that $f(x_1) = f(x_2)$. Now we note

$$x_1 = g(f(x_1)) = g(f(x_2)) = x_2$$

To show that f is surjective, we let $y \in Y$ be arbitrary. An inverse image for y is given by $g(y)$, since $f(g(y)) = y$ by assumption. □

For the category of monoids and of groups, we can use the same strategy as for the category of sets. We leave this as an exercise (Exercises 1.12).

Next we look at the opposite category $\mathcal{C}^{\mathsf{op}}$.

Proposition 1.19. *A morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is a monomorphism in $\mathcal{C}^{\mathsf{op}}$ if and only if $f : y \to_{\mathcal{C}} x$ is a epimorphism in $\mathcal{C}$.*

Proof. Suppose that $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is a monomorphism in $\mathcal{C}^{\mathsf{op}}$. Our goal is to show that $f : y \to_{\mathcal{C}} x$ is an epimorphism in $\mathcal{C}$. Suppose that we have $z : \mathcal{C}$ and $g_1, g_2 : x \to_{\mathcal{C}} z$ such that $g_1 \circ_{\mathcal{C}} f = g_2 \circ_{\mathcal{C}} f$. From g_1 and g_2 we obtain morphisms $g_1, g_2 : z \to_{\mathcal{C}^{\mathsf{op}}} x$. Note the following

$$\begin{aligned} f \circ_{\mathcal{C}^{\mathsf{op}}} g_1 &= g_1 \circ_{\mathcal{C}} f \\ &= g_2 \circ_{\mathcal{C}} f \\ &= f \circ_{\mathcal{C}^{\mathsf{op}}} g_2 \end{aligned}$$

Since f is a monomorphism in $\mathcal{C}^{\mathsf{op}}$, we have $g_1 = g_2$, and thus $f : y \to_{\mathcal{C}} x$ is a epimorphism in $\mathcal{C}$.

Now we assume that $f : y \to_{\mathcal{C}} x$ is an epimorphism in $\mathcal{C}$, and we show that $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is a monomorphism in $\mathcal{C}^{\mathsf{op}}$. We take the same steps: we assume that we have we have $z : \mathcal{C}$ and $g_1, g_2 : z \to_{\mathcal{C}^{\mathsf{op}}} x$ such that $f \circ_{\mathcal{C}^{\mathsf{op}}} g_1 = f \circ_{\mathcal{C}^{\mathsf{op}}} g_2$. We get morphisms $g_1, g_2 : x \to_{\mathcal{C}} z$ from g_1 and g_2, and we have the following equalities.

$$\begin{aligned} g_1 \circ_{\mathcal{C}} f &= f \circ_{\mathcal{C}^{\mathsf{op}}} g_1 \\ &= f \circ_{\mathcal{C}^{\mathsf{op}}} g_2 \\ &= g_2 \circ_{\mathcal{C}} f \end{aligned}$$

Since f is an epimorphism in $\mathcal{C}$, we get $g_1 = g_2$, which finishes the proof that f is a monomorphism in $\mathcal{C}^{\mathsf{op}}$. □

Proposition 1.20. *A morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is a epimorphism in $\mathcal{C}^{\mathsf{op}}$ if and only if f is a monomorphism in $\mathcal{C}$.*

Proof. Left as an exercise (Exercise 1.9). □

Proposition 1.21. *A morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is an isomorphism in $\mathcal{C}^{\mathsf{op}}$ if and only if $f : y \to_{\mathcal{C}} x$ is an isomorphism in $\mathcal{C}$.*

Proof. Suppose that $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ is an isomorphism in $\mathcal{C}^{\mathsf{op}}$. This means that we have $g : y \to_{\mathcal{C}^{\mathsf{op}}} x$ such that $g \circ_{\mathcal{C}^{\mathsf{op}}} f = \mathsf{id}_x$ and $f \circ_{\mathcal{C}^{\mathsf{op}}} g = \mathsf{id}_y$. Note that g gives us a morphism $g : x \to_{\mathcal{C}} y$ such that

$$f \circ_{\mathcal{C}} g = g \circ_{\mathcal{C}^{\mathsf{op}}} f = \mathsf{id}_x$$

$$g \circ_{\mathcal{C}} f = f \circ_{\mathcal{C}^{\mathsf{op}}} g = \mathsf{id}_y$$

Hence, $f : y \to_{\mathcal{C}} x$ is an isomorphism in $\mathcal{C}$.

The converse is proven similarly. If $f : y \to_{\mathcal{C}} x$ is an isomorphism in $\mathcal{C}$, then we have $g : x \to_{\mathcal{C}} y$ such that $f \circ_{\mathcal{C}} g = \mathsf{id}_x$ and $g \circ_{\mathcal{C}} f = \mathsf{id}_y$. Hence, we have morphisms $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ and $g : y \to_{\mathcal{C}^{\mathsf{op}}} x$ such that

$$g \circ_{\mathcal{C}^{\mathsf{op}}} f = f \circ_{\mathcal{C}} g = \mathsf{id}_x$$

$$f \circ_{\mathcal{C}^{\mathsf{op}}} g = g \circ_{\mathcal{C}} f = \mathsf{id}_y$$

Hence, f is an isomorphism in $\mathcal{C}^{\mathsf{op}}$. □

The final example that we look at, is $\widetilde{(X, \leq)}$.

Proposition 1.22. *Every morphism in $\widetilde{(X, \leq)}$ is a monomorphism and an epimorphism.*

Proof. To show that some morphism $f : x \to y$ is a monomorphism, we assume that we have $g_1, g_2 : w \to x$ such that $f \circ g_1 = f \circ g_2$. We must also have $g_1 = g_2$, because all morphisms in $\widetilde{(X, \leq)}$ are equal. Similarly, we can show that every morphism is an epimorphism. □

Proposition 1.23. *We have an isomorphism between x and y in $\widetilde{(X, \leq)}$ if and only if $x \leq y$ and $y \leq x$.*

Proof. An isomorphism $\widetilde{(X, \leq)}$ in x and y consists of morphisms $f : x \to y$ and $g : y \to x$. By definition, these morphisms give us that $x \leq y$ and $y \leq x$. If we have morphisms $x \to y$ if $x \leq y$ and $y \to x$ if $y \leq x$, then we obtain an isomorphism, since we have $f : x \to y$ (since $x \leq y$) and $g : y \to x$ (since $y \leq x$). Because all morphisms are equal in $\widetilde{(X, \leq)}$, the necessary equalities hold, and thus we have an isomorphism. □

Proposition 1.24. *Every isomorphism is a monomorphism and an epimorphism.*

Proof. Left as an exercise (Exercise 1.15). □

Note that the converse of Proposition 1.24 is false.

1.3 Exercises

Exercise 1.1. Complete Example 1.4 by proving that Grp is a category.

Exercise 1.2. Finish the proof that $\mathcal{C}^{\mathsf{op}}$ (Example 1.6) forms a category by proving the remain unitality law.

Exercise 1.3. Show that $\mathcal{C}_1 \times \mathcal{C}_2$ forms a category (Example 1.7) by defining the identity and composition of morphisms, and proving the category laws.

Exercise 1.4. Define the category of preorders PreOrd where a morphism from $(X, \leq)$ to $(Y, \leq)$ is a function $f : X \to Y$ such that $f(x) \leq f(y)$ whenever $x \leq y$.

Exercise 1.5. Let $\mathcal{C}$ be a category such that for all $f, g : x \to y$, we have $f = g$. Show that $\mathcal{C}$ induces a preorder on the objects of $\mathcal{C}$.

Exercise 1.6. The objects of the **coslice category** $x/\mathcal{C}$ are pairs of objects $y : \mathcal{C}$ together with $x \to y$. Define the morphisms of the coslice category and show that it indeed forms a category

Exercise 1.7. We consider the **category of families** Fam. Each object of Fam consists of a set X and for each $x \in X$ a set $Y(x)$. Morphisms from (X_1, Y_1) to (X_2, Y_2) consist of a function $f : X_1 \to X_2$ together with maps $g_x : Y_1(x) \to Y_2(f(x))$ for each $x \in X_1$. Show that Fam indeed forms a category.

Exercise 1.8. Recall that a **graph** (V, E) consists of a set V, whose elements are called **vertices**, and a relation E between V and V. If $E(v_1, v_2)$ holds, then we say there is an **edge** from v_1 to v_2. Show that every graph gives rise to a category whose objects are vertices and such that edges give rise to morphisms.

Exercise 1.9. Prove Proposition 1.20.

Exercise 1.10. Let M be a monoid. Show that M is a group if and only if every morphism in $\mathsf{B}(M)$ is an isomorphism.

Exercise 1.11. Characterize the isomorphism, monomorphisms, and epimorphisms in the product category $\mathcal{C}_1 \times \mathcal{C}_2$.

Exercise 1.12. Characterize the isomorphism, monomorphisms, and epimorphisms in the category Mon of monoids.

Exercise 1.13. Characterize the isomorphism, monomorphisms, and epimorphisms in the category Grp of groups.

Exercise 1.14. Give a bijective homorphism of preorders that is not an isomorphism.

Exercise 1.15. Prove Proposition 1.24.

Exercise 1.16. Give a category $\mathcal{C}$ and a morphism f in $\mathcal{C}$ such that f is a monomorphism and an epimorphism, but not an isomorphism.

2 Constructions in Categories

2.1 Terminal and Initial Objects

Definition 2.1. Let $\mathcal{C}$ be a category and let $x \in \mathcal{C}$ be an object. We say that x **is terminal** if for all objects y there is a unique morphism $!_y : y \to x$.

The condition that for all objects y we have a unique morphism $!_y : y \to x$, is called the **universal property of terminal objects**.

Definition 2.2. Let $\mathcal{C}$ be a category and let $x \in \mathcal{C}$ be an object. We say that x **is initial** if for all objects y there is a unique morphism $!_y : x \to y$.

The condition that for all objects y we have a unique morphism $!_y : x \to y$, is called the **universal property of initial objects**.

Example 2.3. The set $\mathbb{1} = \{*\}$ is a terminal object in the category Set of sets. If we have any set X, then we can construct a function $f : X \to \mathbb{1}$, which maps every x to the unique element $*$ of $\mathbb{1}$. Now suppose that we have two functions $f, g : X \to \mathbb{1}$. Our goal is to show that $f = g$. To do so, we take an arbitrary $x \in X$, and we need to show that $f(x) = g(x)$. This follows from the following chain of equalities

$$f(x) = * = g(x)$$

since all elements in $\mathbb{1}$ are equal to $*$.

The empty set $\emptyset$ is an initial object in Set. Let X be any set. We have a function f from $\emptyset \to X$, namely the empty function. In addition, all functions $f, g : \emptyset \to X$ are equal, because there is no $x \in \emptyset$.

Proposition 2.4. *Given an object $x : \mathcal{C}$, x is terminal in $\mathcal{C}$ if and only if x is initial in $\mathcal{C}^{\mathsf{op}}$.*

Proof. Suppose x is terminal in $\mathcal{C}$. To prove that x is initial in $\mathcal{C}^{\mathsf{op}}$, we take any $y : \mathcal{C}^{\mathsf{op}}$, and we need to prove there is a unique morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$. Note that morphisms $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ are the same as morphisms $f : y \to_{\mathcal{C}} x$. Since x is terminal, there is a unique morphism $f : y \to_{\mathcal{C}} x$. Hence, there also is a unique morphism $f : x \to_{\mathcal{C}^{\mathsf{op}}} y$ and thus x is initial. In the same way, we prove that x is terminal in $\mathcal{C}$ if x is initial in $\mathcal{C}^{\mathsf{op}}$. □

Proposition 2.5. *Given an object $x : \mathcal{C}$, x is initial in $\mathcal{C}$ if and only if x is terminal in $\mathcal{C}^{\mathsf{op}}$.*

Proof. Exercise. □

Remark 2.6. Propositions 2.4 and 2.5 capture an important pattern: the notions of initiality and terminality are dual. Terminality corresponds to initiality, but with the morphisms reversed. In category theory, a wide variety of concepts can be dualised, and we will see plenty of examples throughout these notes.

Example 2.7. We construct a terminal and an initial object in Mon. To construct a terminal object in Mon, we note that $\mathbb{1}$ is a monoid. The unit element $e_{\mathbb{1}}$ is $*$ and the multiplication is the function $\cdot_{\mathbb{1}}$ that maps all pairs (x, y) to $*$. Note that every function $f : M \to \mathbb{1}$ is a homomorphism. We have $f(e_M) = * = e_{\mathbb{1}}$ since all elements in $\mathbb{1}$ are equal, and we have

$$f(x \cdot_M y) = * = * \cdot_{\mathbb{1}} * = f(x) \cdot_{\mathbb{1}} f(y)$$

Hence, there is a unique homomorphism from every monoid M to $\mathbb{1}$.

Next we show that $\mathbb{1}$ also is initial in Mon. Let M be any monoid. We define the function $f : \mathbb{1} \to M$ to be $f(*) = e_M$. Note that f is a homomorphism:

$$f(e_{\mathbb{1}}) = f(*) = e_M$$

Since all $x, y : \mathbb{1}$ are equal to $*$, we also have

$$f(x \cdot_{\mathbb{1}} y) = f(* \cdot_{\mathbb{1}} *) = f(*) = e_M$$

$$f(x) \cdot_M f(y) = f(*) \cdot_M f(*) = e_M \cdot_M e_M = e_M$$

Note that f is unique as well. Let g be any other homomorphism from $\mathbb{1}$ to M. Then we have

$$g(*) = g(e_{\mathbb{1}}) = e_M = f(*)$$

Hence, $\mathbb{1}$ is both initial and terminal in Mon.

Example 2.8. Let us study initial and terminal objects in the category $\widetilde{(X, \leq)}$ for $(X, \leq)$ be a preorder. A terminal object for $\widetilde{(X, \leq)}$ is an object $x \in X$ such that for each $y \in X$ there is a unique morphism $f : y \to_{\widetilde{(X,\leq)}} x$. Since morphisms in $\widetilde{(X, \leq)}$ are unique and since we have a morphism $f : y \to_{\widetilde{(X,\leq)}} x$ if and only if $y \leq x$, we have that x is a terminal object in $\widetilde{(X, \leq)}$ if and only if for all y we have $y \leq x$. Hence, x is the greatest element of $(X, \leq)$.

An initial object for $\widetilde{(X, \leq)}$ is an object $x \in X$ such that for each $y \in X$ there is a unique morphism $f : x \to_{\widetilde{(X,\leq)}} y$. Hence, x is an initial object in $\widetilde{(X, \leq)}$ if and only if for all y we have $x \leq y$, meaning that x is the least element of $(X, \leq)$.

Proposition 2.9. *Let $\mathcal{C}$ be a category and let x and y be terminal objects in $\mathcal{C}$. Then we have a unique isomorphism $f : x \cong y$.*

Proof. Note that we have $f : x \to y$, because y is terminal, and that we have $g : y \to x$, because x is terminal. These morphisms exist due to the "existence" part in the universal property of terminal objects. We have $f \circ g = \mathsf{id}_y$ and $g \circ f = \mathsf{id}_x$ due to the uniqueness part in the universal property of terminal objects. Both $f \circ g$ and id_y are morphisms to the terminal object y, and hence they must be unique. Similarly for $g \circ f = \mathsf{id}_x$. Note that if we have another morphisms $f' : x \to y$, then we have that $f = f'$, because both are morphisms into the terminal object y. □

Proposition 2.10. *Let $\mathcal{C}$ be a category and let x and y be initial objects in $\mathcal{C}$. Then we have a unique isomorphism $f : x \cong y$.*

Proof. Exercise. □

Digression. We can also look at a terminal object in the category that arises from the simply typed λ-calculus. The objects in the category that we discussed, are types and morphisms are terms. To construct a terminal object in this category, we need to look at a version of the λ-calculus with unit types. Specifically, our λ-calculus needs to come with a type 1 and a term $* : 1$ such that all terms of type 1 are equal. Similarly, we can study inititial objects in this category. For those, we need to have a type 0 together with the usual elimination rule for the empty type.

Hence, to obtain a category with a terminal and initial object from the simply typed λ-calculus, we must look at a version that contains the unit type and the empty type.

2.2 Binary Products and Coproducts

Definition 2.11. Suppose that we have a category $\mathcal{C}$ and objects $x, y \in C$. A **binary product for x and y** consists of

- an object $z \in \mathcal{C}$;
- morphisms $\pi_1 : z \to x$ and $\pi_2 : z \to y$;

satisfying the following universal property. Suppose that we have

- an object $w \in \mathcal{C}$;
- a morphism $f : w \to x$;
- a morphism $g : w \to y$

then there is a unique morphism $\langle f, g\rangle : w \to z$ with

$$\pi_1 \circ \langle f, g\rangle = f$$

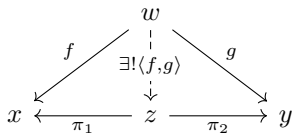
$$\pi_2 \circ \langle f, g\rangle = g$$

We call π_1 and π_2 the **projection morphism**, and we call $\langle f, g\rangle$ the **pairing**. We say that $\mathcal{C}$ has **binary products** if all $x, y \in C$ have a binary product. In case $\mathcal{C}$ has binary products, we denote the binary product of x and y by $x \times y$. Note that $x \times y$ satisfies the universal property described in this definition.

To understand Theorem 2.11, we phrase it using the the language of diagrams. We start with z and the projection morphisms.

$$x \xleftarrow{\pi_1} z \xrightarrow{\pi_2} y$$

We draw the universal property as follows.



Definition 2.12. Suppose that we have a category $\mathcal{C}$ and objects $x, y \in C$. A **binary coproduct for x and y** consists of

- an object $z \in \mathcal{C}$;
- morphisms $\iota_1 : x \to z$ and $\iota_2 : y \to z$;

satisfying the following universal property. Suppose that we have

- an object $w \in \mathcal{C}$;
- a morphism $f : x \to w$;
- a morphism $g : y \to w$

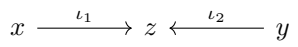
then there is a unique morphism $[f, g] : z \to w$ with

$$[f, g] \circ \iota_1 = f$$

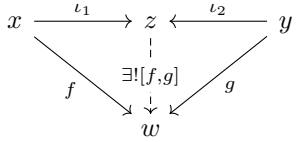
$$[f, g] \circ \iota_2 = g$$

We call ι_1 and ι_2 the **inclusion morphisms**, and we call $[f, g]$ the **copairing**. We say that $\mathcal{C}$ has **binary coproducts** if all $x, y \in C$ have a binary coproduct. In case $\mathcal{C}$ has binary coproducts, we denote the binary coproduct of x and y by $x + y$.

We can visualise binary coproducts in a similar way. The inclusion morphisms are drawn as follows.



The universal property is seen as follows.



Example 2.13. We show that the category Set of sets has binary products. In this example, it is important to distinguish the notation (x, y), which is an element of $X \times Y$, from $\langle f, g \rangle$, which is a morphism in Set that we construct. Let X and Y be sets, and we define the set $X \times Y$ as follows

$$X \times Y = \{(x, y) \mid x \in X, y \in Y\}$$

We claim that $X \times Y$ is the binary product of X and Y. To see why, we define the following morphisms. We start with $\pi_1 : X \times Y \to X$:

$$\pi_1(x, y) = x$$

The morphism $\pi_2 : X \times Y \to Y$ is defined similarly:

$$\pi_2(x, y) = y$$

Next we need to prove the universal property. Let W be any set and suppose that we have functions $f : W \to X$ and $g : W \to Y$. We define $\langle f, g\rangle : W \to X \times Y$ as follows:

$$\langle f, g\rangle(w) = (f(w), g(w))$$

Note that $\pi_1 \circ \langle f, g\rangle = f$, because for every $w \in W$ we have

$$(\pi_1 \circ \langle f, g\rangle)(w) = \pi_1(\langle f, g\rangle(w)) = \pi_1(f(w), g(w)) = f(w)$$

Similarly we prove that $\pi_2 \circ \langle f, g\rangle = g$.

$$(\pi_2 \circ \langle f, g\rangle)(w) = \pi_2(\langle f, g\rangle(w)) = \pi_2(f(w), g(w)) = g(w)$$

We also need to show that $\langle f, g\rangle$ is the unique morphism with this property, so we assume that we have some $h : W \to X \times Y$ such that $\pi_1 \circ h = f$ and $\pi_2 \circ h = g$. Let $w \in W$ be arbitrary, and note that

$$\langle f, g\rangle(w) = (f(w), g(w)) = (\pi_1(h(w)), \pi_2(h(w))) = h(w)$$

Hence, we have $\langle f, g\rangle = h$, meaning that $\langle f, g\rangle$ is the unique morphism with this property. This proves that $X \times Y$ is indeed the binary product of X and Y.

Example 2.14. Next we show that the category Set has binary coproducts. Given sets X and Y, we define the set $X + Y$ as follows

$$X + Y = \{(0, x) \mid x \in X\} \cup \{(1, y) \mid y \in Y\}$$

We call $X + Y$ the **disjoint union** of X and Y. We show that $X + Y$ is the coproduct of X and Y. We start by defining the morphism $\iota_1 : X \to X + Y$:

$$\iota_1(x) = (0, x)$$

We define $\iota_2 : Y \to X + Y$ similarly:

$$\iota_2(y) = (1, y)$$

Now we need to prove the universal property, so we take an arbitrary set W and functions $f : X \to W$ and $g : Y \to W$. We define $[f, g] : X + Y \to W$ as follows

$$[f, g](z) = \begin{cases} f(x) & \text{if } z = (0, x) \\ g(y) & \text{if } z = (1, y) \end{cases}$$

Note that $[f, g]$ is total, because all elements in $X + Y$ are either of the form $(0, x)$ or $(1, y)$. We can directly prove the following equations

$$[f, g](\iota_1(x)) = [f, g](0, x) = f(x)$$
$$[f, g](\iota_2(y)) = [f, g](1, y) = g(y)$$

To prove the uniqueness requirement, we assume that we have some $h : X+Y \to W$ such that $h \circ \iota_1 = f$ and $h \circ \iota_2 = g$. If we have $z \in X + Y$, then we know that z is either of the form $(0, x)$ or of the form $(1, y)$ where $x \in X$ or $y \in Y$. In the first case, we have

$$h(0, x) = h(\iota_1(x)) = f(x) = [f, g](0, x) = [f, g](\iota_1(x))$$

In the second case, we have

$$h(1, y) = h(\iota_2(y)) = g(y) = [f, g](1, y) = [f, g](\iota_2(y))$$

This concludes the proof of the uniqueness requirement, and hence $X + Y$ is indeed a binary coproduct of X and Y.

Proposition 2.15. *Let $\mathcal{C}$ be a category and suppose that we have objects $x, y, z \in \mathcal{C}$. Then z is a binary product of x and y in $\mathcal{C}$ if and only if z is a binary coproduct of x and y in $\mathcal{C}^{\mathsf{op}}$. Dually, z is a binary coproduct of x and y in $\mathcal{C}$ if and only if z is a binary product of x and y in $\mathcal{C}^{\mathsf{op}}$.*

Proof. Exercise □

Example 2.16. We show that the category Mon has binary products. Let M and N be monoids. We first show that $M \times N = \{(m, n) \mid m \in M, n \in N\}$ is a monoid. We define $e_{M\times N}$ to be (e_M, e_N), and we define $(m_1, n_1) \cdot_{M\times N} (m_2, n_2)$ to be $(m_1 \cdot m_2, n_1 \cdot n_2)$. To show that $M \times N$ is the binary product of M and N, we first note that π_1 and π_2 are homomorphisms. For π_1, this is so, because

$$\pi_1(e_{M\times N}) = \pi_1(e_M, e_N) = e_M$$
$$\begin{aligned}\pi_1((m_1, n_1) \cdot_{M\times N} (m_2, n_2)) &= \pi_1(m_1 \cdot m_2, n_1 \cdot n_2)\\ &= m_1 \cdot m_2\\ &= \pi_1(m_1, n_1) \cdot \pi_1(m_2, n_2)\end{aligned}$$

We can use the same reasoning for π_2. If $f : W \to M$ and $g : W \to N$ are homomorphisms, then so is $\langle f, g\rangle$. This is because

$$\begin{aligned}\langle f, g\rangle(e_W) &= (f(e_W), g(e_W))\\ &= (e_M, e_N)\\ &= e_{M\times N}\end{aligned}$$
$$\begin{aligned}\langle f, g\rangle(w_1 \cdot w_2) &= (f(w_1 \cdot w_2), g(w_1 \cdot w_2))\\ &= (f(w_1) \cdot f(w_2), g(w_1) \cdot g(w_2))\\ &= (f(w_1), g(w_1)) \cdot (f(w_2), g(w_2))\\ &= \langle f, g\rangle(w_1) \cdot \langle f, g\rangle(w_2)\end{aligned}$$

The uniqueness of $\langle f, g\rangle$ is proven in the same way as for Set.

Constructing binary coproducts in the category of monoids is more challenging, and we leave it as an exercise.

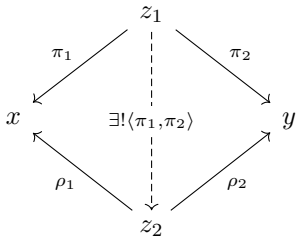
Example 2.17. Let $(X, \leq)$ be a preorder. We look at binary products in $\widetilde{(X, \leq)}$. Given $x, y \in X$, a binary product for x and y is an element $z \in X$ such that $x \leq z$ and $y \leq z$. This z also satisfies another property: for every $w \in X$ with $x \leq w$ and $y \leq w$, we must also have $z \leq w$. Hence, z is a greatest lower bound of x and y.

We can similarly show that binary coproducts of x and y are least upper bounds. Given $x, y \in X$, a binary coproduct for x and y is an element $z \in X$ such that $z \leq x$ and $z \leq y$ such that for all $w \in X$ with $w \leq x$ and $w \leq y$, we have $w \leq z$.

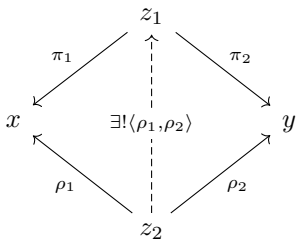
Proposition 2.18. *Let $\mathcal{C}$ be a category and let x and y be objects in $\mathcal{C}$. Suppose that $z_1, z_2 : \mathcal{C}$ are binary products of x and y. Then we have an isomorphism $f : z_1 \cong z_2$.*

Proof. The proof of this proposition consists of several applications of the universal property of binary products described in Definition 2.11.

Since z_1 is a binary product of x and y, we have morphisms $\pi_1 : z_1 \to x$ and $\pi_2 : z_1 \to y$. Similarly, we have morphisms $\rho_1 : z_2 \to x$ and $\rho_2 : z_2 \to y$. We first construct a morphism $f : z_1 \to z_2$, which we define to be $\langle \pi_1, \pi_2\rangle$ as displayed below.



We define $g : z_2 \to z_1$ to be $\langle \rho_1, \rho_2\rangle$.



We want to prove that $f \circ g = \mathsf{id}_{z_2}$. To do so, it suffices to prove that $\rho_1 \circ f \circ g =$

$\rho_1 \circ \mathsf{id}_{z_2}$ and that $\rho_2 \circ f \circ g = \rho_2 \circ \mathsf{id}_{z_2}$. This follows from the following equalities

$$\begin{aligned}\rho_1 \circ f \circ g &= \rho_1 \circ \langle \pi_1, \pi_2 \rangle \circ g \\ &= \pi_1 \circ g \\ &= \pi_1 \circ \langle \rho_1, \rho_2 \rangle \\ &= \rho_1\end{aligned}$$

$$\begin{aligned}\rho_2 \circ f \circ g &= \rho_2 \circ \langle \pi_1, \pi_2 \rangle \circ g \\ &= \pi_2 \circ g \\ &= \pi_2 \circ \langle \rho_1, \rho_2 \rangle \\ &= \rho_2\end{aligned}$$

We prove that $g \circ f = \mathsf{id}_{z_1}$ in the same way. □

Proposition 2.19. *Let $\mathcal{C}$ be a category and let x and y be objects in $\mathcal{C}$. Suppose that $z_1, z_2 : \mathcal{C}$ are binary coproducts of x and y. Then we have an isomorphism $f : z_1 \cong z_2$.*

Proof. Exercise. □

Example 2.20. We introduce the following notation. Let $\mathcal{C}$ be a category with binary products. Given $f : x_1 \to x_2$ and $g : y_1 \to y_2$, we define $f \times g : x_1 \times y_1 \to x_2 \times y_2$ as follows.

$$\begin{array}{ccccc} x_1 & \xleftarrow{\pi_1} & x_1 \times y_1 & \xrightarrow{\pi_2} & y_1 \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle f \times g} & & \downarrow{\scriptstyle g} \\ x_2 & \xleftarrow[\pi_1]{} & x_2 \times y_2 & \xrightarrow[\pi_2]{} & y_2 \end{array}$$

Note we have that $\pi_1 \circ (f \times g) = f \circ \pi_1$ and that $\pi_2 \circ (f \times g) = g \circ \pi_2$. Dually, if $\mathcal{C}$ is a category with binary coproducts, and if we have morphisms $f : x_1 \to x_2$ and $g : y_1 \to y_2$, we define $f + g : x_1 + y_1 \to x_2 + y_2$ as follows.

$$\begin{array}{ccccc} x_1 & \xrightarrow{\iota_1} & x_1 + y_1 & \xleftarrow{\iota_2} & y_1 \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle f + g} & & \downarrow{\scriptstyle g} \\ x_2 & \xrightarrow[\iota_1]{} & x_2 + y_2 & \xleftarrow[\iota_2]{} & y_2 \end{array}$$

We have

$$(f + g) \circ \iota_1 = \iota_1 \circ f$$
$$(f + g) \circ \iota_2 = \iota_2 \circ g$$

Example 2.21. If $\mathcal{C}$ is a category with binary products, then, in general, we only have an isomorphism between $x \times y$ and $y \times x$ (Exercise 2.4). They are not guaranteed to be equal. This pattern in common in category theory: objects are characterised by some universal property, which only guarantees their uniqueness up to isomorphism.

Digression. To guarantee that the category arising from the simply typed λ-calculus has binary products and binary coproducts, we again need to consider suitable extensions to the type systems. If we want this category to have binary products, we must have a type constructor that all types τ and σ and gives us a type $\tau \times \sigma$. For the type $\tau \times \sigma$, we need to have a first and second projection, but also a pairing operation that allows us to construct terms of type $\tau \times \sigma$. In addition, these terms need to satisfy the relevant β- and η-rules: η rules guarantee uniqueness, whereas β-rules give that the required diagrams commute. For binary coproducts, we need to extend the type system with coproduct types. The extension is dual: we must have types $\tau + \sigma$, left and right inclusion for terms, and a term that allows us to do case distinction. We need to require β- and η-rules to guarantee uniqueness and that the required diagrams commute.

2.3 Pullbacks

Definition 2.22. Let $\mathcal{C}$ be a category with objects $x, y, z \in C$. Suppose we have morphisms $f : x \to z$ and $g : y \to z$. A **pullback** for f and g consists of

- an object $w \in \mathcal{C}$
- morphisms $\pi_1 : w \to x$ and $\pi_2 : w \to y$

such that $f \circ \pi_1 = g \circ \pi_2$. We require the following universal property: for all objects $a \in \mathcal{C}$ with morphisms $h_1 : a \to x$ and $h_2 : a \to y$ such that $f \circ h_1 = g \circ h_2$, there is a unique $k : a \to w$ such that $\pi_1 \circ k = h_1$ and $\pi_2 \circ k = h_2$.

We say that $\mathcal{C}$ has pullbacks if every f and g has a pullback.

To understand the definition of pullbacks, we visualise the definition using diagrams. We start with the following morphisms.

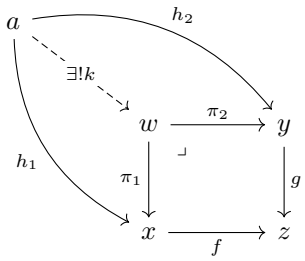
$$\begin{array}{ccc} & & y \\ & & \downarrow{\scriptstyle g} \\ x & \xrightarrow[f]{} & z \end{array}$$

Our goal is to say that w is a pullback of f and g, which means that we have a commuting diagram as follows.

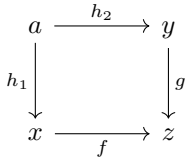
$$\begin{array}{ccc} w & \xrightarrow{\pi_2} & y \\ {\scriptstyle \pi_1}\downarrow & \lrcorner & \downarrow{\scriptstyle g} \\ x & \xrightarrow[f]{} & z \end{array}$$

The symbol $\lrcorner$ indicates that this square is a pullback, which means that the

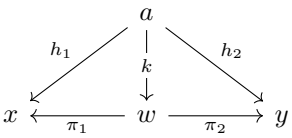
following universal property is satisfied.



Specifically, we assume that we have $h_1 : a \to x$ and $h_2 : a \to y$ making the diagram below commute.



Then we obtain $k : a \to w$ making the triangles below commute



Example 2.23. Let X, Y, Z be sets and suppose that we have functions $f : X \to Z$ and $g : Y \to Z$. The pullback for f and g is the following set

$$W = \{(x, y) \mid x \in X, y \in Y, f(x) = g(y)\}$$

To show that W is indeed the pullback of f and g, we define the following functions. We start with $\pi_1 : W \to X$.

$$\pi_1(x, y) = x$$

Next we define $\pi_2 : W \to Y$.

$$\pi_2(x, y) = y$$

Note that $f \circ \pi_1 = g \circ \pi_2$. Every element of W is of the form (x, y) with $x \in X$, $y \in Y$ and $f(x) = g(y)$. Hence,

$$(f \circ \pi_1)(x, y) = f(\pi_1(x, y)) = f(x) = g(y) = g(\pi_2(x, y)) = (g \circ \pi_2)(x, y)$$

Now suppose that we have some set A with functions $h_1 : A \to X$ and $h_2 : A \to Y$ such that $f \circ h_1 = g \circ h_2$. We define a map $k : A \to W$ as follows

$$k(a) = (h_1(a), h_2(a))$$

Note that $k(a)$ is indeed an element of W since $f(h_1(a)) = g(h_2(a))$ by assumption. In addition,

$$(\pi_1 \circ k)(a) = \pi_1(k(a)) = \pi_1(h_1(a), h_2(a)) = h_1(a)$$

$$(\pi_2 \circ k)(a) = \pi_2(k(a)) = \pi_2(h_1(a), h_2(a)) = h_2(a)$$

To prove that k is unique, we assume that we have $k' : A \to W$ such that $\pi_1 \circ k' = h_1$ and $\pi_2 \circ k' = h_2$. Then we must also have for every $a \in A$

$$k(a) = (h_1(a), h_2(a)) = (\pi_1(k'(a)), \pi_2(k'(a))) = k'(a)$$

Hence, W is indeed a pullback.

Example 2.24. Let $(X, \leq)$ be a preorder. Pullbacks in $\widetilde{(X, \leq)}$ are the same as greatest lower bounds. If we have $x \leq z$ and $y \leq z$, then a pullback consists of $w \in X$ such that for all $a \in X$ with $a \leq x$ and $a \leq y$, we must also have $a \leq w$. Hence, w is the greatest lower bound of x and y.

Proposition 2.25. *Pullbacks are unique up to isomorphism.*

Proof. Exercise. □

2.4 Exercises

Exercise 2.1. Prove Proposition 2.5.

Exercise 2.2. Prove Proposition 2.10.

Exercise 2.3. Prove Proposition 2.15.

Exercise 2.4. Let $\mathcal{C}$ be a category with binary products. Construct an isomorphism between $x \times y$ and $y \times x$.

Exercise 2.5. Let $\mathcal{C}$ be a category with binary coproducts. Construct an isomorphism between $x + (y + z)$ and $(x + y) + z$.

Exercise 2.6. Let $\mathcal{C}$ be a category with binary products and a terminal object $\mathbb{1}$. Construct an isomorphism between $x \times \mathbb{1}$ and x.

Exercise 2.7. Let $\mathcal{C}$ be a category with binary coproducts and an initial object $\mathbb{0}$. Construct an isomorphism between $x + \mathbb{0}$ and x.

Exercise 2.8. Let $\mathcal{C}$ be a category with a terminal object $\mathbb{1}$. Show that an object $x : \mathcal{C}$ is terminal whenever we have an isomorphism $f : x \cong \mathbb{1}$.

Exercise 2.9. Let $\mathcal{C}$ be a category with an initial object $\mathbb{0}$. Show that an object $x : \mathcal{C}$ is initial whenever we have an isomorphism $f : x \cong \mathbb{0}$.

Exercise 2.10. Let $\mathcal{C}$ be a category and let z be a binary product of x and y. Show that an object $w : \mathcal{C}$ is a binary product of x and y as well whenever we have an isomorphism $f : w \cong z$.

Exercise 2.11. Let $\mathcal{C}$ be a category and let z be a binary coproduct of x and y. Show that an object $w : \mathcal{C}$ is a binary product of x and y as well whenever we have an isomorphism $f : w \cong z$.

Exercise 2.12. Let $\mathcal{C}$ be a category with a terminal object $\mathbb{1}$. Show that every morphism $f : \mathbb{1} \to x$ is a monomorphism.

Exercise 2.13. Let $\mathcal{C}$ be a category with an initial object $\mathbb{0}$. Show that every morphism $f : x \to \mathbb{0}$ is an epimorphism.

Exercise 2.14. Let $\mathcal{C}$ be a category with binary products and a terminal object. Show that the following square is a pullback.

$$\begin{array}{ccc} x \times y & \xrightarrow{\pi_2} & y \\ {\scriptstyle \pi_1}\downarrow & & \downarrow{\scriptstyle !_y} \\ x & \xrightarrow[!_y]{} & \mathbb{1} \end{array}$$

Exercise 2.15. Let $\mathcal{C}$ be a category with binary products. Prove that the following square is a pullback.

$$\begin{array}{ccc} x_1 \times y & \xrightarrow{f \times \mathsf{id}_y} & x_2 \times y \\ {\scriptstyle \pi_1}\downarrow & & \downarrow{\scriptstyle \pi_1} \\ x_1 & \xrightarrow[f]{} & x_2 \end{array}$$

Exercise 2.16. Prove Proposition 2.25.

Exercise 2.17. Give the dual notion of pullbacks, called **pushout**.

3 Functors and Natural Transformations

3.1 Functors

Definition 3.1. Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be categories. A **functor** $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ consists of

- for each object $x : \mathcal{C}_1$ an object $F(x) : \mathcal{C}_2$;
- for each morphism $f : x \to_{\mathcal{C}_1} y$ a morphism $F(f) : F(x) \to_{\mathcal{C}_2} F(y)$

such that

$$\mathsf{F}(\mathsf{id}_x) = \mathsf{id}_{F(x)}$$

and for all $f : x \to_{\mathcal{C}_1} y$ and $g : y \to_{\mathcal{C}_1} z$ we have

$$F(g \circ f) = F(g) \circ F(f).$$

In the upcoming examples, we use various categories that we defined before, and for a list of them, we refer the reader to Table 1.

Example 3.2. Let $\mathcal{C}$ be a category. We define the **identity functor** $\mathsf{Id}_{\mathcal{C}} : \mathcal{C} \to \mathcal{C}$ as follows.

- $\mathsf{Id}_{\mathcal{C}}(x) = x$ for $x : \mathcal{C}$
- $\mathsf{Id}_{\mathcal{C}}(f) = f$ for $f : x \to y$

Note that we indeed obtain a functor, because

$$\mathsf{Id}_{\mathcal{C}}(\mathsf{id}_x) = \mathsf{id}_x = \mathsf{id}_{\mathsf{Id}_{\mathcal{C}}(x)}$$

$$\mathsf{Id}_{\mathcal{C}}(g \circ f) = g \circ f = \mathsf{Id}_{\mathcal{C}}(g) \circ \mathsf{Id}_{\mathcal{C}}(f)$$

Example 3.3. Let $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{C}_3$ be functors. We define the **composition functor** $\mathsf{G} \circ \mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_3$ as follows.

- $(\mathsf{G} \circ \mathsf{F})(x) = \mathsf{G}(\mathsf{F}(x))$
- $(\mathsf{G} \circ \mathsf{F})(f) = \mathsf{G}(\mathsf{F}(f))$

We indeed obtain a functor, since we have

$$(\mathsf{G} \circ \mathsf{F})(\mathsf{id}_x) = \mathsf{G}(\mathsf{F}(\mathsf{id}_x)) = \mathsf{G}(\mathsf{id}_{\mathsf{F}(x)}) = \mathsf{id}_{\mathsf{G}(\mathsf{F}(x))}$$

$$(\mathsf{G} \circ \mathsf{F})(g \circ f) = \mathsf{G}(\mathsf{F}(g \circ f)) = \mathsf{G}(\mathsf{F}(g) \circ \mathsf{F}(f)) = \mathsf{G}(\mathsf{F}(g)) \circ \mathsf{G}(\mathsf{F}(f))$$

Example 3.4. Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be categories, and let $y : \mathcal{C}_2$ be an object. We define the **constant functor** $\mathsf{Const}_y : \mathcal{C}_1 \to \mathcal{C}_2$ as follows.

- $\mathsf{Const}_y(x) = y$
- $\mathsf{Const}_y(f) = \mathsf{id}_y$

This is indeed a functor, because

$$\mathsf{Const}_y(\mathsf{id}_x) = \mathsf{id}_y$$

$$\mathsf{Const}_y(g \circ f) = \mathsf{id}_y = \mathsf{id}_y \circ \mathsf{id}_y = \mathsf{Const}_y(g) \circ \mathsf{Const}_y(f)$$

Recall that given sets X and Y, we define their **disjoint union** (Example 2.14) to be

$$X + Y = \{(0, x) \mid x \in X\} \cup \{(1, y) \mid y \in Y\}$$

If we have functions $f : X_1 \to X_2$ and $g : Y_1 \to Y_2$, then we define $f + g : X_1 + Y_1 \to X_2 + Y_2$ as follows.

$$(f + g)(w) = \begin{cases} (0, f(x)) & \text{if } w = (0, x) \text{ for some } x \in X_1 \\ (1, g(y)) & \text{if } w = (1, y) \text{ for some } y \in Y_1 \end{cases}$$

Example 3.5. We define the functor $\mathsf{Maybe} : \mathsf{Set} \to \mathsf{Set}$, which we call the **maybe functor** as follows.

- $\mathsf{Maybe}(X) = X + \mathbb{1}$
- $\mathsf{Maybe}(f) = f + \mathsf{id}_{\mathbb{1}}$

If we have $f : X_1 \to X_2$, then $\mathsf{Maybe}(f)$ is defined to be $f + \mathsf{id}_{\mathbb{1}} : X_1 + \mathbb{1} \to X_2 + \mathbb{1}$. Hence, $\mathsf{Maybe}(f)$ is indeed a morphism from $\mathsf{Maybe}(X_1) \to \mathsf{Maybe}(X_2)$. To show that Maybe is a functor, we prove the functor laws. First, we show that Maybe preserves the identity. Let X be any set and let $z : X + \mathbb{1}$ be any element. We need to show that $(\mathsf{id}_X + \mathsf{id}_{\mathbb{1}})(z) = z$. There are two cases. If $z = (0, x)$ for $x \in X$, then we have

$$(\mathsf{id}_X + \mathsf{id}_{\mathbb{1}})(0, x) = (0, \mathsf{id}_X(x)) = (0, x)$$

Otherwise, we have $z = (1, *)$, and then we have

$$(\mathsf{id}_X + \mathsf{id}_{\mathbb{1}})(1, *) = (1, \mathsf{id}_{\mathbb{1}}(*)) = (1, *)$$

Second, we show that Maybe preserves composition. Suppose that we have function $f : X \to Y$ and $g : Y \to Z$. To show that $(g \circ f) + \mathsf{id}_{\mathbb{1}} = (g + \mathsf{id}_{\mathbb{1}}) \circ (f + \mathsf{id}_{\mathbb{1}})$. Let $w : X + \mathbb{1}$ be arbitrary. Again there are two cases. If $w = (0, x)$ for some $x \in X$, we have

$$\begin{aligned} ((g \circ f) + \mathsf{id}_{\mathbb{1}})(0, x) &= (0, (g \circ f)(x)) \\ &= (0, g(f(x))) \\ &= (g + \mathsf{id}_{\mathbb{1}})(0, f(x)) \\ &= (g + \mathsf{id}_{\mathbb{1}})((f + \mathsf{id}_{\mathbb{1}})(0, x)) \\ &= ((g + \mathsf{id}_{\mathbb{1}}) \circ (f + \mathsf{id}_{\mathbb{1}}))(0, x) \end{aligned}$$

Otherwise, it must be that $w = (1, *)$, and in that case we have

$$\begin{aligned} ((g \circ f) + \mathsf{id}_{\mathbb{1}})(1, *) &= (1, \mathsf{id}_{\mathbb{1}}(*)) \\ &= (g + \mathsf{id}_{\mathbb{1}})(1, *) \\ &= (g + \mathsf{id}_{\mathbb{1}})((f + \mathsf{id}_{\mathbb{1}})(1, *)) \\ &= ((g + \mathsf{id}_{\mathbb{1}}) \circ (f + \mathsf{id}_{\mathbb{1}}))(1, *) \end{aligned}$$

Digression. The functor Maybe is important in functional programming. Programming languages like Haskell are designed to be pure by default. However, for various purposes, one would like to have effectful computations, for instance to deal with input and output, partial computations, and computations that might not terminate. Monads are used in Haskell for these purposes. We discuss monads in more detail in Section 6, but for now it only is important to know that every monad comes with a functor. To deal with partial computation, one uses the Maybe monad in Haskell, and the definition of the functor underlying the Maybe monad is exactly the functor we defined in Example 3.5. The reason why this functor is important for partial computations is as follows: functions $f : X \to Y + \mathbb{1}$ are the same as partial functions from X to Y.

If we have functions $f : X_1 \to X_2$ and $g : Y_1 \to Y_2$ between sets, then we also have a function $f \times g : X_1 \times Y_1 \to X_2 \times Y_2$ defined as follows

$$(f \times g)(x, y) = (f(x), g(y))$$

Note that we can obtain this function using Examples 2.13 and 2.20.

Example 3.6. Let A be any set. We define the functor $(-) \times A : \mathsf{Set} \to \mathsf{Set}$ as follows.

- $((-) \times A)(X) = X \times A$
- $((-) \times A)(f) = f \times \mathsf{id}_A$

We show that $(-) \times A$ is a functor. First, we show that $(-) \times A$ preserves the identity. To do so, we need to show that $(\mathsf{id}_X \times \mathsf{id}_A) = \mathsf{id}_{X \times A}$, so we take an arbitrary $z \in X \times A$. Since we have $z = (x, a)$ with $x \in X$ and $a \in A$, we also have

$$\begin{aligned}(\mathsf{id}_X \times \mathsf{id}_A)(x, a) &= (\mathsf{id}_X(x), \mathsf{id}_A(a)) \\ &= (x, a) \\ &= \mathsf{id}_{X \times A}(x, a)\end{aligned}$$

Second, we show that $(-) \times A$ preserves composition, so we need to show that $(g \circ f) \times \mathsf{id}_A = (g \times \mathsf{id}_A) \circ (f \times \mathsf{id}_A)$, where $f : X \to Y$ and $g : Y \to Z$. If we have any $x \in X$ and $a \in A$, then we also have

$$\begin{aligned}((g \circ f) \times \mathsf{id}_A)(x, a) &= ((g \circ f)(x), \mathsf{id}_A(a)) \\ &= (g(f(x)), a) \\ &= (g(f(x)), \mathsf{id}_A(a)) \\ &= (g \times \mathsf{id}_A)(f(x), a) \\ &= (g \times \mathsf{id}_A)(f(x), \mathsf{id}_A(a)) \\ &= (g \times \mathsf{id}_A)((f \times \mathsf{id}_A)(x, a)) \\ &= ((g \times \mathsf{id}_A) \circ (f \times \mathsf{id}_A))(x, a)\end{aligned}$$

Example 3.7. Let A be any set. We have a functor $\mathcal{L}_A : \mathsf{Set} \to \mathsf{Set}$. Given a set X, we define $\mathcal{L}_A(X)$ to be $X \times A + \mathbb{1}$. Given a function $f : X \to Y$, we define $\mathcal{L}_A(f) : X \times A + \mathbb{1} \to Y \times A + \mathbb{1}$ as follows

$$\mathcal{L}_A(f)(w) = \begin{cases} (0, (f(x), a)) & \text{if } w = (0, (x, a)) \\ (1, *) & \text{if } w = (1, *) \end{cases}$$

We leave it as an exercise to prove that this is indeed a functor.

Digression. Example 3.7 turns up in Section 4. In that section, we study algebras and coalgebras for functors, which are used to study inductive and coinductive data types in functional programming languages like Haskell. We show that we can use the functor $\mathcal{L}_A$ to describe the data type of lists.

Example 3.8. Suppose that we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G} : \mathcal{D}_1 \to \mathcal{D}_2$. We define a functor $\mathsf{F} \times \mathsf{G} : \mathcal{C}_1 \times \mathcal{D}_1 \to \mathcal{C}_2 \times \mathcal{D}_2$ as follows.

- $(\mathsf{F} \times \mathsf{G})(x, y) = (\mathsf{F}(x), \mathsf{G}(y))$
- $(\mathsf{F} \times \mathsf{G})(f, g) = (\mathsf{F}(f), \mathsf{G}(g))$

We leave it as an exercise to verify that $\mathsf{F} \times \mathsf{G}$ is indeed a functor.

Note that we can also define functors $\pi_1 : \mathcal{C}_1 \times \mathcal{C}_2 \to \mathcal{C}_1$ and $\pi_2 : \mathcal{C}_1 \times \mathcal{C}_2 \to \mathcal{C}_2$. The functor π_1 and π_2 map objects (x, y) to x and y respectively.

Example 3.9. Suppose that we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{D}$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{D}$, and suppose that $\mathcal{D}$ has binary products. Given objects $x, y \in D$, we denote their binary product by $x \times_{\mathcal{D}} y$. We define the functor $\mathsf{F} \times_{\mathcal{D}} \mathsf{G} : \mathcal{C}_1 \times \mathcal{C}_2 \to \mathcal{D}$ as follows.

- $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(x, y) = \mathsf{F}(x) \times_{\mathcal{D}} G(y)$
- given $f : x_1 \to x_2$ and $g : y_1 \to y_2$ we define $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f, g) : \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) \to \mathsf{F}(x_2) \times_{\mathcal{D}} \mathsf{G}(y_2)$ to be $F(f) \times \mathsf{G}(g)$

To visualise the action on morphisms, we use the following diagram

$$\begin{array}{ccccc} \mathsf{F}(x_1) & \xleftarrow{\pi_1} & \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) & \xrightarrow{\pi_2} & \mathsf{G}(y_1) \\ \downarrow{\scriptstyle \mathsf{F}(f)} & & \downarrow{\scriptstyle F(f) \times \mathsf{G}(g)} & & \downarrow{\scriptstyle \mathsf{G}(g)} \\ \mathsf{F}(x_2) & \xleftarrow[\pi_1]{} & \mathsf{F}(x_2) \times_{\mathcal{D}} \mathsf{G}(y_2) & \xrightarrow[\pi_2]{} & \mathsf{G}(y_2) \end{array}$$

By definition, $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f, g)$ is the unique morphism making this diagram commute. To show that $\mathsf{F} \times_{\mathcal{D}} \mathsf{G}$ is a functor, we prove the functor laws. We first show that $\mathsf{F} \times_{\mathcal{D}} \mathsf{G}$ preserves the identity. To show that $\mathsf{F}(\mathsf{id}_x) \times \mathsf{G}(\mathsf{id}_y) = \mathsf{id}_{\mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y)}$, we note that $\mathsf{F}(\mathsf{id}_x) \times \mathsf{G}(\mathsf{id}_y)$ is the unique morphism making the following diagram commute.

$$\begin{array}{ccccc} \mathsf{F}(x) & \xleftarrow{\pi_1} & \mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y) & \xrightarrow{\pi_2} & \mathsf{G}(y) \\ \downarrow{\scriptstyle \mathsf{F}(\mathsf{id}_x)} & & \downarrow{\scriptstyle F(\mathsf{id}_x) \times \mathsf{G}(\mathsf{id}_y)} & & \downarrow{\scriptstyle \mathsf{G}(\mathsf{id}_y)} \\ \mathsf{F}(x) & \xleftarrow[\pi_1]{} & \mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y) & \xrightarrow[\pi_2]{} & \mathsf{G}(y) \end{array}$$

Hence, we get that $\mathsf{F}(\mathsf{id}_x) \times \mathsf{G}(\mathsf{id}_y) = \mathsf{id}_{\mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y)}$ from the fact the following diagram commutes.

$$\begin{array}{ccccc}
\mathsf{F}(x) & \xleftarrow{\pi_1} & \mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y) & \xrightarrow{\pi_2} & \mathsf{G}(y) \\
{\scriptstyle \mathsf{F}(\mathsf{id}_x)}\downarrow & & \downarrow{\scriptstyle \mathsf{id}_{\mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y)}} & & \downarrow{\scriptstyle \mathsf{G}(\mathsf{id}_y)} \\
\mathsf{F}(x) & \xleftarrow[\pi_1]{} & \mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(y) & \xrightarrow[\pi_2]{} & \mathsf{G}(y)
\end{array}$$

Next we show that $\mathsf{F} \times_{\mathcal{D}} \mathsf{G}$ preserves composition. Suppose that we have morphisms $f_1 : x_1 \to x_2$ and $f_2 : x_2 \to x_3$ in $\mathcal{C}_1$, and morphisms $g_1 : y_1 \to y_2$ and $g_2 : y_2 \to y_3$ in $\mathcal{C}_2$. The morphism $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_2 \circ f_1, g_2 \circ g_1)$ is defined to be $F(f_2 \circ f_1) \times \mathsf{G}(g_2 \circ g_1)$, which is the unique morphism making the following diagram commutes.

$$\begin{array}{ccccc}
\mathsf{F}(x_1) & \xleftarrow{\pi_1} & \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) & \xrightarrow{\pi_2} & \mathsf{G}(y_1) \\
{\scriptstyle \mathsf{F}(f_2 \circ f_1)}\downarrow & & \downarrow{\scriptstyle F(f_2 \circ f_1) \times \mathsf{G}(g_2 \circ g_1)} & & \downarrow{\scriptstyle \mathsf{G}(g_2 \circ g_1)} \\
\mathsf{F}(x_3) & \xleftarrow[\pi_1]{} & \mathsf{F}(x_3) \times_{\mathcal{D}} \mathsf{G}(y_3) & \xrightarrow[\pi_2]{} & \mathsf{G}(y_3)
\end{array}$$

We visualise $((\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_2, g_2)) \circ ((\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_1, g_1))$ using the following diagram.

$$\begin{array}{ccccc}
\mathsf{F}(x_1) & \xleftarrow{\pi_1} & \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) & \xrightarrow{\pi_2} & \mathsf{G}(y_1) \\
{\scriptstyle \mathsf{F}(f_1)}\downarrow & & \downarrow{\scriptstyle F(f_1) \times \mathsf{G}(g_1)} & & \downarrow{\scriptstyle \mathsf{G}(g_1)} \\
\mathsf{F}(x_2) & \xleftarrow{\pi_1} & \mathsf{F}(x_2) \times_{\mathcal{D}} \mathsf{G}(y_2) & \xrightarrow{\pi_2} & \mathsf{G}(y_2) \\
{\scriptstyle \mathsf{F}(f_2)}\downarrow & & \downarrow{\scriptstyle F(f_2) \times \mathsf{G}(g_2)} & & \downarrow{\scriptstyle \mathsf{G}(g_2)} \\
\mathsf{F}(x_3) & \xleftarrow[\pi_1]{} & \mathsf{F}(x_3) \times_{\mathcal{D}} \mathsf{G}(y_3) & \xrightarrow[\pi_2]{} & \mathsf{G}(y_3)
\end{array}$$

To show that $(\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_2 \circ f_1, g_2 \circ g_1) = ((\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_2, g_2)) \circ ((\mathsf{F} \times_{\mathcal{D}} \mathsf{G})(f_1, g_1))$, it hence suffices to show that the following diagram commutes

$$\begin{array}{ccccc}
\mathsf{F}(x_1) & \xleftarrow{\pi_1} & \mathsf{F}(x_1) \times_{\mathcal{D}} \mathsf{G}(y_1) & \xrightarrow{\pi_2} & \mathsf{G}(y_1) \\
 & & \downarrow{\scriptstyle F(f_1) \times \mathsf{G}(g_1)} & & \\
{\scriptstyle \mathsf{F}(f_2 \circ f_1)}\downarrow & & \mathsf{F}(x_2) \times_{\mathcal{D}} \mathsf{G}(y_2) & & \downarrow{\scriptstyle \mathsf{G}(g_2 \circ g_1)} \\
 & & \downarrow{\scriptstyle F(f_2) \times \mathsf{G}(g_2)} & & \\
\mathsf{F}(x_3) & \xleftarrow[\pi_1]{} & \mathsf{F}(x_3) \times_{\mathcal{D}} \mathsf{G}(y_3) & \xrightarrow[\pi_2]{} & \mathsf{G}(y_3)
\end{array}$$

We only show that the left rectangle commutes, since proving the commutativity

of the right rectangle is similar.

$$\begin{aligned}\pi_1 \circ \mathsf{F}(f_2) \times \mathsf{G}(g_2) \circ \mathsf{F}(f_1) \times \mathsf{G}(g_1) &= \mathsf{F}(f_2) \circ \pi_1 \circ \mathsf{F}(f_1) \times \mathsf{G}(g_1) \\ &= \mathsf{F}(f_2) \circ \mathsf{F}(f_1) \circ \pi_1 \\ &= \mathsf{F}(f_2 \circ f_1) \circ \pi_1\end{aligned}$$

Example 3.10. Suppose that we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{D}$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{D}$, and suppose that $\mathcal{D}$ has binary coproducts. We have a functor $\mathsf{F} +_{\mathcal{D}} \mathsf{G} : \mathcal{C}_1 \times \mathcal{C}_2 \to \mathcal{D}$, whose definition is dual to Example 3.9. We leave it to the reader to verify that we indeed have this functor.

3.2 Natural Transformations

Definition 3.11. Let $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ and $G : \mathcal{C}_1 \to \mathcal{C}_2$ be functors. A **natural transformation** $\tau : \mathsf{F} \Rightarrow \mathsf{G}$ consists of a morphism $\tau(x) : \mathsf{F}(x) \to \mathsf{G}(x)$ such that for all $f : x \to y$ the following diagram commutes.

$$\begin{array}{ccc} \mathsf{F}(x) & \xrightarrow{\tau(x)} & \mathsf{G}(x) \\ {\scriptstyle \mathsf{F}(f)}\downarrow & & \downarrow{\scriptstyle \mathsf{G}(f)} \\ \mathsf{F}(y) & \xrightarrow[\tau(y)]{} & \mathsf{G}(y) \end{array}$$

This means that $\mathsf{G}(f) \circ \tau(x) = \tau(y) \circ \mathsf{F}(f)$. Alternative, we say that a collection of maps $\tau(x) : \mathsf{F}(x) \to \mathsf{G}(x)$ is **natural** whenever it forms a natural transformation.

Example 3.12. Let $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ be a functor. We define the **identity natural transformation** $\mathsf{id}_{\mathsf{F}} : \mathsf{F} \Rightarrow \mathsf{F}$ as follows. For each $x : \mathcal{C}_1$, we define $\mathsf{id}_{\mathsf{F}}(x) : \mathsf{F}(x) \to \mathsf{F}(x)$ to be the map $\mathsf{id}_{\mathsf{F}(x)}$. Note that we indeed have a natural transformation, because the following diagram commutes.

$$\begin{array}{ccc} \mathsf{F}(x) & \xrightarrow{\mathsf{id}_{\mathsf{F}(x)}} & \mathsf{F}(x) \\ {\scriptstyle \mathsf{F}(f)}\downarrow & & \downarrow{\scriptstyle \mathsf{F}(f)} \\ \mathsf{F}(y) & \xrightarrow[\mathsf{id}_{\mathsf{F}(y)}]{} & \mathsf{F}(y) \end{array}$$

This is because

$$\mathsf{F}(f) \circ \mathsf{id}_{\mathsf{F}(x)} = \mathsf{F}(f) = \mathsf{id}_{\mathsf{F}(y)} \circ \mathsf{F}(f)$$

Example 3.13. Suppose that we have three functors $\mathsf{F}, \mathsf{G}, \mathsf{H} : \mathcal{C}_1 \to \mathcal{C}_2$, and that we have natural transformations $\tau_1 : \mathsf{F} \Rightarrow \mathsf{G}$ and $\tau_2 : \mathsf{G} \Rightarrow \mathsf{H}$. We define their **composition** $\tau_2 \circ \tau_1 : \mathsf{F} \Rightarrow \mathsf{H}$ as follows. For each $x : \mathcal{C}_1$, we define $(\tau_2 \circ \tau_1)(x) : \mathsf{F}(x) \to \mathsf{H}(x)$ to be $\tau_2(x) \circ \tau_1(x)$. We indeed have a natural

transformation, because the following diagram commutes.

$$\begin{array}{ccccc}
\mathsf{F}(x) & \xrightarrow{\tau_1(x)} & \mathsf{G}(x) & \xrightarrow{\tau_2(x)} & \mathsf{H}(x) \\
{\scriptstyle \mathsf{F}(f)}\downarrow & & \downarrow{\scriptstyle \mathsf{G}(f)} & & \downarrow{\scriptstyle \mathsf{H}(f)} \\
\mathsf{F}(y) & \xrightarrow[\tau_1(y)]{} & \mathsf{G}(y) & \xrightarrow[\tau_2(y)]{} & \mathsf{H}(y)
\end{array}$$

The left- and right squares commute, because τ_1 and τ_2 are natural transformations. We can also present this proof using equational reasoning as follows.

$$\begin{aligned}
\mathsf{H}(f) \circ \tau_2(x) \circ \tau_1(x) &= \tau_2(y) \circ \mathsf{G}(f) \circ \tau_1(x) \\
&= \tau_2(y) \circ \tau_1(y) \circ \mathsf{F}(f)
\end{aligned}$$

Example 3.14. Suppose we have functors $\mathsf{F}_1, \mathsf{F}_2 : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{C}_3$, and that we have a natural transformation $\tau : \mathsf{F}_1 \Rightarrow \mathsf{F}_2$. We define the **left whiskering** $\mathsf{G} \lhd \tau : \mathsf{G} \circ \mathsf{F}_1 \Rightarrow \mathsf{G} \circ \mathsf{F}_2$ as follows. For $x : \mathcal{C}_1$, we define $(\mathsf{G} \lhd \tau)(x) : \mathsf{G}(\mathsf{F}_1(x)) \to \mathsf{G}(\mathsf{F}_2(x))$ to be $\mathsf{G}(\tau(x))$. To prove naturality, we need to show the following diagram commutes for every $f : x \to y$.

$$\begin{array}{ccc}
\mathsf{G}(\mathsf{F}_1(x)) & \xrightarrow{\mathsf{G}(\tau(x))} & \mathsf{G}(\mathsf{F}_2(x)) \\
{\scriptstyle \mathsf{G}(\mathsf{F}_1(f))}\downarrow & & \downarrow{\scriptstyle \mathsf{G}(\mathsf{F}_2(f))} \\
\mathsf{G}(\mathsf{F}_1(y)) & \xrightarrow[\mathsf{G}(\tau(y))]{} & \mathsf{G}(\mathsf{F}_2(y))
\end{array}$$

This is because

$$\begin{aligned}
\mathsf{G}(\mathsf{F}_2(f)) \circ \mathsf{G}(\tau(x)) &= \mathsf{G}(\mathsf{F}_2(f) \circ \tau(x)) \\
&= \mathsf{G}(\tau(y) \circ \mathsf{F}_1(f)) \\
&= \mathsf{G}(\tau(y)) \circ \mathsf{G}(\mathsf{F}_1(f))
\end{aligned}$$

Example 3.15. Suppose we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G}_1, \mathsf{G}_2 : \mathcal{C}_2 \to \mathcal{C}_3$, and that we have a natural transformation $\tau : \mathsf{G}_1 \Rightarrow \mathsf{G}_2$. We define the **right whiskering** $\tau \rhd F : \mathsf{G}_1 \circ \mathsf{F} \Rightarrow \mathsf{G}_2 \circ \mathsf{F}$ as follows. For $x : \mathcal{C}_1$, we define $(\tau \rhd F)(x) : \mathsf{G}_1(\mathsf{F}(x)) \to \mathsf{G}_2(\mathsf{F}(x))$ to be $\tau(\mathsf{F}(x))$. To prove naturality, we need to show the following diagram commutes for every $f : x \to y$.

$$\begin{array}{ccc}
\mathsf{G}_1(\mathsf{F}(x)) & \xrightarrow{\tau(\mathsf{F}(x))} & \mathsf{G}_2(\mathsf{F}(x)) \\
{\scriptstyle \mathsf{G}_1(\mathsf{F}(f))}\downarrow & & \downarrow{\scriptstyle \mathsf{G}_2(\mathsf{F}(f))} \\
\mathsf{G}_1(\mathsf{F}(y)) & \xrightarrow[\tau(\mathsf{F}(y))]{} & \mathsf{G}_2(\mathsf{F}(y))
\end{array}$$

This diagram commutes, because τ is natural: it is the naturality square for $\mathsf{F}(f)$.

Example 3.16. Suppose we have functors $\mathsf{F}_1, \mathsf{F}_2 : \mathcal{C}_1 \to \mathcal{C}_2$ and $\mathsf{G}_1, \mathsf{G}_2 : \mathcal{C}_2 \to \mathcal{C}_3$, and that we have a natural transformations $\tau_1 : \mathsf{F}_1 \Rightarrow \mathsf{F}_2$ and $\tau_2 : \mathsf{G}_1 \Rightarrow \mathsf{G}_2$. Then we have a **horizontal composition** $\tau_1 * \tau_2 : \mathsf{G}_1 \circ \mathsf{F}_1 \Rightarrow \mathsf{G}_2 \circ \mathsf{F}_2$. We leave it to the reader to define this natural transformation.

Example 3.17. Recall the functor Const_y from Example 3.4. Let $f : y_1 \to y_2$ be a morphisms. We define the **constant natural transformation** $\mathsf{Const}_f : \mathsf{Const}_{y_1} \Rightarrow \mathsf{Const}_{y_2}$ as follows. Given $x \in \mathcal{C}_1$, we define $\mathsf{Const}_f(x) : y_1 \to y_2$ to be f. To prove that this is indeed a natural transformation, we need to show that the following diagram commutes.

$$\begin{array}{ccc} y_1 & \xrightarrow{f} & y_2 \\ {\scriptstyle \mathsf{id}}\downarrow & & \downarrow{\scriptstyle \mathsf{id}} \\ y_1 & \xrightarrow[f]{} & y_2 \end{array}$$

This is so, because

$$\mathsf{id} \circ f = f = f \circ \mathsf{id}$$

Example 3.18. Recall the functor Maybe from 3.5. We define a natural transformation $\eta_{\mathsf{Maybe}} : \mathsf{id}_{\mathsf{Set}} \Rightarrow \mathsf{Maybe}$. For each set X, we define $\eta_{\mathsf{Maybe}}(X) : X \to X + \mathbb{1}$ to be the function that maps x to $(0, x)$. To prove naturality, we have to prove the following square commutes for each function $f : X \to Y$.

$$\begin{array}{ccc} X & \xrightarrow{\eta_{\mathsf{Maybe}}(X)} & X + \mathbb{1} \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle f + \mathsf{id}_{\mathbb{1}}} \\ Y & \xrightarrow[\eta_{\mathsf{Maybe}}(Y)]{} & Y + \mathbb{1} \end{array}$$

We take an arbitrary $x \in X$ and we note that

$$\begin{aligned} ((f + \mathsf{id}_{\mathbb{1}}) \circ \eta_{\mathsf{Maybe}}(X))(x) &= (f + \mathsf{id}_{\mathbb{1}})(\eta_{\mathsf{Maybe}}(X)(x)) \\ &= (f + \mathsf{id}_{\mathbb{1}})(0, x) \\ &= (0, f(x)) \\ &= \eta_{\mathsf{Maybe}}(Y)(f(x)) \\ &= (\eta_{\mathsf{Maybe}}(Y) \circ f)(x) \end{aligned}$$

Example 3.19. We define a natural transformation $\mu_{\mathsf{Maybe}} : \mathsf{Maybe} \circ \mathsf{Maybe} \Rightarrow \mathsf{Maybe}$. For each set X, we need to define a morphism $\mu_{\mathsf{Maybe}}(X) : (X + \mathbb{1}) + \mathbb{1} \to X + \mathbb{1}$. If we have $z \in (X + \mathbb{1}) + \mathbb{1}$, then there are three cases:

$$z = (0, (0, x)), \qquad z = (0, (1, *)), \qquad z = (1, *)$$

We define $\mu_{\mathsf{Maybe}}(X)(z) : X + \mathbb{1}$ as follows

$$\mu_{\mathsf{Maybe}}(X)(z) = \begin{cases} (0, x) & \text{if } z = (0, (0, x)) \text{ for some } x \in X \\ (1, *) & \text{otherwise} \end{cases}$$

To prove naturality, we need to show that the following diagram commutes for each $f : X \to Y$.

$$\begin{array}{ccc} (X + \mathbb{1}) + \mathbb{1} & \xrightarrow{\mu_{\mathsf{Maybe}}(X)} & X + \mathbb{1} \\ \downarrow{\scriptstyle (f+\mathsf{id}_{\mathbb{1}})+\mathsf{id}_{\mathbb{1}}} & & \downarrow{\scriptstyle f+\mathsf{id}_{\mathbb{1}}} \\ (Y + \mathbb{1}) + \mathbb{1} & \xrightarrow[\mu_{\mathsf{Maybe}}(Y)]{} & Y + \mathbb{1} \end{array}$$

Let $z \in (X + \mathbb{1}) + \mathbb{1}$ be any element. There are three cases. If $z = (0, (0, x))$, then we have

$$\begin{aligned} ((f + \mathsf{id}_{\mathbb{1}}) \circ \mu_{\mathsf{Maybe}}(X))(0, (0, x)) &= (f + \mathsf{id}_{\mathbb{1}})(\mu_{\mathsf{Maybe}}(X)(0, (0, x))) \\ &= (f + \mathsf{id}_{\mathbb{1}})(0, x) \\ &= (0, f(x)) \\ &= \mu_{\mathsf{Maybe}}(Y)(0, (0, f(x))) \\ &= \mu_{\mathsf{Maybe}}(Y)(((f + \mathsf{id}_{\mathbb{1}}) + \mathsf{id}_{unit})(0, (0, x))) \\ &= (\mu_{\mathsf{Maybe}}(Y) \circ ((f + \mathsf{id}_{\mathbb{1}}) + \mathsf{id}_{unit}))(0, (0, x)) \end{aligned}$$

The cases for $z = (0, (1, *))$ and $z = (1, *)$ are similar.

Digression. Examples 3.18 and 3.19 give the unit and multiplication of the maybe monad. The unit represents pure composition, whereas multiplication allows us to compose partial computation. We look at this example further in Section 6.

Example 3.20. Suppose that we have functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{D}$ and $\mathsf{G} : \mathcal{C}_2 \to \mathcal{D}$, and suppose that $\mathcal{D}$ has binary products. Recall that the functor $\mathsf{F} \times_D \mathsf{G} : \mathcal{C}_1 \times \mathcal{C}_2 \to \mathcal{D}$ from Example 3.9 maps (x, y) to $\mathsf{F}(x) \times_{\mathcal{D}} \mathsf{G}(x)$, and that the functor $\pi_1 : \mathcal{C}_1 \times \mathcal{C}_2 \to \mathcal{C}_1$ maps (x, y) to x. We define the **first projection** $\mathsf{pr}_1 : \mathsf{F} \times_{\mathcal{D}} \mathsf{G} \Rightarrow \mathsf{F} \circ \pi_1$ as follows. Given $x \in \mathcal{C}_1$ and $y \in \mathcal{C}_2$, we define $\mathsf{pr}_1(x, y) : \mathsf{F}(x) \times \mathsf{G}(y) \to \mathsf{F}(x)$. To prove naturality, we assume that we have morphisms $f : x_1 \to x_2$ and $g : y_1 \to y_2$ and we need to prove the following square commutes.

$$\begin{array}{ccc} \mathsf{F}(x_1) \times \mathsf{G}(y_1) & \xrightarrow{\mathsf{pr}_1} & \mathsf{F}(x_1) \\ \downarrow{\scriptstyle \mathsf{F}(f)\times\mathsf{G}(g)} & & \downarrow{\scriptstyle \mathsf{F}(f)} \\ \mathsf{F}(x_2) \times \mathsf{G}(y_2) & \xrightarrow[\mathsf{pr}_1]{} & \mathsf{F}(x_2) \end{array}$$

This diagram commutes because of how $\mathsf{F}(f) \times \mathsf{G}(g)$ is defined (Example 2.20).

We leave it as an exercise to define the **second projection** $\mathsf{pr}_2 : \mathsf{F} \times_{\mathcal{D}} \mathsf{G} \Rightarrow \mathsf{G} \circ \pi_2$.

3.3 Categories of Functors

Definition 3.21. Let $\mathcal{C}_1$ and $\mathcal{C}_2$ be categories. We define the **functor category** $[\mathcal{C}_1, \mathcal{C}_2]$ as follows.

- Objects of $[\mathcal{C}_1, \mathcal{C}_2]$ are functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$.
- Morphisms from $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$ to $\mathsf{G} : \mathcal{C}_1 \to \mathcal{C}_2$ are natural transformations $\tau : \mathsf{F} \Rightarrow \mathsf{G}$.

The identity morphism is the identity natural transformation (Example 3.12) and composition is the composition of natural transformations (Example 3.13).

Proposition 3.22. *We indeed have a category* $[\mathcal{C}_1, \mathcal{C}_2]$.

Proof. We need to prove associativity and unitality. If we have a natural transformation $\tau : \mathsf{F} \Rightarrow \mathsf{G}$ where $\mathsf{F}, \mathsf{G} : \mathcal{C}_1 \to \mathcal{C}_2$, we need to show that $\tau \circ \mathsf{id}_{\mathsf{F}} = \tau$. To do so, it suffices to show that for each $x : \mathcal{C}_1$ we have $(\tau \circ \mathsf{id}_{\mathsf{F}})(x) = \tau(x)$. Note the following

$$\begin{aligned}(\tau \circ \mathsf{id}_{\mathsf{F}})(x) &= \tau(x) \circ \mathsf{id}_{\mathsf{F}(x)} \\ &= \tau(x)\end{aligned}$$

We prove $\mathsf{id}_{\mathsf{G}} \circ \tau = \tau$ in the same way.

$$\begin{aligned}(\mathsf{id}_{\mathsf{G}} \circ \tau)(x) &= \mathsf{id}_{\mathsf{G}(x)} \circ \tau(x) \\ &= \tau(x)\end{aligned}$$

If we have natural transformations $\tau_1 : \mathsf{F} \Rightarrow \mathsf{G}$ and $\tau_2 : \mathsf{G} \Rightarrow \mathsf{H}$, then we show that $\tau_3 \circ (\tau_2 \circ \tau_1) = (\tau_3 \circ \tau_2) \circ \tau_1$. We take $x : \mathcal{C}_1$ and note that

$$\begin{aligned}(\tau_3 \circ (\tau_2 \circ \tau_1))(x) &= \tau_3(x) \circ (\tau_2 \circ \tau_1)(x) \\ &= \tau_3(x) \circ (\tau_2(x) \circ \tau_1(x)) \\ &= (\tau_3(x) \circ \tau_2(x)) \circ \tau_1(x) \\ &= (\tau_3 \circ \tau_2)(x) \circ \tau_1(x) \\ &= ((\tau_3 \circ \tau_2) \circ \tau_1)(x)\end{aligned}$$

□

Proposition 3.23. *A natural transformation* $\tau : \mathsf{F} \Rightarrow \mathsf{G}$ *is an isomorphism in* $[\mathcal{C}_1, \mathcal{C}_2]$ *if and only if* $\tau(x)$ *is an isomorphism in* $\mathcal{C}_1$ *for each* $x : \mathcal{C}_1$.

Proof. Suppose that $\tau : \mathsf{F} \Rightarrow \mathsf{G}$ is an isomorphism. This means that we have a natural transformation $\theta : \mathsf{G} \Rightarrow \mathsf{F}$ such that $\tau \circ \theta = \mathsf{id}_{\mathsf{G}}$ and $\theta \circ \tau = \mathsf{id}_{\mathsf{F}}$. For each $x \in \mathcal{C}_1$, we thus have

- a morphism $\theta(x) : \mathsf{G}(x) \to \mathsf{F}(x)$
- we have $\tau(x) \circ \theta(x) = \mathsf{id}_{\mathsf{G}(x)}$ since $\tau \circ \theta = \mathsf{id}_{\mathsf{G}}$
- we have $\theta(x) \circ \tau(x) = \mathsf{id}_{\mathsf{F}(x)}$ since $\theta \circ \tau = \mathsf{id}_{\mathsf{F}}$

Hence, $\tau(x)$ is an isomorphism with inverse $\theta(x)$.

Now suppose that each $\tau(x) : \mathsf{F}(x) \to \mathsf{G}(x)$ is an isomorphism, which means that we have a morphism $\tau(x)^{-1} : \mathsf{G}(x) \to \mathsf{F}(x)$ such that

$$\tau(x) \circ \tau(x)^{-1} = \mathsf{id}_{\mathsf{G}(x)}, \qquad \tau(x)^{-1} \circ \tau(x) = \mathsf{id}_{\mathsf{F}(x)}$$

We construct a natural transformation $\theta : \mathsf{G} \Rightarrow \mathsf{F}$ as follows. For each $x \in \mathcal{C}_1$, we define $\theta(x) = \tau(x)^{-1}$. To show that θ is natural, we need to show the following diagram commutes for each $f : x \to y$.

$$\begin{array}{ccc} \mathsf{G}(x) & \xrightarrow{\tau(x)^{-1}} & \mathsf{F}(x) \\ {\scriptstyle \mathsf{G}(f)}\downarrow & & \downarrow{\scriptstyle \mathsf{F}(f)} \\ \mathsf{G}(y) & \xrightarrow[\tau(y)^{-1}]{} & \mathsf{F}(y) \end{array}$$

This is so, because

$$\begin{aligned} \mathsf{F}(f) \circ \tau(x)^{-1} &= \mathsf{id} \circ \mathsf{F}(f) \circ \tau(x)^{-1} \\ &= \tau(y)^{-1} \circ \tau(y) \circ \mathsf{F}(f) \circ \tau(x)^{-1} \\ &= \tau(y)^{-1} \circ \mathsf{G}(f) \circ \tau(x) \circ \tau(x)^{-1} \\ &= \tau(y)^{-1} \circ \mathsf{G}(f) \circ \mathsf{id} \\ &= \tau(y)^{-1} \circ \mathsf{G}(f) \end{aligned}$$

Note that we have $\tau \circ \theta = \mathsf{id}$ and $\theta \circ \tau = \mathsf{id}$. For each $x : \mathcal{C}_1$, we have

$$(\tau \circ \theta)(x) = \tau(x) \circ \tau(x)^{-1} = \mathsf{id}$$

$$(\theta \circ \tau)(x) = \tau(x)^{-1} \circ \tau(x) = \mathsf{id}$$

Hence, τ is an isomorphism in $[\mathcal{C}_1, \mathcal{C}_2]$. □

Definition 3.24. We define the category Cat as follows.

- Objects of Cat are categories
- Morphisms from $\mathcal{C}_1$ to $\mathcal{C}_2$ are functors $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$

We define the identity morphism in Example 3.2 and composition in Example 3.3. We invite the reader to prove that neutrality and associativity laws.

Digression. There are two remarks to make about Definition 3.24. First, in the definition of category we require that there is a class of objects and a class of morphism. To define this category, we need restrict ourselves to **small** categories: categories which have a set of objects and a set of morphisms. This is necessary to guarantee that we have a class of such categories.

Second, there is no mention of natural transformations in Definition 3.24 despite the fact that natural transformations are an essential part of category

theory. This is the reason why one often looks at the **2-category of categories**. While a category has objects and morphisms (for instance, sets and functions), a 2-category has objects, morphisms, and 2-cells (for instance, categories, functors, and natural transformations). Studying categories using 2-categories gives one extra power, and it motivates the study of higher categories. One motivation for using higher categories is that we can express a wider range of universal properties. Universal properties in categories refer to objects and morphisms, whereas universal properties in 2-categories can refer to 2-cells as well.

3.4 Exercises

Exercise 3.1. Let $\mathsf{F} : \mathcal{C} \to \mathcal{D}$ be a functor. If $f : x \to y$ is an isomorphism, then $\mathsf{F}(f)$ is an isomorphism as well.

Exercise 3.2. Finish Example 3.7 by proving the functor laws for $\mathcal{L}_A$.

Exercise 3.3. Finish Example 3.8 by proving the functor laws.

Exercise 3.4. Finish Example 3.10 by defining the functor $\mathsf{F} +_{\mathcal{D}} \mathsf{G}$ and proving that it is indeed a functor.

Exercise 3.5. Complete Example 3.20 by constructing the second projection $\pi_2 : \mathsf{F} \times_{\mathcal{D}} \mathsf{G} \Rightarrow \mathsf{G}$.

Exercise 3.6. Suppose that we have a functor $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$. Construct the functor $\mathsf{F}^{\mathsf{op}} : \mathcal{C}_1^{\mathsf{op}} \to \mathcal{C}_2^{\mathsf{op}}$ and prove that it is indeed a functor.

Exercise 3.7. Suppose that we have a functor $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$, and that we have predicates P_1 on the objects of $\mathcal{C}_1$ and P_2 on the objects of $\mathcal{C}_2$. Construct a functor $\mathsf{Full}(\mathsf{F}) : \mathsf{Full}(\mathcal{C}_1, \mathsf{P}_1, \to)\mathsf{Full}(\mathcal{C}_2, \mathsf{P}_2,)$ whenever we have $\mathsf{P}_2(\mathsf{F}(x))$ if we have $\mathsf{P}_1(x)$.

Exercise 3.8. Suppose that we have a functor $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$. Construct a functor $\mathsf{F}/x : \mathcal{C}_1/x \to \mathcal{C}_2/\mathsf{F}(x)$ for each $x \in \mathcal{C}_1$.

Exercise 3.9. Suppose that we have a functor $\mathsf{F} : \mathcal{C}_1 \to \mathcal{C}_2$. Construct a functor $\mathsf{F}^{\to} : \mathcal{C}_1^{\to} \to \mathcal{C}_2^{\to}$.

Exercise 3.10. Show that Cat has an initial object.

Exercise 3.11. Show that Cat has a terminal object.

Exercise 3.12. Show that Cat has binary products.

Exercise 3.13. Show that Cat has binary coproducts.

4 Algebras and Coalgebras

In this section, we study two notions: **algebras** for a functor and **coalgebras** for functor. As their name suggests, algebras are used in the context of algebra. We use algebras to describe sets with certain operations (without any requirement that some equations are satisfied). Algebras are used in various contexts. In computer science, we can use algebras for a functor to describe data types. The **initial algebra** is of particular importance in that context, because it gives semantics to algebraic data types. In mathematics, we use algebras to describe the operations in some algebraic structure.

An example that we would like to capture, are sets with two operations: a nullary operation and a unary operation. Concretely, we look at sets X together with an element $z \in X$ (nullary operations are the same as elements) and a function $s : X \to X$. We will see in Example 4.5 that we can capture such structures using algebras for a functor. The initial algebra for this functor is the natural numbers, and, if we use Haskell notation, we can describe this initial algebra using the following algebraic data type.

```
data N = Z | S N
```

Coalgebras are the dual notion. While algebras allow us to describe structures by specifying how we can construct elements, coalgebras allow us to describe structures by specifying how we can destruct elements. For instance, coalgebras allow us to describe structures consisting of a set X together with functions $h : X \to A$ and $t : X \to X$. The terminal coalgebra is of particular importance, and the terminal coalgebra for the structure we just describe, is the collection of streams, i.e., infinite lists. In Haskell syntax, we can describe this data type as follows.

```
data Str a = Cons { hd :: a , tl :: Str a }

hd :: Str a → a
hd (Cons a as) = a

tl :: Str a → Str a
tl (Cons a as) = as
```

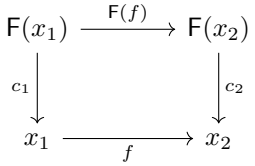
Coalgebras are used in computer science to describe systems by specifying how we can observe the behaviour of the system.

4.1 Algebras for a Functor

Definition 4.1. Let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. An **algebra for** F consists of an object $x : \mathcal{C}$ together with a morphism $c : \mathsf{F}(x) \to x$. We call x the **carrier** of this algebra, and we say that c is an **F-algebra structure for** x.

Definition 4.2. Let (x_1, c_1) and (x_2, c_2) be algebras for F. An **algebra morphism from** (x_1, c_1) **to** (x_2, c_2) consists of a morphism $f : x_1 \to x_2$ such that

the following diagram commutes.

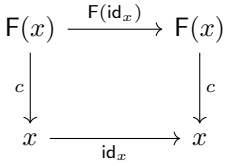


We define the **category Alg_F of algebras for F** as follows.

- objects of Alg_F are algebras for F;
- morphisms from (x_1, c_1) to (x_2, c_2) are algebra morphisms.

Proposition 4.3. *Alg_F is indeed a category.*

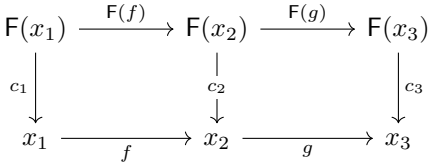
Proof. We define the identity morphism on (x, c) as id_x.



This diagram commutes because

$$\mathsf{id}_x \circ c = c = c \circ \mathsf{id}_{\mathsf{F}(x)} = c \circ \mathsf{F}(\mathsf{id}_x)$$

To define composition, we assume that we have an algebra morphism f from (x_1, c_1) to (x_2, c_2) and an algebra morphism g from (x_2, c_2) to (x_3, c_3). We display them below



We define their composition to be $g \circ f$. Note that $g \circ f$ is indeed an algebra morphism, because

$$\begin{aligned} c_3 \circ \mathsf{F}(g \circ f) &= c_3 \circ \mathsf{F}(g) \circ \mathsf{F}(f) && \text{because } \mathsf{F} \text{ is a functor} \\ &= g \circ c_2 \circ \mathsf{F}(f) && \text{because } g \text{ is an algebra morphism} \\ &= g \circ f \circ c_1 && \text{because } f \text{ is an algebra morphism} \end{aligned}$$

Note that the category laws for Alg_F follow directly from the corresponding laws in $\mathcal{C}$. □

Example 4.4. We have a functor $\mathsf{U}_F : \mathsf{Alg}_\mathsf{F} \to \mathcal{C}$, called the **underlying functor**. It is defined as follows

- $\mathsf{U}_F(x, c) = x$ where (x, c) is an algebra for F
- $\mathsf{U}_F(f) = f$ where f is an algebra morphism from (x_1, c_1) to (x_2, c_2).

One can directly prove that U_F preserves the identity and composition.

To get familiarity with algebras, we look at some examples. We start by looking at algebras for the functor $\mathsf{Maybe} : \mathsf{Set} \to \mathsf{Set}$. Recall that $\mathsf{Maybe}(X)$ is defined to be $X + \mathbb{1}$.

Proposition 4.5. *Every algebras for* Maybe *gives rise to a set X together with an element $z : X$ and a function $s : X \to X$, and vice versa.*

Proof. We can prove this proposition using the universal property of the coproduct (Definition 2.12). Morphisms $f : X + Y \to Z$ are the same as pairs of morphisms $g_1 : X \to Z$ and $g_2 : Y \to Z$. Given $f : X + Y \to Z$, we define $g_1 = f \circ \iota_1$ and $g_2 = f \circ \iota_2$. Given $g_1 : X \to Z$ and $g_2 : Y \to Z$, we define $f = [g_1, g_2]$.

We also prove this proposition more concretely. We show that for every set X we have an isomorphism between the set of functions $c : X + \mathbb{1} \to X$ and the collection of elements $z : X$ and $s : X \to X$.

First, we define a map f that maps pairs of elements $z : X$ and functions $s : X \to X$ to functions $c : X + \mathbb{1} \to X$. We define a function $f(z, s) : X + \mathbb{1} \to X$ as follows

$$f(z, s)(w) = \begin{cases} s(x) & \text{if } w = (0, x) \text{ for some } x \in X \\ z & \text{if } w = (1, *) \end{cases}$$

Next we define a map g that maps functions $c : X + \mathbb{1} \to X$ to elements $z : X$ and functions $s : X \to X$. We write $g_1(c)$ for $c(1, *) : X$, and $g_2(c)$ for the function that maps x to $c(0, x) : X$. The function g maps every c to the pair $(g_1(c), g_2(c))$.

Note that f and g are inverses to each other. Suppose that we have $z : X$ and $s : X \to X$. Then $f(z, s)$ gives us a function then $g(f(z, s))$ is the pair $(g_1(f(z, s)), g_2(f(z, s)))$. Note

$$g_1(f(z, s)) = f(z, s)(1, *) = z$$

$$g_2(f(z, s)))(x) = f(z, s)(0, x) = s(x)$$

Next we assume that we have $c : X + \mathbb{1} \to X$, and any element $w \in X + \mathbb{1}$. There are two cases. If $w = (0, x)$, then we have

$$f(g(c))(0, x) = g_2(c)(x) = c(0, x)$$

$$f(g(c))(1, *) = g_1(c) = c(1, *)$$

Hence, f and g indeed form an isomorphism. □

Example 4.6. We define an algebra for Maybe as follows.

- The carrier is $\mathbb{1}$

- The morphism $!_{\mathsf{Maybe}(\mathbb{1})} : \mathsf{Maybe}(\mathbb{1}) \to \mathbb{1}$.

We denote this algebra by $\mathbb{1}_{\mathsf{Maybe}}$.

Proposition 4.7. *The algebra $\mathbb{1}_{\mathsf{Maybe}}$ is terminal.*

Proof. Let (x, c) be any algebra for Maybe. Our goal is to show that there is a unique morphism $f : x \to \mathbb{1}$ making the following diagram commute.

$$\begin{array}{ccc} \mathsf{Maybe}(x) & \xrightarrow{\mathsf{Maybe}(f)} & \mathsf{Maybe}(\mathbb{1}) \\ {\scriptstyle c}\downarrow & & \downarrow{\scriptstyle !_{\mathsf{Maybe}(\mathbb{1})}} \\ x & \xrightarrow[f]{} & \mathbb{1} \end{array}$$

We first note that since $\mathbb{1}$ is the terminal object in Set (Example 2.3), all morphisms $f, g : x \to \mathbb{1}$ are equal. Hence, it suffices to show that we have a morphism $f : x \to \mathbb{1}$. We define f to be $!_x$. To show that $f = !_x$ is a morphism of algebras, we need to show that the following diagram commutes.

$$\begin{array}{ccc} \mathsf{Maybe}(x) & \xrightarrow{\mathsf{Maybe}(!_x)} & \mathsf{Maybe}(\mathbb{1}) \\ {\scriptstyle c}\downarrow & & \downarrow{\scriptstyle !_{\mathsf{Maybe}(\mathbb{1})}} \\ x & \xrightarrow[!_x]{} & \mathbb{1} \end{array}$$

Concretely, we need to show that the morphism $\mathsf{s}!_{\mathsf{Maybe}(\mathbb{1})} \circ \mathsf{Maybe}(!_x)$ and $!_x \circ c$ are equal. Since both are morphisms from $\mathsf{Maybe}(x)$ to $\mathbb{1}$ and since $\mathbb{1}$ is terminal, they must be equal. □

Example 4.8. Let $k \in \mathbb{N}$ be any natural number with $k > 0$, and define $\mathbb{N}_k$ to be the set $\{i \in \mathbb{N} | i < k\}$. We construct an algebra for Maybe whose carrier is $\mathbb{N}_k$. To do so, we use Proposition 4.5. We define the map $s : \mathbb{N}_k \to \mathbb{N}_k$ as follows

$$s(i) = \begin{cases} i+1 & \text{if } i+1 < k \\ 0 & \text{otherwise} \end{cases}$$

We define $z \in \mathbb{N}_k$ to be 0. We denote this algebra by $(\mathbb{N}_k, s, 0)$.

Example 4.9. We construct two Maybe-algebra structures for $\mathbb{Z}$. Note that we have a map $\mathsf{S}_{\mathbb{Z}} : \mathbb{Z} \to \mathbb{Z}$, which maps x to $x + 1$, and a map $\mathsf{P}_{\mathbb{Z}} : \mathbb{Z} \to \mathbb{Z}$, which maps x to $x - 1$. From these maps we get two Maybe-algebra structures, namely $(\mathbb{Z}, \mathsf{S}_{\mathbb{Z}}, 0)$ and $(\mathbb{Z}, \mathsf{P}_{\mathbb{Z}}, 0)$. Note that there are many other possibilities as well, for instance $(\mathbb{Z}, \mathsf{S}_{\mathbb{Z}}, 37)$.

Example 4.10. We have the following Maybe-algebra: $(\mathbb{N}, \mathsf{S}, 0)$. Here $\mathsf{S}(n) = n + 1$.

Proposition 4.11. $(\mathbb{N}, \mathsf{S}, 0)$ *is initial.*

Proof. Suppose that we have any algebra (X, s, z). Our goal is to show that we have a unique morphism $f : \mathbb{N} \to X$ that makes the following diagram commute.

$$\begin{array}{ccc} \mathbb{N} + \mathbb{1} & \xrightarrow{f+\mathsf{id}_{\mathbb{1}}} & X + \mathbb{1} \\ {\scriptstyle [\mathsf{S},0]}\downarrow & & \downarrow{\scriptstyle [s,z]} \\ \mathbb{N} & \xrightarrow[f]{} & X \end{array}$$

We define f using the following recursive definition.

$$f(n) = \begin{cases} z(*) & \text{if } n = 0 \\ s(f(k)) & \text{if } n = k + 1 \end{cases}$$

Note that this f makes the diagram above commute. This is because

$$\begin{aligned} [s, z]((f + \mathsf{id}_{\mathbb{1}})(0, n)) &= [s, z](0, f(n)) \\ &= s(f(n)) \\ &= f(\mathsf{S}(n)) \\ &= f([\mathsf{S}, 0](0, n)) \end{aligned}$$

$$\begin{aligned} [s, z]((f + \mathsf{id}_{\mathbb{1}})(1, *)) &= [s, z](1, *) \\ &= z(*) \\ &= f(0) \\ &= f([\mathsf{S}, 0](1, *)) \end{aligned}$$

Now let $g : \mathbb{N} \to X$ be any other algebra morphism. We show by induction that for all $n \in \mathbb{N}$ we have $g(n) = f(n)$. For the base case, we take $n = 0$, and we note

$$\begin{aligned} f(0) &= z(*) \\ &= [s, z](1, *) \\ &= [s, z]((g + \mathsf{id}_{\mathbb{1}})(1, *)) \\ &= ([s, z] \circ (g + \mathsf{id}_{\mathbb{1}}))(1, *) \\ &= (g \circ [\mathsf{S}, 0])(1, *) \\ &= g([\mathsf{S}, 0](1, *)) \\ &= g(0) \end{aligned}$$

Here we use that g is an algebra morphism meaning that $[\mathsf{S}, 0] \circ (g + \mathsf{id}_{\mathbb{1}}) = g \circ [\mathsf{S}, 0]$.

For the inductive case, we assume that $g(n) = f(n)$, and we need to show

that $g(n+1) = f(n+1)$. This follows from the following

$$\begin{aligned} f(n+1) &= s(f(n)) \\ &= s(g(n)) \\ &= [s, z](0, g(n)) \\ &= [s, z]((g + \mathsf{id}_{\mathbb{1}})(0, n)) \\ &= ([s, z] \circ (g + \mathsf{id}_{\mathbb{1}}))(0, n) \\ &= g([\mathsf{S}, 0](0, n)) \\ &= g(n+1) \end{aligned}$$

Hence $g = f$, and thus $(\mathbb{N}, \mathsf{S}, 0)$ is initial. □

Digression. The proof of Proposition 4.11 is interesting, and explains the relevance of initial algebras. Using that $(\mathbb{N}, \mathsf{S}, 0)$ is initial, we can define certain recursive functions using $\mathbb{N}$. Specifically, initiality for Maybe captures the following recursive definition for functions $f : \mathbb{N} \to X$.

$$f(n) = \begin{cases} z & \text{if } n = 0 \\ s(f(k)) & \text{if } n = k + 1 \end{cases}$$

Here $z \in X$ is some fixed element and $s : X \to X$ is a map. This recursive definition is the **iteration principle** for the natural numbers, which intuitively says that we can define functions $f : \mathbb{N} \to X$ by iterating some given function $s : X \to X$ on some input $z \in X$.

From the perspective of programming, initial algebras allow us to describe inductive data types: we describe the constructor using a functor, and initiality gives us a recursion principle.

Next we look at algebra for the functor $\mathcal{L}_A$. Given a set A, recall that $\mathcal{L}_A(X)$ is defined to be $X \times A + \mathbb{1}$. We can characterise algebras for $\mathcal{L}_A$ as follows.

Proposition 4.12. *Algebras for $\mathcal{L}_A$ are the same as sets X together with an element $n : X$ and a function $c : X \times A \to X$.*

Proof. Proven in the same way as Proposition 4.5. □

Example 4.13. We construct a $\mathcal{L}_A$-algebra. Its carrier is $\mathbb{N}$, we take $n : \mathbb{N}$ to be 0, and we define $c : \mathbb{N} \times A \to \mathbb{N}$ as follows

$$c(n, a) = \mathsf{S}(n)$$

Example 4.14. We consider the set $\mathsf{List}(A)$ of lists with elements from A.

$$\mathsf{List}(A) = \{(a_1, \ldots, a_n) \mid a_i \in A\}$$

Note that n could be 0 meaning that we have an element $() \in \mathsf{List}(A)$, and we denote this element by Nil. We construct an $\mathcal{L}_A$-algebra structure on $\mathsf{List}(A)$.

The element $n : \mathsf{List}(A)$ is defined to be Nil, and the operation $c : \mathsf{List}(A) \times A \to \mathsf{List}(A)$ is defined as the following function

$$\mathsf{Cons}((a_1, \ldots, a_n), b) = (b, a_1, \ldots, a_n)$$

We denote this algebra by $(\mathsf{List}(A), \mathsf{Nil}, \mathsf{Cons})$.

Proposition 4.15. $(\mathsf{List}(A), \mathsf{Nil}, \mathsf{Cons})$ *is initial.*

Proof. Let (X, n, c) be any $\mathcal{L}_A$-algebra. We need to show that there is a unique morphism f that makes the following diagram commute.

$$\begin{array}{ccc} \mathsf{List}(A) \times A + \mathbb{1} & \xrightarrow{(f \times \mathsf{id}_A) + \mathsf{id}_{\mathbb{1}}} & X \times A + \mathbb{1} \\ \Big\downarrow{\scriptstyle [\mathsf{Cons},\mathsf{Nil}]} & & \Big\downarrow{\scriptstyle [c,n]} \\ \mathsf{List}(A) & \xrightarrow[f]{} & X \end{array}$$

To do so, we first note that for every $l \in \mathsf{List}(A)$ we either have that $l = \mathsf{Nil}$ or there is $l' \in \mathsf{List}(A)$ and $a \in A$ such that $l = \mathsf{Cons}(l', a)$. We define f using induction over lists.

$$f(l) = \begin{cases} n & \text{if } l = \mathsf{Nil} \\ c(f(l'), a) & \text{if } l = \mathsf{Cons}(l', a) \end{cases}$$

We leave it as an exercise to the reader to prove that f is an $\mathcal{L}_A$-algebra morphism.

Next we show that f is the unique algebra morphism with this property. Let $g : \mathsf{List}(A) \to X$ be any other $\mathcal{L}_A$-algebra morphism. We need to show that for each $l \in \mathsf{List}(A)$, we have $g(l) = f(l)$, and to do so, we use induction over lists. The base is $l = \mathsf{Nil}$, and then we have

$$f(\mathsf{Nil}) = n = g(\mathsf{Nil})$$

We have $g(\mathsf{Nil}) = n$, because g is an $\mathcal{L}_A$-algebra morphism.

For the induction step, we assume that we have a list $l' \in \mathsf{List}(A)$ with $g(l') = f(l')$ and $a \in A$. We need to show

$$\begin{aligned} f(\mathsf{Cons}(l', a)) &= c(f(l'), a) \\ &= c(g(l'), a) \\ &= g(\mathsf{Cons}(l', a)) \end{aligned}$$

Since g is an algebra morphism, we indeed have $c(g(l'), a) = g(\mathsf{Cons}(l', a))$. □

Example 4.16. We can combine Proposition 4.15 and Example 4.13 to obtain a morphism $\mathsf{length} : \mathsf{List}(A) \to \mathbb{N}$. By inspecting the proof of Proposition 4.15, we see that length is defined as follows.

$$\mathsf{length}(l) = \begin{cases} 0 & \text{if } l = \mathsf{Nil} \\ \mathsf{length}(l') + 1 & \text{if } l = \mathsf{Cons}(l', a) \end{cases}$$

Hence, the function length calculates the length of a list.

4.2 Coalgebras for a Functor

Definition 4.17. Let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. A **coalgebra for** F consists of an object $x : \mathcal{C}$ together with a morphism $d : x \to \mathsf{F}(x)$. We call x the **carrier** of this coalgebra, and we say that d is an **F-coalgebra structure for** x.

Let (x_1, d_1) and (x_2, d_2) be coalgebras for F. A **coalgebra morphism from** (x_1, d_1) **to** (x_2, d_2) consists of a morphism $f : x_1 \to x_2$ such that the following diagram commutes.

$$\begin{array}{ccc} x_1 & \xrightarrow{\quad f \quad} & x_2 \\ {\scriptstyle d_1}\downarrow & & \downarrow{\scriptstyle d_2} \\ \mathsf{F}(x_1) & \xrightarrow[\mathsf{F}(f)]{} & \mathsf{F}(x_2) \end{array}$$

We define the **category $\mathsf{CoAlg}_\mathsf{F}$ of coalgebras for F** as follows.

- objects of $\mathsf{CoAlg}_\mathsf{F}$ are coalgebras for F;
- morphisms from (x_1, d_1) to (x_2, d_2) are coalgebra morphisms.

Example 4.18. We have a functor $\mathsf{U}_F : \mathsf{CoAlg}_\mathsf{F} \to \mathcal{C}$, called the **underlying functor**. It is defined as follows

- $\mathsf{U}_F(x, c) = x$ where (x, c) is a coalgebra for F
- $\mathsf{U}_F(f) = f$ where f is a coalgebra morphism from (x_1, c_1) to (x_2, c_2).

One can directly prove that U_F preserves the identity and composition.

Example 4.19. Let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. This functor F induces a functor $\mathsf{F}^{\mathsf{op}} : \mathcal{C}^{\mathsf{op}} \to \mathcal{C}^{\mathsf{op}}$ (Exercise 3.6). A coalgebra for F^{op} consists of an object $x : \mathcal{C}$ together with a morphism $d : x \to_{\mathcal{C}^{\mathsf{op}}} \mathsf{F}(x)$, and d gives a morphism $d : \mathsf{F}(x) \to_{\mathcal{C}} x$. Hence, coalgebras for F^{op} are the same as algebras for F.

As a concrete example, we look at coalgebras for the functor $(-) \times A : \mathsf{Set} \to \mathsf{Set}$ (Example 3.6), which was defined to be $((-) \times A)(X) = X \times A$. Note that coalgebras for $(-) \times A$ consist of a set X together with two maps $t : X \to X$ and $h : X \to A$.

Example 4.20. We show that we have a coalgebra for $(-) \times A$ whose carrier is A. We need to construct a $t : A \to A$ and $h : A \to A$, and for both we take the identity id_A. We denote this coalgebra by $(A, \mathsf{id}_A, \mathsf{id}_A)$.

Example 4.21. We define the set $\mathsf{Stream}(A)$ to be the set of all functions $f : \mathbb{N} \to A$. The function $\mathsf{Head} : \mathsf{Stream}(A) \to A$ maps $f : \mathbb{N} \to A$ to $f(0) : A$. Given $f : \mathbb{N} \to A$, we define $\mathsf{Tail}(f) : \mathbb{N} \to A$ to be $\mathsf{Tail}(f)(n) = f(n + 1)$. This coalgebra is denoted by $(\mathsf{Stream}(A), \mathsf{Head}, \mathsf{Tail})$.

Proposition 4.22. $(\mathsf{Stream}(A), \mathsf{Head}, \mathsf{Tail})$ *is terminal.*

Proof. Let (X, h, t) be any coalgebra for $(-) \times A$. We need to show there is a unique function $f : X \to \mathsf{Stream}(A)$ that makes the following diagram commute.

$$\begin{array}{ccc} X & \xrightarrow{f} & \mathsf{Stream}(A) \\ {\scriptstyle \langle t,h \rangle}\downarrow & & \downarrow{\scriptstyle \langle \mathsf{Tail},\mathsf{Head} \rangle} \\ X \times A & \xrightarrow[f \times \mathsf{id}_A]{} & \mathsf{Stream}(A) \times A \end{array}$$

We define f as follows

$$f(x)(n) = h(t^n(x))$$

Note

$$\mathsf{Head}(f(x)) = f(x)(0) = h(x)$$
$$\mathsf{Tail}(f(x))(n) = f(x)(n+1) = h(t^{n+1}(x)) = f(t(x))(n)$$

The uniqueness of f is left as an exercise. □

Example 4.23. We can combine Proposition 4.22 and 4.20 to obtain a function $\mathsf{const}_{\mathsf{Stream}} : A \to \mathsf{Stream}(A)$. By unfolding the description given in the proof of Proposition 4.22, we see that $\mathsf{const}_{\mathsf{Stream}}$ maps every a to as follows

$$\mathsf{const}_{\mathsf{Stream}}(a)(n) = \mathsf{id}(\mathsf{id}^n(a)) = a$$

Hence, this function maps every a to the constant function.

Digression. We discussed that initial algebras allow us to view inductive data types categorically. The constructors are specified using the functor F and initiality gives us a recursion principle. With terminal coalgebras one can study coinductive data types. Coinductive types are specified using destructors rather than constructors. For instance, the type $\mathsf{Stream}(A)$ of streams is specified by two destructors. We have $\mathsf{Head} : \mathsf{Stream}(A) \to A$ that picks the first element and $\mathsf{Tail} : \mathsf{Stream}(A) \to \mathsf{Stream}(A)$ that removes the first element from the stream. Terminality allows us to define streams using corecursion.

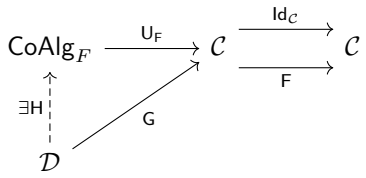
Digression. One nice aspect of studying Cat as a 2-category is that we can characterise certain categorical constructions via 2-categorical universal properties. In particular, we can characterise Alg_F and CoAlg_F via a 2-categorical universal property.

$$\mathsf{Alg}_F \xrightarrow{\mathsf{U_F}} \mathcal{C} \underset{\mathsf{Id}_{\mathcal{C}}}{\overset{\mathsf{F}}{\rightrightarrows}} \mathcal{C}$$

We can construct a natural transformation $\tau : \mathsf{F} \circ \mathsf{U_F} \Rightarrow \mathsf{Id}_{\mathcal{C}} \circ \mathsf{U_F}$. Given an algebra (X, c), we define $\tau(X, c) : F(X) \to X$ to be c. The universal property of $\mathsf{Alg_F}$ allows us to make functors $\mathsf{H} : \mathcal{D} \to \mathsf{Alg_F}$) in the following way.

$$\begin{array}{ccccc} \mathsf{Alg_F} & \xrightarrow{\mathsf{U_F}} & \mathcal{C} & \underset{\mathsf{Id}_{\mathcal{C}}}{\overset{\mathsf{F}}{\rightrightarrows}} & \mathcal{C} \\ {\scriptstyle \exists \mathsf{H}}\uparrow & \nearrow_{\mathsf{G}} & & & \\ \mathcal{D} & & & & \end{array}$$

For coalgebras, we have a similar universal property.



4.3 Exercises

Exercise 4.1. Prove Proposition 4.12.

Exercise 4.2. Show that CoAlg_F is a category by constructing the identity morphisms and composition, and by showing the laws.

Exercise 4.3. Finish the proof of Proposition 4.22 by proving that f is the unique coalgebra.

Exercise 4.4. Let $\mathcal{C}$ be a category with a terminal object, and let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. Show that Alg_F has a terminal object.

Exercise 4.5. Let $\mathcal{C}$ be a category with binary products, and let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. Show that Alg_F has binary products.

Exercise 4.6. Let $\mathcal{C}$ be a category with an initial object, and let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. Show that $\mathsf{CoAlg}_\mathsf{F}$ has a terminal object.

Exercise 4.7. Let $\mathcal{C}$ be a category with binary coproducts, and let $\mathsf{F} : \mathcal{C} \to \mathcal{C}$ be a functor. Show that $\mathsf{CoAlg}_\mathsf{F}$ has binary coproducts.

Exercise 4.8. Let $a : A$. Use Proposition 4.15 to construct a map $\mathsf{const}_{\mathsf{List}} : \mathbb{N} \to \mathsf{List}(A)$ that maps every n to the list $(a, \ldots, a)$ with length n. Prove that $\mathsf{length} \circ \mathsf{const}_{\mathsf{List}} = \mathsf{id}_{\mathbb{N}}$.

Exercise 4.9. Let $f : A \to B$ be a function. Use Proposition 4.15 to construct a map $\mathsf{List}(f) : \mathsf{List}(A) \to \mathsf{List}(B)$. Prove that $\mathsf{List}(\mathsf{id}_A) = \mathsf{id}_{\mathsf{List}(A)}$ and that $\mathsf{List}(g \circ f) = \mathsf{List}(g) \circ \mathsf{List}(f)$.

Exercise 4.10. Using Proposition 4.22 to define a function $o : \mathsf{Stream}(A) \to \mathsf{Stream}(A)$ that maps $f : \mathbb{N} \to A$ to the function $g(n) = f(2 \cdot n + 1)$.

Exercise 4.11. Let $f : A \to B$ be a function. Use Proposition 4.22 to construct a map $\mathsf{Stream}(f) : \mathsf{Stream}(A) \to \mathsf{Stream}(B)$. Prove that $\mathsf{Stream}(\mathsf{id}_A) = \mathsf{id}_{\mathsf{Stream}(A)}$ and that $\mathsf{Stream}(g \circ f) = \mathsf{Stream}(g) \circ \mathsf{Stream}(f)$.

5 Adjunctions

Adjunctions provide a powerful tool to describe universal properties in category theory. They provide us with a large amount of generality: we can describe, for instance, terminal objects, binary products, initial objects, and binary coproducts using adjunction. But there is more: we can also give a nice universal property for $\mathsf{List}(A)$. In Example 4.14, we describe a universal property for $\mathsf{List}(A)$ by fixing the set A. For each set A, we gave a functor $\mathcal{L}_A$, and we proved that $\mathsf{List}(A)$ is an initial algebra.

With adjunctions, we can describe a more uniform universal property for lists (Example 5.6). Specifically, we see the set $\mathsf{List}(A)$ as the "free monoid" on A. This means, concretely, that to make a map r from $\mathsf{List}(A)$ to X, we need to give

- $f : A \to X$, which describes the image of lists of a single element
- $n : X$, which describes the image of the empty list
- $a : X \times X \to X$, which describes the image of appended list

Hence, we can make map from $\mathsf{List}(A)$ to any monoid X by giving a map $f : A \to X$. We obtain this universal by constructing an adjunction. In the context of Haskell, this universal property corresponds to the fact that lists form an instance of the Foldable class.

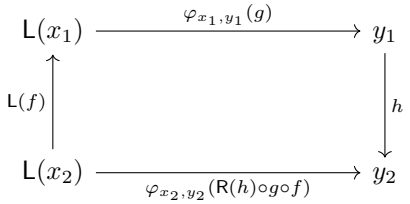
5.1 Adjunctions and Examples

Definition 5.1. Let $\mathcal{C}$ and $\mathcal{D}$ be categories, and suppose that we have functors $\mathsf{L} : \mathcal{C} \to \mathcal{D}$ and $\mathsf{R} : \mathcal{D} \to \mathcal{C}$. We say that L **is left adjoint to** R and that R **is right adjoint to** L, denoted $\mathsf{L} \dashv \mathsf{R}$, whenever for all $x : \mathcal{C}$ and $y : \mathcal{D}$ we have functions

- $\varphi_{x,y}$ that maps $g : x \to_{\mathcal{C}} \mathsf{R}(y)$ to $\varphi_{x,y}(g) : \mathsf{L}(x) \to_{\mathcal{D}} y$;
- $\psi_{x,y}$ that maps $g : \mathsf{L}(x) \to_{\mathcal{D}} y$ to $\psi_{x,y}(g) : x \to_{\mathcal{C}} \mathsf{R}(y)$

such that

- for all $g : x \to_{\mathcal{C}} \mathsf{R}(y)$, we have $\psi_{x,y}(\varphi_{x,y}(g)) = g$
- for all $g : \mathsf{L}(x) \to_{\mathcal{D}} y$, we have $\varphi_{x,y}(\psi_{x,y}(g)) = g$
- for all $f : x_2 \to_{\mathcal{C}} x_1$, $g : x_1 \to_{\mathcal{C}} \mathsf{R}(y_1)$, and $h : y_1 \to_{\mathcal{D}} y_2$, the following diagram commutes



Note that we have the following composition.

$$x_2 \xrightarrow{f} x_1 \xrightarrow{g} \mathsf{R}(y_1) \xrightarrow{\mathsf{R}(h)} \mathsf{R}(y_2)$$

Often we draw adjunctions as follows.

$$\mathcal{C} \underset{\mathsf{R}}{\overset{\mathsf{L}}{\rightleftarrows}} \mathcal{D} \quad (\perp)$$

Adjunctions give us a general method to describe universal properties. If we have $\mathsf{L} \dashv \mathsf{R}$, then we make a map $\mathsf{L}(x) \to y$ from a map $x \to \mathsf{R}(y)$ and vice versa. In the following examples, we see concrete examples of such universal properties.

Example 5.2. Let $\mathcal{C}$ be a category. Recall that we have a functor $\mathsf{Id}_\mathcal{C} : \mathcal{C} \to \mathcal{C}$ that maps objects x to x. We show that $\mathsf{Id}_\mathcal{C} \dashv \mathsf{Id}_\mathcal{C}$. We define

- given $g : x \to_\mathcal{C} \mathsf{Id}_\mathcal{C}(y)$, we define $\varphi_{x,y}(g)$ to be $g : x \to_\mathcal{C} y$ (note that we have $g : \mathsf{Id}_\mathcal{C}(x) \to_\mathcal{C} y$ since $\mathsf{Id}_\mathcal{C}(x) = x$);
- given $g : \mathsf{Id}_\mathcal{C}(x) \to_\mathcal{C} y$, we define $\psi_{x,y}(g)$ to be $g : x \to_\mathcal{C} \mathsf{Id}_\mathcal{C}(y)$.

If we have $g : x \to_\mathcal{C} \mathsf{Id}_\mathcal{C}(y)$, then

$$\psi_{x,y}(\varphi_{x,y}(g)) = \psi_{x,y}(g) = g,$$

and if we have $g : \mathsf{Id}_\mathcal{C}(x) \to_\mathcal{C} y$, then

$$\varphi_{x,y}(\psi_{x,y}(g)) = \varphi_{x,y}(g) = g$$

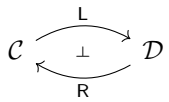
Finally, suppose we have $f : x_2 \to_\mathcal{C} x_1$, $g : x_1 \to_\mathcal{C} \mathsf{Id}_\mathcal{C}(y_1)$, and $h : y_1 \to_\mathcal{C} y_2$. By unfolding the definitions, we see that $\varphi_{x_1,y_1}(g) = g$, $\mathsf{Id}_\mathcal{C}(f) = f$, and that $\varphi_{x_2,y_2}(\mathsf{Id}_\mathcal{C}(h) \circ g \circ f) = h \circ g \circ f$. Hence, it suffices that the following diagram commutes

$$\begin{array}{ccc} x_1 & \xrightarrow{g} & y_1 \\ {\scriptstyle f}\uparrow & & \downarrow{\scriptstyle h} \\ x_2 & \xrightarrow[h \circ g \circ f]{} & y_2 \end{array}$$

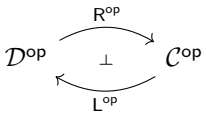
This diagram commutes by definition.

We saw the concept of dualisation before: categorical notions are phrased using objects and morphisms, and we can dualise categorical notions by reversing the morphisms. For instance, we saw the notions of terminal object (Definition 2.1) and of initial object (Definition 2.2). These notions are dual in the sense that initial objects in $\mathcal{C}^{\mathsf{op}}$ are the same as terminal objects in $\mathcal{C}$ (Proposition 2.4). Adjunctions can also be dualised.

Example 5.3. Suppose that we have an adjunction $\mathsf{L} \dashv \mathsf{R}$ as displayed below.



We show that we have an adjunction $\mathsf{R}^{\mathsf{op}} \dashv \mathsf{L}^{\mathsf{op}}$ as



Let $x : \mathcal{D}$ and $y : \mathcal{C}$, and suppose we have a morphism $g : x \to_{\mathcal{D}^{\mathsf{op}}} \mathsf{L}^{\mathsf{op}}(y)$, which is the same as a morphism $g : \mathsf{L}(y) \to_{\mathcal{D}} x$. We define $\varphi^{\mathsf{op}}_{x,y}(g)$ to be $\psi_{y,x}(g)$.

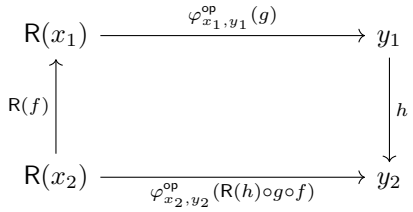
Suppose we have a morphism $g : \mathsf{R}^{\mathsf{op}}(x) \to_{\mathcal{C}^{\mathsf{op}}} y$, which is the same as a morphism $g : y \to_{\mathcal{C}} \mathsf{R}(x)$. We define $\psi^{\mathsf{op}}_{x,y}(g)$ to be $\varphi_{y,x}(g)$.

Now we verify the required laws for adjunctions. Note that

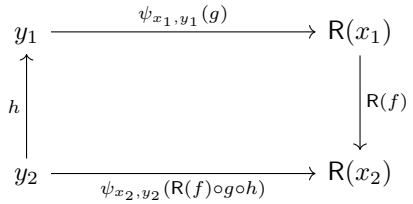
$$\begin{aligned}\varphi^{\mathsf{op}}_{x,y}(\psi^{\mathsf{op}}_{x,y}(g)) &= \psi_{y,x}(\varphi_{y,x}(g)) \\ &= g\end{aligned}$$

$$\begin{aligned}\psi^{\mathsf{op}}_{x,y}(\varphi^{\mathsf{op}}_{x,y}(g)) &= \varphi_{y,x}(\psi_{y,x}(g)) \\ &= g\end{aligned}$$

Let $f : x_2 \to_{\mathcal{D}^{\mathsf{op}}} x_1$, $g : x_1 \to_{\mathcal{D}^{\mathsf{op}}} \mathsf{L}(y_1)$, and $h : y_1 \to_{\mathcal{C}^{\mathsf{op}}} y_2$ be morphisms. We need to show that the following diagram commutes.

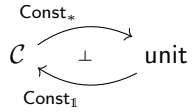


By definition of the opposite category, we have $f : x_1 \to_{\mathcal{D}} x_2$, $g : \mathsf{L}(y_1) \to_{\mathcal{D}} x_1$, and $h : y_2 \to_{\mathcal{C}} y_1$, and we must show the following diagram commutes



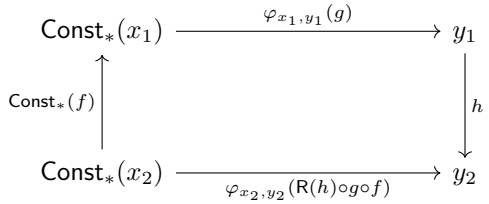
We delay the proof of this equation to Proposition 5.7.

Example 5.4. Let $\mathcal{C}$ be a category with a terminal object $\mathbb{1}$. We show that the functor $\mathsf{Const}_* : \mathcal{C} \to \mathsf{unit}$ has a right adjoint. Specifically, we show that $\mathsf{Const}_* \dashv \mathsf{Const}_{\mathbb{1}}$ as displayed below.



We construct the maps $\varphi_{x,*}$ and $\psi_{x,*}$ for all $x : \mathcal{C}$. Given $g : x \to_{\mathcal{C}} \mathbb{1}$, we define $\varphi_{x,*}(g) : * \to_{\mathbb{1}} *$ to be the identity. Given $g : * \to_{\mathbb{1}} *$ (which is the identity), we define $\psi_{x,*}(g) : x \to_{\mathcal{C}} \mathbb{1}$ to be $!_x$.

Next we show the desired laws. Note that $\varphi_{x,*}(\psi_{x,*}(g)) = g$ since both are morphisms in unit. To show that $\psi_{x,*}(\varphi_{x,*}(g)) = g$, we note that both are morphisms to a terminal object and hence they are equal. Finally, we assume that we have $f : x_2 \to_{\mathcal{C}} x_1$, $g : x_1 \to_{\mathcal{C}} \mathbb{1}$, and $h : * \to_{\mathsf{unit}} *$, and we need to show that the following diagram commutes.

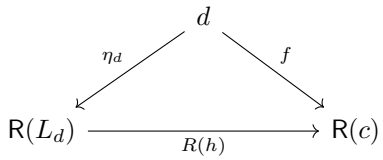


Since this is a diagram in unit, it must commute. Hence, we have an adjunction.

Proposition 5.5. *Let $\mathsf{R} : \mathcal{C} \to \mathcal{D}$ be a functor. Then R has a left adjoint if we have*

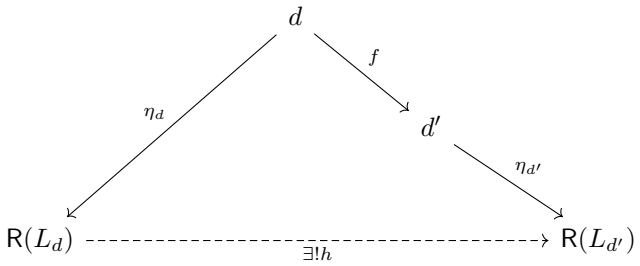
- *for each $d \in D$ an object $L_d \in \mathcal{C}$*
- *for each $d \in D$ a morphism $\eta_d : d \to_{\mathcal{D}} \mathsf{R}(L_d)$*

such that for all $c \in C$ and $f : d \to_{\mathcal{D}} \mathsf{R}(c)$ there is a unique $h : L_d \to c$ making the following triangle commute.



Proof. We only show how the functor $\mathsf{L} : \mathcal{D} \to \mathcal{C}$ is constructed. Given $d \in \mathcal{D}$, we define $\mathsf{L}(d)$ to be L_d. If we have a morphism $f : d \to d'$, then we have a

morphism h due to the universal property described in the proposition.



We define $\mathsf{L}(f)$ to be h. □

Example 5.6. We show that the functor $\mathsf{U}_{\mathsf{Mon}} : \mathsf{Mon} \to \mathsf{Set}$ has a left adjoint. To do so, we use Proposition 5.5.

Let $X \in \mathsf{Set}$ be any set. We define G_X to be the type of lists of X (see Example 4.14). Note that G_X is indeed a monoid: if we have two lists $(a_1, \ldots, a_n)$ and $(b_1, \ldots, b_m)$, then we define their multiplication to be their concatenation $(a_1, \ldots, a_n, b_1, \ldots, b_m)$. Concatenation is associative and the empty list is a neutral element. The function $\eta_X : X \to_{\mathcal{D}} \mathsf{U}_{\mathsf{Mon}}(G_X)$ sends every $x \in X$ to the list (x) with a single element.

Now we verify the universal property. Let M be a monoid and let $f : X \to M$ be a function. We define $h : G_X \to M$ as follows

$$h(a_1, \ldots, a_n) = f(a_1) \cdot \ldots \cdot f(a_n)$$

We define $h() = e$ where () is the empty list. Note that the desired triangle commutes, because

$$h(\eta_X(x)) = h(x) = f(x)$$

In addition, h is a homomorphisms. Since $h() = e$, we only needs to verify that h preserves composition.

$$\begin{aligned} h(a_1, \ldots, a_n, b_1, \ldots, b_m) &= f(a_1) \cdot \ldots \cdot f(a_n) \cdot f(b_1) \cdot \ldots \cdot f(b_m) \\ &= h(a_1, \ldots a_n) \cdot h(b_1, \ldots, b_m) \end{aligned}$$

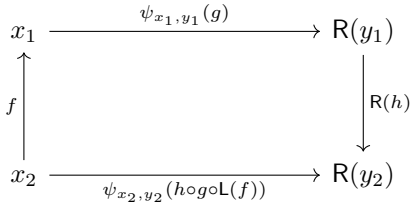
Note that here we use that multiplication is associative.

We also note that h is unique with this property. Let $h' : G_X \to M$ be any homomorphism such that $h'(\eta_X(x)) = f(x)$. Then we have

$$\begin{aligned} h(a_1, \ldots, a_n) &= f(a_1) \cdot \ldots \cdot f(a_n) \\ &= h'(\eta_X(a_1)) \cdot \ldots \cdot h'(\eta_X(a_n)) \\ &= h'(\eta_X(a_1) \cdot \ldots \cdot \eta_X(a_n)) \\ &= h'(a_1, \ldots a_n) \end{aligned}$$

5.2 Adjunctions via Units and Counits

Proposition 5.7. *Suppose* $\mathsf{L} \dashv \mathsf{R}$ *is an adjunction where* $\mathsf{L} : \mathcal{C} \to \mathcal{D}$*. Then for all* $f : x_2 \to_{\mathcal{C}} x_1$*,* $g : \mathsf{L}(x_1) \to_{\mathcal{D}} y_1$*, and* $h : y_1 \to_{\mathcal{D}} y_2$*, the following diagram commutes*



Here we use the following composition.

$$\mathsf{L}(x_2) \xrightarrow{\mathsf{L}(f)} \mathsf{L}(x_1) \xrightarrow{g} y_1 \xrightarrow{h} y_2$$

Proof. Let $f : x_2 \to_{\mathcal{C}} x_1$, $g : \mathsf{L}(x_1) \to_{\mathcal{D}} y_1$, and $h : y_1 \to_{\mathcal{D}} y_2$ be morphisms. We need to show that

$$\psi_{x_2,y_2}(h \circ g \circ \mathsf{L}(f)) = \mathsf{R}(h) \circ \psi_{x_1,y_1}(g) \circ f$$

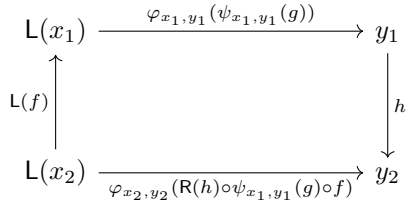
By the inverse laws, we have

$$\mathsf{R}(h) \circ \psi_{x_1,y_1}(g) \circ f = \psi_{x_2,y_2}(\varphi_{x_2,y_2}(\mathsf{R}(h) \circ \psi_{x_1,y_1}(g) \circ f))$$

By cancelling ψ_{x_2,y_2}, it suffices to show

$$h \circ g \circ \mathsf{L}(f) = \varphi_{x_2,y_2}(\mathsf{R}(h) \circ \psi_{x_1,y_1}(g) \circ f)$$

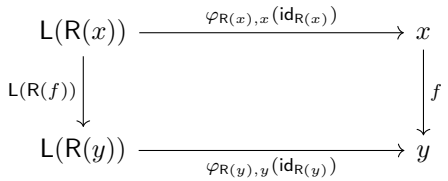
From the definition of adjunctions, we know that the following diagram commutes



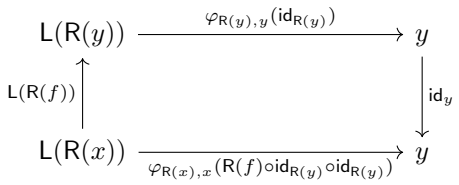
Since $\varphi_{x_1,y_1}(\psi_{x_1,y_1}(g)) = g$, the desired statement follows. □

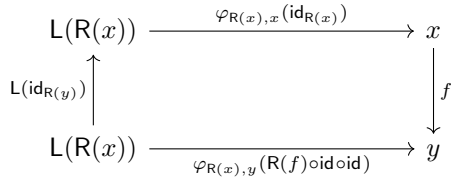
Proposition 5.8. *Let* $\mathsf{L} \dashv \mathsf{R}$ *be an adjunction where* $\mathsf{L} : \mathcal{C} \to \mathcal{D}$*. We have a natural transformation* $\varepsilon : \mathsf{L} \circ \mathsf{R} \Rightarrow \mathsf{Id}_{\mathcal{D}}$*.*

Proof. Let $x \in \mathcal{D}$. We define $\varepsilon(x) : \mathsf{L}(\mathsf{R}(x)) \to_{\mathcal{D}} x$ to be $\varphi_{\mathsf{R}(x),x}(\mathsf{id}_{\mathsf{R}(x)})$. Given $f : x \to_{\mathcal{D}} y$, we need to show the following diagram commutes.

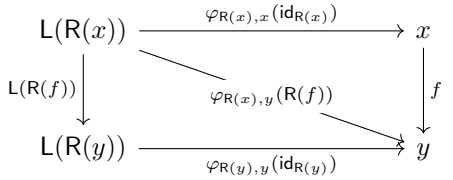


We know the following two diagrams commmute.





By combining these diagrams, we get the following commutative square.

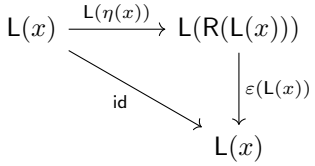


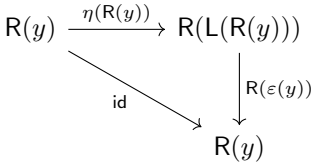
Hence, ε is natural. □

Proposition 5.9. *Let* $\mathsf{L} \dashv \mathsf{R}$ *be an adjunction where* $\mathsf{L} : \mathcal{C} \to \mathcal{D}$. *We have a natural transformation* $\eta : \mathsf{Id}_{\mathcal{C}} \Rightarrow \mathsf{R} \circ \mathsf{L}$.

Proof. Left as an exercise. □

Proposition 5.10. *Let* $\mathsf{L} \dashv \mathsf{R}$ *be an adjunction where* $\mathsf{L} : \mathcal{C} \to \mathcal{D}$. *The following triangles commutes for each* $x \in \mathcal{C}$ *and* $y \in \mathcal{D}$.





Proof. Left as an exercise. □

Digression. In this subsection, we showed that the notion of adjunction can be formulated solely using functors and natural transformation. The advantage of such an approach is that it allows us to formulate the notion of adjunction relative to a 2-category, which gives us additional generality since we can instantiate it to various categorical structures.

5.3 Adjunctions and Limits

Proposition 5.11. *Let* $\mathsf{L} \dashv \mathsf{R}$ *be an adjunction where* $\mathsf{L} : \mathcal{C} \to \mathcal{D}$*. Show that* $\mathsf{R}(f)$ *is a monomorphism for every monomorphism* $f : x \to_{\mathcal{D}} y$*.*

Proof. Let $f : x \to_{\mathcal{D}} y$ be an arbitrary monomorphism. Our goal is to show that $\mathsf{R}(f) : \mathsf{R}(x) \to_{\mathcal{C}} \mathsf{R}(y)$ is a monomorphism. Suppose that we have an object $w \in \mathcal{C}$ and moprhism $g_1, g_2 : w \to \mathsf{R}(x)$ such that $\mathsf{R}(f) \circ g_1 = \mathsf{R}(f) \circ g_2$. We need to show that $g_1 = g_2$. It suffices to show that $\varphi_{w,x}(g_1) = \varphi_{w,x}(g_2)$. Since f is a monomorphism, it suffices to show that $f \circ \varphi_{w,x}(g_1) = f \circ \varphi_{w,x}(g_2)$. Note that the following diagrams commutes

$$\begin{array}{ccc} \mathsf{L}(w) & \xrightarrow{\varphi_{w,x}(g_1)} & x \\ {\scriptstyle \mathsf{L}(\mathsf{id}_w)}\uparrow & & \downarrow{\scriptstyle f} \\ \mathsf{L}(w) & \xrightarrow[\varphi_{w,y}(\mathsf{R}(f)\circ g_1)]{} & y \end{array} \qquad \begin{array}{ccc} \mathsf{L}(w) & \xrightarrow{\varphi_{w,x}(g_2)} & x \\ {\scriptstyle \mathsf{L}(\mathsf{id}_w)}\uparrow & & \downarrow{\scriptstyle f} \\ \mathsf{L}(w) & \xrightarrow[\varphi_{w,y}(\mathsf{R}(f)\circ g_2)]{} & y \end{array}$$

Hence, it suffices to show $\mathsf{R}(f) \circ g_1 = \mathsf{R}(f) \circ g_2$, which is what we assumed. □

Proposition 5.12. *Let* $\mathsf{L} \dashv \mathsf{R}$ *be an adjunction where* $\mathsf{L} : \mathcal{C} \to \mathcal{D}$*. If* $x \in \mathcal{D}$ *is a terminal object, then* $\mathsf{R}(x)$ *is a terminal object as well.*

Proof. We need to show that for all $y \in \mathcal{C}$ there is a unique morphism $f : y \to \mathsf{R}(x)$. First, we show that f exists. Note that we have a morphism $!_{\mathsf{L}(y)} : \mathsf{L}(y) \to x$. We define

$$f = \psi_{y,x}(!_{\mathsf{L}(y)}) : y \to \mathsf{R}(x).$$

Now we show that f is the unique morphism to $\mathsf{R}(x)$. Let $f' : y \to \mathsf{R}(x)$. Note that we have

$$\psi_{y,x}(\varphi_{y,x}(f')) = f'.$$

Since $f = \psi_{y,x}(!_{\mathsf{L}(y)})$, it suffices to show that

$$\varphi_{y,x}(f') = !_{\mathsf{L}(y)}.$$

This is so, because x is terminal. Hence $\mathsf{R}(x)$ is terminal as well. □

5.4 Exercises

Exercise 5.1. Let $\mathcal{C}$ be a category with an initial object $\mathbb{0}$. Show that the functor $\mathsf{Const}_* : \mathcal{C} \to \mathsf{unit}$ has a left adjoint.

Exercise 5.2. Let $\mathcal{C}$ be a category with binary products. Let $\Delta : \mathcal{C} \to \mathcal{C} \times \mathcal{C}$ be the functor that maps x to the pair (x, x). Show that Δ has a right adjoint.

Exercise 5.3. Let $\mathcal{C}$ be a category with binary coproducts. Let $\Delta : \mathcal{C} \to \mathcal{C} \times \mathcal{C}$ be the functor that maps x to the pair (x, x). Show that Δ has a left adjoint.

Exercise 5.4. Finish the proof of Proposition 5.5.

Exercise 5.5. Prove Proposition 5.9.

Exercise 5.6. Prove Proposition 5.10.

Exercise 5.7. We write Set_* for the category whose objects are sets X with an element $x \in X$, and whose morphisms from (X, x) to (Y, y) are functions $f : X \to Y$ such that $f(x) = y$. The objects of Set_* are called **pointed sets**. We have a functor $\mathsf{U}_{\mathsf{Set}_*} : \mathsf{Set}_* \to \mathsf{Set}$ that maps every pointed set (X, x) to X.

Construct a left adjoint for $\mathsf{U}_{\mathsf{Set}_*}$.

Exercise 5.8. In the previous section, we looked at coalgebras for the functor $(-) \times A : \mathsf{Set} \to \mathsf{Set}$ (Example 3.6), which was defined to be $((-) \times A)(X) = X \times A$. Construct a right adjoint for $\mathsf{U}_{(-)\times A} : \mathsf{CoAlg}_{(-)\times A} \to \mathcal{C}$.

Hint: use streams (Example 4.21).

Exercise 5.9. Let $\mathsf{L} \dashv \mathsf{R}$ be an adjunction where $\mathsf{L} : \mathcal{C} \to \mathcal{D}$. If $x \in \mathcal{C}$ is an initial object, then $\mathsf{L}(x)$ is an initial object as well.

Exercise 5.10. Let $\mathsf{L} \dashv \mathsf{R}$ be an adjunction where $\mathsf{L} : \mathcal{C} \to \mathcal{D}$. Suppose that we have objects $x, y, z \in \mathcal{D}$ and let z be a binary product of x and y. Show that $\mathsf{R}(z)$ is a binary product of $\mathsf{R}(x)$ and $\mathsf{R}(y)$.

Exercise 5.11. Let $\mathsf{L} \dashv \mathsf{R}$ be an adjunction where $\mathsf{L} : \mathcal{C} \to \mathcal{D}$. Suppose that we have objects $x, y, z \in \mathcal{C}$ and let z be a binary product of x and y. Show that $\mathsf{L}(z)$ is a binary product of $\mathsf{L}(x)$ and $\mathsf{L}(y)$.

Exercise 5.12. Let $\mathsf{L} \dashv \mathsf{R}$ be an adjunction where $\mathsf{L} : \mathcal{C} \to \mathcal{D}$. Show that $\mathsf{L}(f)$ is an epimorphism for every epimorphism $f : x \to_{\mathcal{C}} y$.

Exercise 5.13. Suppose that we have adjunctions $\mathsf{L} \dashv \mathsf{R}_1$ and $\mathsf{L} \dashv \mathsf{R}_2$ where $\mathsf{L} : \mathcal{C} \to \mathcal{D}$. Construct a natural isomorpism between R_1 and R_2.

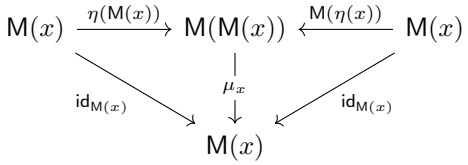
6 Monads

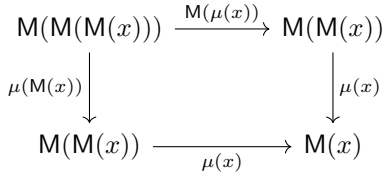
6.1 Monads and Examples

Definition 6.1. A **monad** on a category $\mathcal{C}$ consists of

- a functor $\mathsf{M} : \mathcal{C} \to \mathcal{C}$;
- a natural transformation $\eta : \mathsf{Id}_{\mathcal{C}} \Rightarrow \mathsf{M}$ (called the **unit**);
- a natural transformation $\mu : \mathsf{M} \circ \mathsf{M} \Rightarrow \mathsf{M}$ (called the **multiplication**);

such that the following diagrams commute





In functional programming, monads are used to deal with effectful computations. We can implement monads as follows.

```
class Monad m where
  pure :: a → m a
  bind :: (a → m b) → m a → m b
```

The function `pure` gives a pure computation that has no effects. We can compose effectful computations using `bind`. We can show this gives rise to a monad, and to guarantee the laws of a monad, we need to assume certain laws.

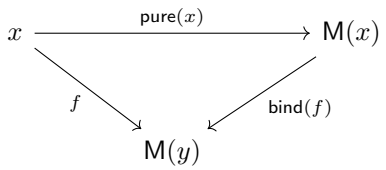
Proposition 6.2. *To give a monad on $\mathcal{C}$, it suffices to give*

- *a function $M : \mathcal{C} \to \mathcal{C}$;*
- *for all $x \in \mathcal{C}$ a morphism $\mathsf{pure}(x) : x \to \mathsf{M}(x)$;*
- *for all objects $x, y \in \mathcal{C}$ and morphisms $f : x \to \mathsf{M}(y)$, a morphism $\mathsf{bind}(f) : \mathsf{M}(x) \to \mathsf{M}(y)$*

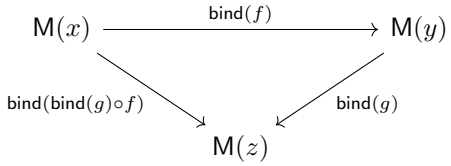
such that the following equations hold

- *for each $x \in \mathcal{C}$, we have $\mathsf{bind}(\mathsf{pure}(x)) = \mathsf{id}_{\mathsf{M}(x)}$;*

- *for each $x, y \in C$ and $f : x \to \mathsf{M}(y)$ the following diagram commutes*



- *for all $f : x \to \mathsf{M}(y)$ and $g : y \to \mathsf{M}(z)$, the following diagram commutes*

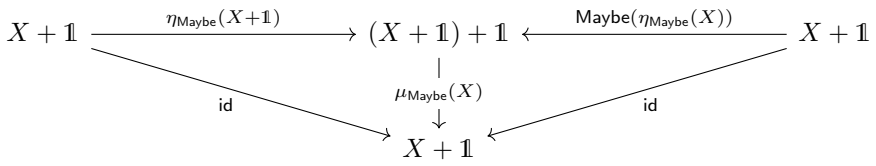


Proof. Left as an exercise. □

Example 6.3. We show in two ways that $\mathsf{Maybe} : \mathsf{Set} \to \mathsf{Set}$ (Example 3.5) is a monad. First, we instantiate directly Definition 6.1.

- As the unit, we take $\eta_{\mathsf{Maybe}} : \mathsf{id}_{\mathsf{Set}} \Rightarrow \mathsf{Maybe}$ (3.18). Recall that for each set X we defined $\eta_{\mathsf{Maybe}}(X)$ to be the function that maps $x \in X$ to $(0, x) \in \mathsf{Maybe}(X)$.
- For the multiplication, we take $\mu_{\mathsf{Maybe}} : \mathsf{Maybe} \circ \mathsf{Maybe} \Rightarrow \mathsf{Maybe}$ (Example 3.19).

We need to verify the monad laws.



First we show for all $z \in X + \mathbb{1}$ that

$$\mu_{\mathsf{Maybe}}(X)(\eta_{\mathsf{Maybe}}(X + \mathbb{1})(z)) = z$$

There are two cases. If $z = (0, x)$, then we have

$$\mu_{\mathsf{Maybe}}(X)(\eta_{\mathsf{Maybe}}(X + \mathbb{1})((0, x))) = \mu_{\mathsf{Maybe}}(X)((0, (0, x))) = (0, x)$$

The other monads laws are left to the reader.

We can also construct this monad using Proposition 6.2.

- The function $M : \mathsf{Set} \to \mathsf{Set}$ maps sets X to $X + \mathbb{1}$.

- For all sets X, we define $\mathsf{pure}(X) : X \to X + \mathbb{1}$ as the map that sends x to $(0, x)$.
- For all functions $f : X \to Y + \mathbb{1}$, we define $\mathsf{bind}(f) : X + \mathbb{1} \to Y + \mathbb{1}$ as follows

$$\mathsf{bind}(f)(z) = \begin{cases} f(x) & \text{if } z = (0, x) \\ (1, *) & \text{otherwise} \end{cases}$$

In Haskell, we implement this monad as follows.

```haskell
data Maybe a = Just a | Nothing

pure :: a → Maybe a
pure x = Just x

bind :: (a → Maybe b) → Maybe a → Maybe b
bind f (Just x) = f x
bind f Nothing = Nothing
```

We can slightly generalise the maybe monad to the exception monad.

Example 6.4. Let E be any set. The functor Except_E maps X to the disjoint union $X + E$. We can show that Except_E is a monad similarly to how we show that Maybe is a monad. We leave it to the reader to do this verification.

We can implement the exception monad in Haskell as follows.

```haskell
data Either a b = Left a | Right b

pure :: a → Either e a
pure x = Right x

bind :: (a → Either e b) → Either e a → Either e b
bind f (Left x) = Left x
bind f (Right x) = f x
```

Example 6.5. Let S be any set. We construct the reader monad on Set using Proposition 6.2.

- Given a set A, we define $\mathsf{M}(A)$ to be the set $S \to A$ of all functions from S to A. We denote this set by $\mathsf{Reader}_S(A)$.
- Given a set A, we define the function $\mathsf{pure}_{\mathsf{Reader}_S} : A \to \mathsf{Reader}_S(A)$: for $a \in A$, we have $\mathsf{pure}_{\mathsf{Reader}_S}(a)(s) = a$.
- Suppose we have $f : A \to \mathsf{Reader}_S(B)$. We define $\mathsf{bind}_{\mathsf{Reader}_S}(f) : \mathsf{Reader}_S(A) \to \mathsf{Reader}_S(B)$ as follows.

$$\mathsf{bind}_{\mathsf{Reader}_S}(f)(g)(s) = f(g(s))(s)$$

We leave the verification of the laws as an exercise to the reader.

The reader monad is implemented as follows.

```
type Reader s a = s → a

pure :: a → Reader s a
pure x = λw → x

bind :: (a → Reader s b) → Reader s a → Reader s b
bind f g = λw → f (g w) w
```

Example 6.6. Let S be any set. We construct the continuation monad on Set using Proposition 6.2.

- Given a set A, we define $\mathsf{M}(A)$ to be the set $(A \to S) \to S$ of all functions that send functions $f : A \to S$ to elements of S. We denote this set by $\mathsf{Cont}_S(A)$.
- Given a set A, we define the function $\mathsf{pure}_{\mathsf{Cont}_S} : A \to \mathsf{Cont}_S(A)$: for $a \in A$, we have $\mathsf{pure}_{\mathsf{Cont}_S}(a)(s) = s(a)$.
- Suppose we have $f : A \to \mathsf{Cont}_S(B)$. We define $\mathsf{bind}_{\mathsf{Cont}_S}(f) : \mathsf{Cont}_S(A) \to \mathsf{Cont}_S(B)$ as follows.

$$\mathsf{bind}_{\mathsf{Cont}_S}(f)(g)(s) = g(x \mapsto f(x)(s))$$

Here $x \mapsto f(x)(k)$ is the function that maps x to $f(x)(k)$.

We leave the verification of the monad laws as an exercise.

The continuation monad is implemented as follows.

```
type Cont s a = (a → s) → s

pure :: a → Cont s a
pure x = λk → k x

bind :: (a → Cont s b) → Cont s a → Cont s b
bind f g = λk → g (λx → f x k)
```

Example 6.7. Let S be any set. We construct the state monad on Set using Proposition 6.2.

- Given a set A, we define $\mathsf{M}(A)$ to be the set $S \to S \times A$ of all functions from S to the product $S \times A$. We denote this set by $\mathsf{State}_S(A)$.
- Given a set A, we define the function $\mathsf{pure}_{\mathsf{State}_S} : A \to \mathsf{State}_S(A)$: for $a \in A$, we have $\mathsf{pure}_{\mathsf{State}_S}(a)(s) = (s, a)$.
- Suppose we have $f : A \to \mathsf{State}_S(B)$. We define $\mathsf{bind}_{\mathsf{State}_S}(f) : \mathsf{State}_S(A) \to \mathsf{State}_S(B)$ as follows.

$$\mathsf{bind}_{\mathsf{State}_S}(f)(g)(s) = f(\pi_2(g(s)))(\pi_1(g(s)))$$

We leave the verification of the monad laws as an exercise.

We implement the state monad as follows.

```
type State s a = s → (s , a)

pure :: a → State s a
pure x = λw → (w , x)

bind :: (a → State s b) → State s a → State s b
bind f g = λw → case g w of (w' , x) → f x w'
```

Example 6.8. Next we construct the list monad on Set, and we do so using Proposition 6.2.

- We define $M(A)$ to be $\mathsf{List}(A)$ (set of lists of A).
- The function $\mathsf{pure}_{\mathsf{List}}(A) : A \to \mathsf{List}(A)$ sends a to the list (a).
- Let $f : A \to \mathsf{List}(B)$ be a function. The function $\mathsf{bind}_{\mathsf{List}}(f) : \mathsf{List}(A) \to \mathsf{List}(B)$ maps a list $(a_1, \ldots, a_n)$ to the concatenation of the lists $f(a_1), \ldots, f(a_n)$.

We verify the monad laws as follows. We show that $\mathsf{bind}_{\mathsf{List}}(\mathsf{pure}_{\mathsf{List}}(A))$ is the identity for every set A. Given a list $(a_1, \ldots, a_n)$, we defined the list $\mathsf{bind}_{\mathsf{List}}(\mathsf{pure}_{\mathsf{List}}(A))(a_1, \ldots, a_n)$ to be the concatenation of (a_i), which is $(a_1, \ldots, a_n)$.

Let $f : A \to \mathsf{List}(B)$ be a function. We show that $\mathsf{bind}_{\mathsf{List}}(f) \circ \mathsf{pure}_{\mathsf{List}}(A) = f$. The function $\mathsf{bind}_{\mathsf{List}}(f) \circ \mathsf{pure}_{\mathsf{List}}(A)$ maps $a \in A$ to the concatenation of the list $f(a)$, because $\mathsf{pure}_{\mathsf{List}}(A)(a) = (a)$. Hence, we indeed have $\mathsf{bind}_{\mathsf{List}}(f) \circ \mathsf{pure}_{\mathsf{List}}(A) = f$.

Let $f : A \to \mathsf{List}(B)$ and $g : B \to \mathsf{List}(C)$ be functions. Let $(a_1, \ldots, a_n) \in \mathsf{List}(A)$. We show that

$$\mathsf{bind}(g)(\mathsf{bind}(f)(a_1, \ldots, a_n)) = \mathsf{bind}(\mathsf{bind}(g) \circ f)(a_1, \ldots, a_n)$$

First, we note that $\mathsf{bind}(\mathsf{bind}(g) \circ f)(a_1, \ldots, a_n)$ is the concatenation of $(\mathsf{bind}(g) \circ f)(a_1), \ldots, (\mathsf{bind}(g) \circ f)(a_n)$. Each $f(a_i)$ is a list $(b_1^i, \ldots, b_{m_i}^i)$, so the list $\mathsf{bind}(g)(f(a_i))$ is the concatenation of the list $(g(b_1^i), \ldots, g(b_{m_i}^i))$. Next we note that $\mathsf{bind}(f)(a_1, \ldots, a_n)$ is the concatenation of $f(a_1), \ldots, f(a_n)$. Recalling that $f(a_i)$ is the list $(b_1^i, \ldots, b_{m_i}^i)$, we see that $\mathsf{bind}(g)(\mathsf{bind}(f)(a_1, \ldots, a_n))$ is

$$\mathsf{bind}(g)(b_1^1, \ldots, b_{m_1}^1, \ldots, b_1^n, \ldots, b_{m_n}^n)$$

which is the same as $\mathsf{bind}(\mathsf{bind}(g) \circ f)(a_1, \ldots, a_n)$.

An implementation of the list monad is given below.

```
pure :: a → [a]
pure x = [x]

bind :: (a → [b]) → [a] → [b]
bind f [] = []
bind f (x : xs) = f x ++ bind f xs
```

Example 6.9. Recall that a finite probability distribution on X is a map $p : X \to [0,1]$ such that $p(x) \neq 0$ for only finitely many x and such that

$$\sum_{x \in X} p(x) = 1.$$

Note that the sum above is defined, because $p(x)$ only is nonzero for finitely many $x \in X$. We can represent finite probability distrubtions as a list. For instance, if we have a set X with elements $x_1, x_2 \in X$ with $x_1 \neq x_2$, then the list

$$((x_1, \frac{1}{2}), (x_2, \frac{1}{2}))$$

represents the probability distrubtion p with $p(x_1) = p(x_2) = \frac{1}{2}$ and $p(y) = 0$ for every $y \notin \{x_1, x_2\}$.

We have a monad Dist on Set that maps every set X to the collection of finite probability distributions on X. We leave it to the reader to verify that this indeed forms a monad.

We implement the distribution monad as follows.

```
type Dist a = [(a, Float)]

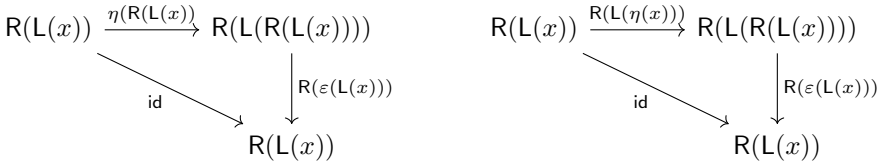
pure :: a → Dist a
pure x = [(x, 1)]

bind :: (a → Dist b) → Dist a → Dist b
bind f [] = []
bind f ((x, p) : xs) = map (λ(y, q) → (y, p * q)) (f x) ++ bind f xs
```

Example 6.10. Let $\mathsf{L} \dashv \mathsf{R}$ be an adjunction where $\mathsf{L} : \mathcal{C} \to \mathcal{D}$. We have a monad on $\mathcal{C}$, namely $\mathsf{R} \circ \mathsf{L}$. The unit of the monad is given in Proposition 5.9. The multiplication is given as the following composition (using the notation from Examples 3.14 and 3.15).

$$\mathsf{R} \circ \mathsf{L} \circ \mathsf{R} \circ \mathsf{L} \xRightarrow{\mathsf{R} \triangleleft \varepsilon \triangleright \mathsf{L}} \mathsf{R} \circ \mathsf{L}$$

For the laws, we need to prove that the various diagrams commute. We start with unitality.



These diagrams commute by Proposition 5.10. For associativity, we need to

prove the following diagram commutes.



This diagram commutes, because R is a functor and because ε is natural.

Digression. The notion of monad can be dualised as well, which gives us the notion of comonad. A comonad on $\mathcal{C}$ consists of

- a functor $\mathsf{E} : \mathcal{C} \to \mathcal{C}$;
- a natural transformation $\varepsilon : \mathsf{E} \Rightarrow \mathsf{Id}_{\mathcal{C}}$ (called the **counit**);
- a natural transformation $\delta : \mathsf{E} \to \mathsf{E} \circ \mathsf{E}$ (called the **comultiplication**).

Comonads also need to satisfy various laws that we do not discuss here. Not only do adjunctions $\mathsf{L} \dashv \mathsf{R}$ be an adjunction (where $\mathsf{L} : \mathcal{C} \to \mathcal{D}$) give rise to a monad on $\mathcal{C}$, they also give rise to a comonad on $\mathcal{D}$. Comonads have various applications, and they are particularly relevant to linear logic, where they are used to interpret exponentation.

6.2 The Kleisli Category

Definition 6.11. Let M be a monad on $\mathcal{C}$. We define the **Kleisli category**, denoted by Kl_{M}, as follows

- objects of Kl_{M} are objects of $\mathcal{C}$;
- given $x, y \in \mathsf{Kl}_{\mathsf{M}}$, the collection $x \to_{\mathsf{Kl}_{\mathsf{M}}} y$ is defined to be $x \to_{\mathcal{C}} \mathsf{M}(y)$.

Given $x \in \mathsf{Kl}_{\mathsf{M}}$, the identity $x \to_{\mathsf{Kl}_{\mathsf{M}}} x$ is defined to be $\eta(x) : x \to_{\mathcal{C}} \mathsf{M}(x)$. Given $f : x \to_{\mathsf{Kl}_{\mathsf{M}}} y$ and $g : y \to_{\mathsf{Kl}_{\mathsf{M}}} z$, their composition is defined to be the following composition.

$$x \xrightarrow{f} \mathsf{M}(y) \xrightarrow{\mathsf{M}(g)} \mathsf{M}(\mathsf{M}(z)) \xrightarrow{\mu(z)} \mathsf{M}(z)$$

Lemma 6.12. *The Kleisli category is indeed a category.*

Proof. We start with unitality. We need to show that the following morphism is equal to f.

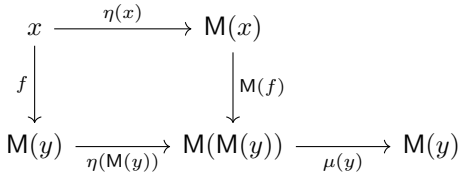
$$x \xrightarrow{f} \mathsf{M}(y) \xrightarrow{\mathsf{M}(\eta(y))} \mathsf{M}(\mathsf{M}(y)) \xrightarrow{\mu(y)} \mathsf{M}(y)$$

By the monad laws, we have $\mu(y) \circ \mathsf{M}(\eta(y)) = \mathsf{id}$, and hence this composition is equal to f.

We also need to show that the following composition is f

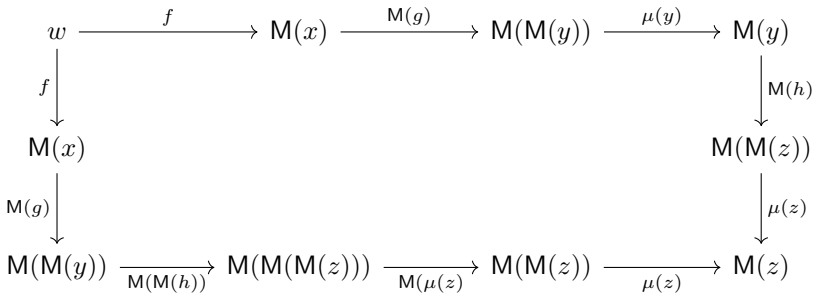
$$x \xrightarrow{\eta(x)} \mathsf{M}(x) \xrightarrow{\mathsf{M}(f)} \mathsf{M}(\mathsf{M}(y)) \xrightarrow{\mu(y)} \mathsf{M}(y)$$

To do so, we use the following diagram.

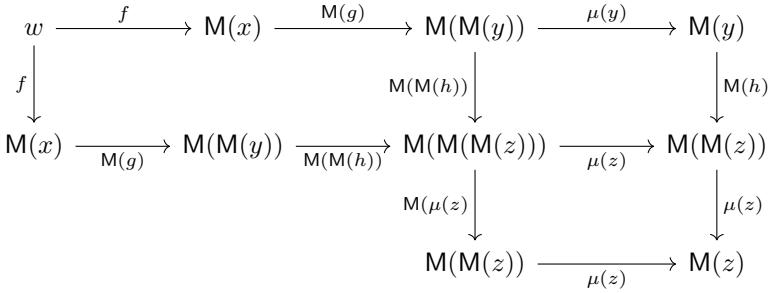


The left square commutes, because η is natural. The composition $\mu(y) \circ \eta(\mathsf{M}(y))$ is the identity by the monad laws. Hence, the desired monad law holds.

For associativity, we need to show the following diagram commutes



We draw this diagram in a slightly different way.



The left rectangle commutes, because the sides denote exactly the same morphism. The top right square commutes due to naturality of μ, and the bottom right square commutes due to the monad laws. Hence, composition is associate as well. □

Proposition 6.13. *We have a functor* L *from* $\mathcal{C}$ *to* Kl_{M}.

Proof. This functor maps $x : \mathcal{C}$ to $x : \mathsf{Kl}_{\mathsf{M}}$. We leave it as an exercise to verify this indeed forms a functor. □

Proposition 6.14. *We have a functor* R *from* Kl_{M} *to* $\mathcal{C}$*.*

Proof. This functor maps $x : \mathsf{Kl}_{\mathsf{M}}$ to $\mathsf{M}(x) : \mathcal{C}$. We leave it as an exercise to verify this indeed forms a functor. □

Proposition 6.15. *We have* $\mathsf{L} \dashv \mathsf{R}$*.*

Proof. Let $f : \mathsf{L}(x) \to_{\mathsf{Kl}_{\mathsf{M}}} y$ be a morphism. Note that f is the same as a morphism $f : x \to_{\mathcal{C}} \mathsf{M}(y)$. We define the morphism $\varphi_{x,y}(f) : x \to_{\mathcal{C}} \mathsf{R}(y)$ to be f where we note that $\mathsf{R}(y) = \mathsf{M}(y)$. Note that $\psi_{x,y}$ can be defined in the same way and that the resulting maps are inverses. We leave it to the reader to verify the necessary equations. □

Morphisms in the Kleisli category represent effectful computations.

- Morphisms in $\mathsf{Kl}_{\mathsf{Maybe}}$ are the same as morphisms $f : X \to Y + \mathbb{1}$. These correspond to partial morphisms from X to Y, and they represent computations that are not always defined.
- Morphisms in $\mathsf{Kl}_{\mathsf{Except}_E}$ are the same as morphisms $f : X \to Y + E$. These morphisms represent computations that might return an exception.
- Morphisms in $\mathsf{Kl}_{\mathsf{Reader}_S}$ are the same as morphisms $f : X \to (S \to Y)$, which are the same as morphisms $g : X \times S \to Y$. These morphisms represent computations that read an additional input from S.
- Morphisms in $\mathsf{Kl}_{\mathsf{Cont}_S}$ are the same as morphisms $f : X \to (Y \to S) \to S$, which are the same as morphisms $g : X \times (Y \to S) \to S$. Such morphisms represent computations in a continuation-passing style. Rather than only getting an $x \in X$, which is the input of our computation, we also get a function $f : Y \to S$ that describes what should be done with the output.
- Morphisms in $\mathsf{Kl}_{\mathsf{State}_S}$ are the same as morphisms $f : X \to S \to S \times Y$, which are the same as morphisms $g : S \times X \to S \times Y$. These morphisms represent stateful computations: our computations have an input $x \in X$ and a state $s \in S$, and they return an output $y \in Y$ and a new state $s' \in S$.
- Morphisms in $\mathsf{Kl}_{\mathsf{List}}$ are the same as morphisms $f : X \to \mathsf{List}(Y)$. These morphisms represent nondeterministic computations: input do not necessarily give a single output, but a whole list of outputs.
- Morphisms in $\mathsf{Kl}_{\mathsf{Dist}}$ are the same as morphisms $f : X \to \mathsf{Dist}(Y)$. These morphisms represent probablistic computations: every input gives a list of outputs and each output comes with a certain probability.

Digression. One theme throughout the digressions in these notes, is that 2-category theory is a nice framework to study categorical constructions. Contuining on this theme, it has to be noted that the Kleisli category can nicely be characterised using a 2-categorical framework. Specifically, a large part of the theory of monads can be developed formally in a 2-category.

6.3 Exercises

Exercise 6.1. Prove Proposition 6.2.

Exercise 6.2. Show that Except_E forms a monad (Example 6.4).

Exercise 6.3. Verify the monad laws for Example 6.5.

Exercise 6.4. Verify the monad laws for Example 6.6.

Exercise 6.5. Verify the monad laws for Example 6.7.

Exercise 6.6. Show that Dist forms a monad (Example 6.9).

Exercise 6.7. Show that L, defined in Proposition 6.13, is a functor.

Exercise 6.8. Show that R, defined in Proposition 6.14, is a functor.

Exercise 6.9. Finish the proof of Proposition 6.15 by verifying the necessary equations.

Exercise 6.10. Construct a monad $\mathcal{P}_{\mathsf{fin}}$ on Set that maps every set X to the set of finite subsets of X.

Exercise 6.11. Let M be a monoid. Construct a monad Writer_M on Set that maps every set X to $M \times X$.

7 The Yoneda Lemma

7.1 Presheaves

Definition 7.1. Let $\mathcal{C}$ be a category. A **presheaf** on $\mathcal{C}$ is a functor $F : \mathcal{C}^{\mathsf{op}} \to \mathsf{Set}$.

Digression. We often only look at presheaves over small categories, i.e., categories that have a set of objects and a set of morphisms. This restriction is necessary for various purposes: its usefulness comes from the fact that it allows us to do certain constructions of sets where we use all objects in $\mathcal{C}$.

Example 7.2. Let $y \in \mathcal{C}$ be an object. We define the **representable presheaf** repr_y on $\mathcal{C}$ as follows:

- for $x \in \mathcal{C}$, we define $\mathsf{repr}_y(x)$ to be the set $\mathcal{C}_1(x, y)$ of morphisms
- given $f : \mathcal{C}_1^{\mathsf{op}}(x_2, x_1)$, we define the function $\mathsf{repr}_y(f)$ that maps $g : \mathcal{C}_1(x_2, y)$ to $g \circ f : \mathcal{C}_1(x_1, y)$.

We need to check the functor laws for repr_y. First, we check that $\mathsf{repr}_y(\mathsf{id}_x)$ is the identity function. This function maps $g : \mathcal{C}_1(x, y)$ to $g \circ \mathsf{id}_x$, which is equal to g by the category laws. Hence, the identity is preserved.

Next, we assume that we have $f_1 : \mathcal{C}_1^{\mathsf{op}}(x_2, x_1)$ and $f_2 : \mathcal{C}_1^{\mathsf{op}}(x_3, x_2)$ we need to check that $\mathsf{repr}_y(f_1 \circ_{\mathcal{C}^{\mathsf{op}}} f_2)$ is the composition $\mathsf{repr}_y(f_1) \circ \mathsf{repr}_y(f_2)$. Let $g : \mathcal{C}_1(x_3, y)$. Then

$$\mathsf{repr}_y(f_1 \circ_{\mathcal{C}^{\mathsf{op}}} f_2)(g) = g \circ (f_1 \circ_{\mathcal{C}^{\mathsf{op}}} f_2) = g \circ (f_2 \circ f_1) = (g \circ f_2) \circ f_1$$

$$\mathsf{repr}_y(f_1)(\mathsf{repr}_y(f_2)(g)) = \mathsf{repr}_y(f_1)(g \circ f_2) = (g \circ f_2) \circ f_1$$

These two are equal, and hence repr_y preserves composition as well.

Proposition 7.3 (Yoneda Lemma)**.** *Let $F : \mathcal{C}^{\mathsf{op}} \to \mathsf{Set}$ be a presheaf on $\mathcal{C}$, and let $y \in \mathcal{C}$. We have a bijection between*

- *elements of $F(y)$*
- *natural transformations from repr_y to F*

Proof. Let $z \in F(y)$. We define a natural transformation $\tau^z : \mathsf{repr}_y \Rightarrow F$ as follows.

- Let $x \in \mathcal{C}$. We need to define a function τ^z_x from $\mathsf{repr}_y(x)$ to $F(x)$. The map τ^z_x sends morphisms $g : \mathcal{C}_1(x, y)$ to $F(g)(z)$. Note that $F(g) : F(y) \to F(x)$ is a function.
- Let $f : \mathcal{C}_1^{\mathsf{op}}(x_2, x_1)$. We need to check that the following diagram commutes

$$\begin{array}{ccc} \mathsf{repr}_y(x_1) & \xrightarrow{\tau^z_{x_1}} & F(x_1) \\ {\scriptstyle \mathsf{repr}_y(f)}\uparrow & & \uparrow{\scriptstyle F(f)} \\ \mathsf{repr}_y(x_2) & \xrightarrow[\tau^z_{x_2}]{} & F(x_2) \end{array}$$

Let $g : \mathcal{C}_1(x_2, y)$. Then

$$\tau^z_{x_1}(\mathsf{repr}_y(f)(g)) = \tau^z_{x_1}(g \circ f) = F(g \circ f)(z)$$

$$F(f)(\tau^z_{x_2}(g)) = F(f)(F(g)(z))$$

Note that we have $F(g \circ f)(z) = F(f)(F(g)(z))$ since F is functorial.

Now let $\theta : \mathsf{repr}_y \Rightarrow F$ be any natural transformation. We define $z_\theta \in F(y)$ to be $\theta(y)(\mathsf{id}_y)$. Note that $\mathsf{id}_y \in \mathsf{repr}_y(y)$, so $\theta(y)(\mathsf{id}_y) \in F(y)$.

We need to verify that these maps are inverses. Let $z \in F(y)$. Then we have

$$z_{\tau^z} = \tau_z(y)(\mathsf{id}_y) = F(\mathsf{id}_y)(z) = z$$

The final step holds, because F is functorial.

Now let $\tau : \mathsf{repr}_y \Rightarrow F$ be any natural transformation. We want to check that $\tau = \tau^{z_\tau}$. Let $f : \mathcal{C}_1(x, y)$ be a morphism. We calculate $\tau^{z_\tau}(x)(f)$:

$$\tau^{z_\tau}(x)(f) = F(f)(z_\tau) = F(f)(\tau(y)(\mathsf{id}_y))$$

Since τ is natural, we have $\tau(x) \circ \mathsf{repr}_y(f) = F(f) \circ \tau(y)$, so, in particular, we have

$$\begin{aligned}(\tau(x) \circ \mathsf{repr}_y(f))(\mathsf{id}_y) &= \tau(x)(\mathsf{repr}_y(f)(\mathsf{id}_y)) \\ &= \tau(x)(\mathsf{id}_y \circ f)\end{aligned}$$

We alo have

$$(F(f) \circ \tau(y))(\mathsf{id}_y) = F(f)(\tau(y)(\mathsf{id}_y))$$

Hence, we have

$$\tau^{z_\tau}(x)(f) = F(f)(\tau(y)(\mathsf{id}_y)) = \tau(x)(\mathsf{id}_y \circ f) = \tau(x)(f)$$

This proves the desired final equality, and thus we get the desired bijection. □

7.2 Exercises

Exercise 7.1. Let $f : y_1 \to y_2$ be a morphism in a category $\mathcal{C}$. Construct a natural transformation $\mathsf{repr}_f : \mathsf{repr}_{y_1} \Rightarrow \mathsf{repr}_{y_2}$.

Exercise 7.2. Let x be a terminal object in $\mathcal{C}$. Construct a natural isomorphism between repr_x and the constant presheaf $\mathsf{Const}_{\mathbb{1}} : \mathcal{C}^{\mathsf{op}} \to \mathsf{Set}$ that maps everything to the set $\mathbb{1}$.

Exercise 7.3. Let z be a binary product of x and y. Construct a natural isomorphism between repr_z and the presheaf that maps w to $\mathsf{repr}_x(w) \times \mathsf{repr}_y(w)$.

8 Heyting Algebras

In the final two sections, we study applications of category theory to logic. Specifically, we look at how to use category theory to describe the notion of model for first-order predicate logic. For completenes, we recall the full syntax of first-order predicate logic in Section A. Note that our logic is constructive, and we do not assume the law of excluded middle ($\varphi \vee \neg\varphi$).

Our notion of categorical model for first-order predicate logic, is that of a **first-order hyperdoctrine**. We introduce this notion in multiple steps. Specifically, we start by describing the semantics of fragments of our system, and step for step we come closer to the full system. The first fragment that we consider, is **propositional logic**. In propositional logic, formulas do not depend on variables, and one can describe the semantics using **Heyting algebras**. Next we extend our system with variables of substitution, and this extension uses presheaves. The result is the notion of a hyperdoctrine, which gives us a notion of model of a large fragment of first-order predicate logic. It incoorporates predicates and the connectives from propositional logic. However, what is missing, are the quantifiers and equality, and we consider these in the next section.

To understand what it means to give semantics to a logical system, we look in more detail at propositional logic. Propositional logic has two key components: formulas and derivation. To give semantics to propositional logic, we must

- give a set Φ of **semantic formulas**;
- give a map from formulas in propositional logic to Φ, which we call the **interpretation**, and we denote the interpretation of a formula φ by $[\![\varphi]\!]$;
- give a binary relation $\leq$ on Φ, which represents **semantic provability**

such that $[\![\varphi]\!] \leq [\![\psi]\!]$ whenever we can prove ψ from φ. In addition, each connective in propositional logic gives rise to a corresponding operation on Φ: for instance, conjunction gives us a binary operation $\wedge : \Phi \times \Phi \to \Phi$. Essentially, our goal is to find a mathematical structure that incorporates both formulas and provability. There is a large variety of such structures, and our focus is on Heyting algebras.

8.1 Heyting Algebras

Definition 8.1. A **(distributive) lattice** consists of a set Ω together with operations

$$\top \in \Omega, \quad \bot \in \Omega, \quad \wedge : \Omega \times \Omega \to \Omega, \quad \vee : \Omega \times \Omega \to \Omega$$

We require the following laws:

- $\top \wedge \varphi = \varphi = \varphi \wedge \top$ and $\bot \vee \varphi = \varphi = \varphi \vee \bot$
- $\varphi \wedge \varphi = \varphi$ and $\varphi \vee \varphi = \varphi$

- $\varphi \wedge \psi = \psi \wedge \varphi$ and $\varphi \vee \psi = \psi \vee \varphi$
- $\varphi \wedge (\psi \wedge \chi) = (\varphi \wedge \psi) \wedge \chi$ and $\varphi \vee (\psi \vee \chi) = (\varphi \vee \psi) \vee \chi$
- $\varphi \vee (\varphi \wedge \psi) = \varphi$ and $\varphi \wedge (\varphi \vee \psi) = \varphi$
- $\varphi \vee (\psi \wedge \chi) = (\varphi \vee \psi) \wedge (\varphi \vee \chi)$
- $\varphi \wedge (\psi \vee \chi) = (\varphi \wedge \psi) \vee (\varphi \wedge \chi)$

Note that the final two equations $\varphi \vee (\psi \wedge \chi) = (\varphi \vee \psi) \wedge (\varphi \vee \chi)$ and $\varphi \wedge (\psi \vee \chi) = (\varphi \wedge \psi) \vee (\varphi \wedge \chi)$ are derivable, but we do not look into this.

In every lattice Ω, we can define a partial order by saying that $\varphi \leq \psi$ if and only if $\varphi \wedge \psi = \varphi$.

Proposition 8.2. *The relation $\leq$ is indeed a partial order.*

Proof. We need to verify that $\leq$ is reflexive, transitive, and antisymmetric. For reflexivity, we need to show that $\varphi \leq \varphi$. This holds, because one of the laws of lattices dictate that

$$\varphi \wedge \varphi = \varphi.$$

For anti-symmetry, we need to show that $\varphi = \psi$ whenever $\varphi \leq \psi$ and $\psi \leq \varphi$. If we assume that $\varphi \wedge psi = \varphi$ and $\psi \wedge \varphi = \psi$, then we must also have

$$\varphi = \varphi \wedge \psi = \psi \wedge \varphi = \psi.$$

We leave transivity as an exercise. □

Definition 8.3. A **Heyting algebra** is a lattice Ω together with an operation $\Rightarrow\colon \Omega \times \Omega \to \Omega$ such that $\chi \wedge \varphi \leq \psi$ if and only if $\chi \leq \varphi \Rightarrow \psi$.

Example 8.4. The set $\{\bot, \top\}$ is a Heyting algebra. The operations are defined as below. One can verify the laws using case distinctions.

φ	ψ	$\varphi \wedge \psi$	$\varphi \vee \psi$	$\varphi \Rightarrow \psi$
$\bot$	$\bot$	$\bot$	$\bot$	$\top$
$\bot$	$\top$	$\bot$	$\top$	$\top$
$\top$	$\bot$	$\bot$	$\top$	$\bot$
$\top$	$\top$	$\top$	$\top$	$\top$

Example 8.5. Let X be a set. Then the power set $\mathcal{P}(X) = \{Y | Y \subseteq X\}$ is a complete Heyting algebra. We first show that $\mathcal{P}(X)$ is a lattice. The lattice operations are defined as follows

- $\top = X$;
- $\bot = \emptyset$;
- $Y_1 \wedge Y_2 = Y_1 \cap Y_2$;

- $Y_1 \vee Y_2 = Y_1 \cup Y_2$.

We leave it as an exercise for the reader to verify the lattice equations. In this lattice, we have $Y_1 \leq Y_2$ if and only if $Y_1 \subseteq Y_2$.

Next we show that $\mathcal{P}(X)$ is a Heyting algebra. We define

$$Y_1 \Rightarrow Y_2 = \{x \in X | \text{if } x \in Y_1, \text{ then } x \in Y_2\}$$

We need to show that $Z \subseteq Y_1 \Rightarrow Y_2$ if and only if $Z \cap Y_1 \subseteq Y_2$. Suppose that $Z \subseteq Y_1 \Rightarrow Y_2$, and let $x \in Z \cap Y_1$. We need to show that $x \in Y_2$. Since $x \in Z \cap Y_1$, we have that $x \in Z$. We must then also have $x \in Y_1 \Rightarrow Y_2$, meaning that $x \in Y_2$ if $x \in Y_1$. Since $x \in Y_1$, we get that $x \in Y_2$.

Suppose that $Z \cap Y_1 \subseteq Y_2$. We need to show that $Z \subseteq Y_1 \Rightarrow Y_2$. Let $x \in Z$. We need to show that $x \in Y_2$ if $x \in Y_1$. Since we know that $Z \cap Y_1 \subseteq Y_2$, this follows directly.

Digression. Every Heyting algebra Ω gives rise to a category, since every Heyting algebra is a partial order. This category has a terminal object, given by $\top$, and binary products, given by $\wedge$. We also have an initial object, $\bot$, and binary coproducts, namely $\vee$. The operation $\Rightarrow$ can also be characterised categorically, namely as a right adjoint.

In a Heyting algebra, we can interpret propositional logic as described in Section A. The interpretation of formulas is defined as follows:

- $[\![\top]\!] = \top$
- $[\![\bot]\!] = \bot$
- $[\![\varphi \wedge \psi]\!] = [\![\varphi]\!] \wedge [\![\psi]\!]$
- $[\![\varphi \vee \psi]\!] = [\![\varphi]\!] \vee [\![\psi]\!]$
- $[\![\varphi \Rightarrow \psi]\!] = [\![\varphi]\!] \Rightarrow [\![\psi]\!]$

Provability is interpreted using $\leq$ meaning that we need to show that $[\![\varphi]\!] \leq [\![\psi]\!]$ whenever $\varphi \vdash \psi$. We verify this by considering all the derivation rules in propositional logic, and by showing that the corresponding inequality holds in arbitrary Heyting algebras. We do this in the following proposition.

Proposition 8.6. *Let Ω be a Heyting algebra. We have the following inequalities*

1. *$\varphi \leq \varphi$;*
2. *if $\varphi_1 \leq \varphi_2$ and $\varphi_2 \leq \varphi_3$, then $\varphi_1 \leq \varphi_3$;*
3. *if $\varphi_1 \leq \psi$, then $\varphi_1 \wedge \varphi_2 \leq \psi$;*
4. *if $\varphi_2 \leq \psi$, then $\varphi_1 \wedge \varphi_2 \leq \psi$;*
5. *$\varphi \leq \top$;*

6. $\bot \leq \varphi$;
7. *if* $\varphi \leq \psi_1$ *and* $\varphi \leq \psi_2$, *then* $\varphi \leq \psi_1 \wedge \psi_2$;
8. *if* $\varphi \leq \psi_1 \wedge \psi_2$, *then* $\varphi \leq \psi_1$;
9. *if* $\varphi \leq \psi_1 \wedge \psi_2$, *then* $\varphi \leq \psi_2$;
10. *if* $\varphi \leq \psi_1$, *then* $\varphi \leq \psi_1 \vee \psi_2$;
11. *if* $\varphi \leq \psi_2$, *then* $\varphi \leq \psi_1 \vee \psi_2$;
12. *if* $\varphi \leq \psi_1 \vee \psi_2$ *and* $\psi_1 \leq \chi$ *and* $\psi_2 \leq \chi$, *then* $\varphi \leq \chi$;
13. *if* $\varphi \leq \psi_1 \vee \psi_2$ *and* $\varphi \wedge \psi_1 \leq \chi$ *and* $\varphi \wedge \psi_2 \leq \chi$, *then* $\varphi \leq \chi$;
14. *if* $\varphi \wedge \psi \leq \chi$, *then* $\varphi \leq \psi \Rightarrow \chi$;
15. *if* $\varphi \wedge \psi$ *and* $\varphi \leq \psi \Rightarrow \chi$, *then* $\varphi \leq \chi$.

Proof. We only discuss a selection of these inequalities. First we look at (10), so we suppose that $\varphi \leq \psi_1$, which means that $\varphi \wedge \psi_1 = \varphi$. We need to show that $\varphi \leq \psi_1 \vee \psi_2$, meaning that we need to prove that $\varphi \wedge (\psi_1 \vee \psi_2) = \varphi$. This holds, because

$$\begin{aligned}\varphi \wedge (\psi_1 \vee \psi_2) &= (\varphi \wedge \psi_1) \wedge (\psi_1 \vee \psi_2) \\ &= \varphi \wedge (\psi_1 \wedge (\psi_1 \vee \psi_2)) \\ &= \varphi \wedge \psi_1 \\ &= \varphi\end{aligned}$$

Next we look at (12) and (13). Suppose that

- $\varphi \leq \psi_1 \vee \psi_2$, so $\varphi \wedge (\psi_1 \vee \psi_2) = \varphi$;
- $\psi_1 \leq \chi$, so $\psi_1 \wedge \chi = \psi_1$;
- $\psi_2 \leq \chi$, so $\psi_2 \wedge \chi = \psi_2$.

We need to show that $\varphi \leq \chi$, so we need to show that $\varphi \wedge \chi = \varphi$. This following by the following chain of equalities

$$\begin{aligned}\varphi \wedge \chi &= (\varphi \wedge (\psi_1 \vee \psi_2)) \wedge \chi \\ &= \varphi \wedge ((\psi_1 \vee \psi_2) \wedge \chi) \\ &= \varphi \wedge (\chi \wedge (\psi_1 \vee \psi_2)) \\ &= \varphi \wedge ((\chi \vee \psi_1) \wedge (\chi \vee \psi_2)) \\ &= \varphi \wedge ((\psi_1 \vee \chi) \wedge (\psi_2 \vee \chi)) \\ &= \varphi \wedge (\psi_1 \wedge \psi_2) \\ &= \varphi\end{aligned}$$

For the proof of (13), we use (12). We assume that we have

- $\varphi \leq \psi_1 \vee \psi_2$, so $\varphi \wedge (\psi_1 \vee \psi_2) = \varphi$;
- $\varphi \wedge \psi_1 \leq \chi$, so $(\varphi \wedge \psi_1) \wedge \chi = \varphi \wedge \psi_1$;
- $\varphi \wedge \psi_2 \leq \chi$, so $(\varphi \wedge \psi_2) \wedge \chi = \varphi \wedge \psi_2$.

Our goal is to show that $\varphi \wedge \chi = \varphi$. We are going to apply (12) using $\varphi \Rightarrow \chi$. For that, we note that we have

- $\varphi \leq \psi_1 \vee \psi_2$, so $\varphi \wedge (\psi_1 \vee \psi_2) = \varphi$;
- $\psi_1 \leq \varphi \Rightarrow \chi$, because $\varphi \wedge \psi_1 \leq \chi$;
- $\psi_2 \leq \varphi \Rightarrow \chi$, because $\varphi \wedge \psi_2 \leq \chi$.

Hence, we get that $\varphi \leq \varphi \Rightarrow \chi$. This is equivalent to $\varphi\varphi \wedge \varphi \leq \chi$, which is exactly what we wanted to prove.

We leave the other equations as an exercise to the reader. □

Definition 8.7. Let Ω_1 and Ω_2 be Heyting algebras. A **morphism** $f : \Omega_1 \to \Omega_2$ is a function $f : \Omega_1 \to \Omega_2$ such that

- $f(\top) = \top$;
- $f(\bot) = \bot$;
- $f(\varphi \wedge \psi) = f(\varphi) \wedge f(\psi)$;
- $f(\varphi \vee \psi) = f(\varphi) \vee f(\psi)$;
- $f(\varphi \Rightarrow \psi) = f(\varphi) \Rightarrow f(\psi)$.

Definition 8.8. We define the category Heyt as the category whose objects are Heyting algebras and whose morphisms are morphisms of Heyting algebras.

8.2 Exercises

Exercise 8.1. Verify that the order $\leq$ on a lattice Ω is transitive.

Exercise 8.2. Let $(X, \mathcal{O})$ be a topological space. Show that $\mathcal{O}$ is a Heyting algebra.

Exercise 8.3. Let $(X, \leq)$ be a partial order. A subset Y of X is said to be downward closed we have $y_1 \in Y$ whenever $y_1 \leq y_2$ and $y_2 \in Y$. Show that the set of downward closed sets is a Heyting algebra.

Exercise 8.4. Give an example of a partial order that is not a Heyting algebra.

Exercise 8.5. Prove the remaining equations of Proposition 8.7.

9 Hyperdoctrines

9.1 Hyperdoctrines

Definition 9.1. hyperdoctrine

soundness of fragment of predicate logic

Example 9.2. hyperdoctrine arising from a Heyting algebra

9.2 Quantifiers as Adjoints

A First-Order Predicate Logic

The version of first-order predicate logic that we consider, has the following components:

- Types: we have a set Ty of types;
- Terms: given types $\Gamma, A : \mathsf{Ty}$, we have a set $\mathsf{Tm}_\Gamma(A)$ of terms of type A using variables from Γ;
- Formulas: given a type $\Gamma : \mathsf{Ty}$, we have a set $\mathsf{Formula}(\Gamma)$ of formulas that use variables from Γ;
- Proofs: given a type $\Gamma : \mathsf{Ty}$ and two formulas $\Delta, \varphi : \mathsf{Formula}(\Gamma)$, we have a predicate $\Delta \vdash_\Gamma \varphi$ that says whether φ can be derived from Δ.

The key feature of this logic is that formulas can use variables. We can see types as the list of variables that our formulas are allowed to use.

We have the following types in our system.

- The unit type: $\mathbb{1}$. The type $\mathbb{1}$ represents the empty list of variables, so a formula using variables from $\mathbb{1}$ does not use any variables.
- The product type $\Gamma \times A$. The product type allows us to extend some list of variables with an additional variable.

There are more types of interest that we would want to consider (like function types), but for simplicity we disregard any other type former.

We have the following term formers in our system.

- We have a term $\mathrm{var} \in \mathsf{Tm}_\Gamma(\Gamma)$ for each type Γ
- Given terms $t_1 \in \mathsf{Tm}_A(B)$ and $t_2 \in \mathsf{Tm}_B(C)$, we have a term $t_2[t_1] \in \mathsf{Tm}_A(C)$.
- Given $t \in \mathsf{Tm}_\Gamma(A \times B)$, we have $\pi_1(t) : \mathsf{Tm}_\Gamma(A)$.
- Given $t \in \mathsf{Tm}_\Gamma(A \times B)$, we have $\pi_2(t) : \mathsf{Tm}_\Gamma(B)$.
- Given $t_1 \in \mathsf{Tm}_\Gamma(A)$ and $t_2 \in \mathsf{Tm}_\Gamma(B)$, we have $\langle t_1, t_2 \rangle : \mathsf{Tm}_\Gamma(A \times B)$
- We have a term $* \in \mathsf{Tm}_\Gamma(\mathbb{1})$ for each type Γ.

We think of var as being the variable, as it allows us to use our variables. The term $t_2[t_1]$ represents substitution: we replace the variables in t_2 by t_1.

There also are various identifications on terms:

$$t[\mathrm{var}] = t, \quad \mathrm{var}[t] = t$$

$$t_3[t_2][t_1] = t_3[t_2[t_1]]$$

$$\pi_1\langle t_1, t_2 \rangle = t_1, \quad \pi_2\langle t_1, t_2 \rangle = t_2$$

$$\langle t_1, t_2 \rangle[s] = \langle t_1[s], t_2[s] \rangle$$

If $t \in \mathsf{Tm}_\Gamma(A \times B)$, we have $t = \langle \pi_1(t), \pi_2(t) \rangle$, and if $t \in \mathsf{Tm}_\Gamma(\mathbb{1})$, we have $t = *$.

Our system comes with a diverse amount of connectives. We have **propositional connectives** coming from propositional logic, and they are

- Truth: $\top$
- Falsity: $\bot$
- Conjunction: $\wedge$
- Disjunction: $\vee$
- Implication: $\Rightarrow$
- Universal quantification: $\forall$
- Existential quantification: $\exists$
- Equality: $\equiv$

The truth formula $\top$ represents a formula that is vacuously true, meaning that we can always prove it. The falsity formula $\bot$ represents an absurd formula, meaning that everything follows from it. The conjunction $\varphi \wedge \psi$ means that both φ and ψ are true, and the disjunction $\varphi \vee \psi$ says that at least one of φ and ψ is true. The implication $\varphi \Rightarrow \psi$ says that ψ follows from φ. We have two quantifiers: the universal quanifier $\forall$, which says some formula holds for every element, and the existential quantifier $\exists$, which says that some formula holds for at least one element. For the quantifiers, we use a slightly nonstandard notation.

While one would usually write something like $\forall(x \in X), \varphi(x)$, we write $\forall\varphi$. The reason why we do so, is because it allows us to disregard various syntactic details. For instance, the formulas $\forall(x \in X), \varphi(x)$ and $\forall(y \in X), \varphi(y)$ are usually identified, which is known as α-equivalence. Intuitively, a formula does not change if we change the names of bound variables. If our variables are named, then we also have to worry about name capture and we need to have a sufficient amount of fresh free variables. By disregarding named variables and by writing $\forall\varphi$, it thus becomes significantly simpler to formally describe the semantics on the system. The main cost is that our formulas are slightly harder to read.

Below we describe the rules for our logic. There are two kinds of rules in our system. On the one hand, there are **structural rules**. These rules are not specific to any connective, and they cover general rules for substitution and dealing with hypotheses. On the other hand, there are **logical rules**. The logical rules describe the behaviour of certain connectives, and they come in various flavours. Specifically, we have

- **introduction rules**, which describe how we can prove some connective;

- **elimination rules**, which describe what we can do with a proof of some connective;
- **substitution rules**, which describe what happens when we perform a substitution on some formula using that connective.

We also explicate **formation rules**. The purpose of formation rules is to describe how we can form some connective, and they explicate the variable dependency of connectives.

A.1 Stuctural Rules

Substitution:

$$\frac{}{\varphi[\mathsf{id}] = \varphi}$$

$$\frac{}{\varphi[s_2][s_1] = \varphi[s_1 \cdot s_2]}$$

$$\frac{\Delta \vdash_{\Gamma_1} \varphi}{\Delta[s] \vdash_{\Gamma_2} \varphi[s]}$$

Hypotheses:

$$\frac{}{\varphi \vdash_\Gamma \varphi}$$

$$\frac{\Delta_1 \vdash_\Gamma \varphi}{\Delta_1 \wedge \Delta_2 \vdash_\Gamma \varphi}$$

$$\frac{\Delta_2 \vdash_\Gamma \varphi}{\Delta_1 \wedge \Delta_2 \vdash_\Gamma \varphi}$$

Cut rule:

$$\frac{\Delta \vdash_\Gamma \varphi \qquad \varphi \vdash_\Gamma \psi}{\Delta \vdash_\Gamma \psi}$$

A.2 Truth Formula

Formation:

$$\frac{\Gamma : \mathsf{Ty}}{\top : \mathsf{Formula}(\Gamma)}$$

Introduction:

$$\frac{\Delta : \mathsf{Formula}(\Gamma)}{\Delta \vdash_\Gamma \top}$$

Substitution:

$$\frac{}{\top[s] = \top}$$

A.3 Falsity formula

Formation:

$$\frac{\Gamma : \mathsf{Ty}}{\bot : \mathsf{Formula}(\Gamma)}$$

Elimination:

$$\frac{\Delta : \mathsf{Formula}(\Gamma) \qquad \varphi : \mathsf{Formula}(\Gamma) \qquad \Delta \vdash_\Gamma \bot}{\Delta \vdash_\Gamma \varphi}$$

Substitution:

$$\frac{}{\bot[s] = \bot}$$

A.4 Conjunction

Formation:

$$\frac{\varphi : \mathsf{Formula}(\Gamma) \qquad \psi : \mathsf{Formula}(\Gamma)}{\varphi \wedge \psi : \mathsf{Formula}(\Gamma)}$$

Introduction:

$$\frac{\Delta \vdash_\Gamma \varphi \qquad \Delta \vdash_\Gamma \psi}{\Delta \vdash_\Gamma \varphi \wedge \psi}$$

Elimination:

$$\frac{\Delta \vdash_\Gamma \varphi \wedge \psi}{\Delta \vdash_\Gamma \varphi}$$

$$\frac{\Delta \vdash_\Gamma \varphi \wedge \psi}{\Delta \vdash_\Gamma \psi}$$

Substitution:

$$\frac{}{(\varphi \wedge \psi)[s] = \varphi[s] \wedge \psi[s]}$$

A.5 Disjunction

Formation:

$$\frac{\varphi : \mathsf{Formula}(\Gamma) \qquad \psi : \mathsf{Formula}(\Gamma)}{\varphi \vee \psi : \mathsf{Formula}(\Gamma)}$$

Introduction:

$$\frac{\Delta \vdash_\Gamma \varphi}{\Delta \vdash_\Gamma \varphi \vee \psi}$$

$$\frac{\Delta \vdash_\Gamma \psi}{\Delta \vdash_\Gamma \varphi \vee \psi}$$

Elimination:

$$\frac{\Delta \vdash_\Gamma \varphi \vee \psi \qquad \Delta \wedge \varphi \vdash_\Gamma \chi \qquad \Delta \wedge \psi \vdash_\Gamma \chi}{\Delta \vdash_\Gamma \psi}$$

Substitution:

$$\frac{}{(\varphi \vee \psi)[s] = \varphi[s] \vee \psi[s]}$$

A.6 Implication

Formation:

$$\frac{\varphi : \mathsf{Formula}(\Gamma) \qquad \psi : \mathsf{Formula}(\Gamma)}{\varphi \Rightarrow \psi : \mathsf{Formula}(\Gamma)}$$

Introduction:

$$\frac{\Delta \wedge \varphi \vdash_\Gamma \psi}{\Delta \vdash_\Gamma \varphi \Rightarrow \psi}$$

Elimination:

$$\frac{\Delta \vdash_\Gamma \varphi \qquad \Delta \vdash_\Gamma \varphi \Rightarrow \psi}{\Delta \vdash_\Gamma \psi}$$

Substitution:

$$\frac{}{(\varphi \Rightarrow \psi)[s] = \varphi[s] \Rightarrow \psi[s]}$$

A.7 Universal Quantification

Formation:

$$\frac{\varphi : \mathsf{Formula}(\Gamma \times A)}{\forall\varphi : \mathsf{Formula}(\Gamma)}$$

Introduction:

$$\frac{\Delta[\pi_1] \vdash_{\Gamma \times A} \varphi}{\Delta \vdash_\Gamma \forall\varphi}$$

Elimination:

$$\frac{\Delta \vdash_\Gamma \forall\varphi \qquad t : \mathsf{Tm}_\Gamma(A)}{\Delta \vdash_\Gamma \varphi[\langle \mathsf{id}, t\rangle]}$$

Substitution:

$$\frac{}{(\forall\varphi)[s] = \forall(\varphi[\langle \pi_1 \cdot s, \pi_2\rangle])}$$

A.8 Existential Quantification

Formation:

$$\frac{\varphi : \mathsf{Formula}(\Gamma \times A)}{\exists\varphi : \mathsf{Formula}(\Gamma)}$$

Introduction:

$$\frac{\Delta \vdash_\Gamma \varphi[\langle \mathsf{id}, t\rangle]}{\Delta \vdash_\Gamma \exists\varphi}$$

Elimination:

$$\frac{\Delta \vdash_\Gamma \exists\varphi \qquad \Delta[\pi_1] \wedge \varphi \vdash_\Gamma \psi[\pi_1]}{\Delta \vdash_\Gamma \psi}$$

Substitution:

$$\frac{}{(\exists\varphi)[s] = \exists(\varphi[\langle \pi_1 \cdot s, \pi_2\rangle])}$$

A.9 Equality

Formation:

$$\frac{t_1 : \mathsf{Tm}_\Gamma(A) \qquad t_2 : \mathsf{Tm}_\Gamma(A)}{t_1 \equiv t_2 : \mathsf{Formula}(\Gamma)}$$

Introduction:

$$\frac{t : \mathsf{Tm}_\Gamma(A)}{\Delta \vdash_\Gamma t \equiv t}$$

Elimination:

$$\frac{\Delta \vdash_\Gamma t_1 \equiv t_2 \qquad \Delta \vdash_\Gamma \varphi[\langle \mathsf{id}, t_1\rangle]}{\Delta \vdash_\Gamma \varphi[\langle \mathsf{id}, t_2\rangle]}$$

Substitution:

$$\frac{}{(t_1 \equiv t_2)[s] = (s \cdot t_1 \equiv s \cdot t_2)}$$